

SPARK · DELTA · DATABRICKS · FABRIC

The PySpark Practitioner's Handbook 4.0

How Spark thinks and runs on Databricks and Fabric.

Nearly a hundred figures like these: the mechanism behind every concept, drawn.

COMPANION

PySpark Cheat Sheet 4.0

221 code patterns to keep open while you code.

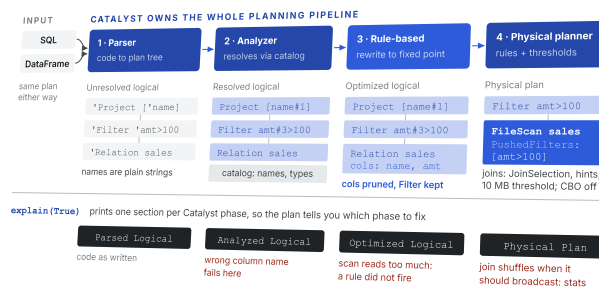
FREE

WEBSITE www.ssanjaychandra.com

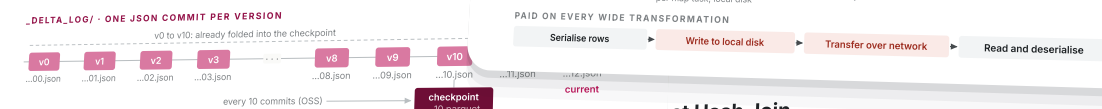
GITHUB github.com/ssanjaychandra123

LINKEDIN linkedin.com/in/ssanjaychandra

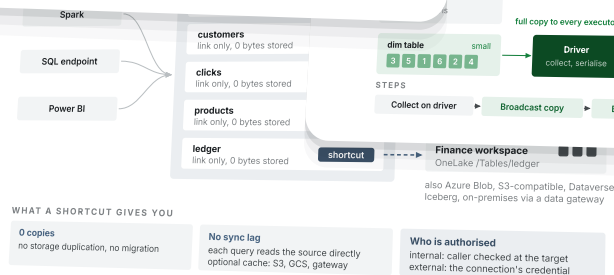
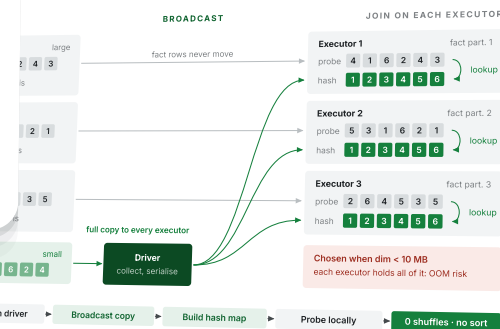
Catalyst Optimizer



Transaction Log



1st Hash Join



Sanjay Chandra

GLOBAL COE COMMUNITY

5
PARTS

13
CHAPTERS

97
CONCEPTS

The Periodic Table of PySpark.

01 Dec Driver, Executor, Cores, Slots	02 Csh Cluster sizing heuristics	03 Jst Job, Stage, Task	04 Sc Spark Connect	05 Le Lazy Evaluation	06 Nwt Narrow vs Wide Transformations	07 Sh Shuffle	08 Puv PySpark UDF variants	09 Amb ANSI mode by default	10 Vt VARIANT type	11 Co Catalyst Optimizer	12 Rbo Rule-Based Optimization	13 Pp Predicate Pushdown	14 Cp Column Pruning
15 Pr Partition Pruning	16 Dpp Dynamic Partition Pruning	17 Aqe Adaptive Query Execution (AQE)	18 Su The Spark UI	19 Reo Reading the explain() output	20 Ppj Physical Plan and Join Strategy	21 Bhj Broadcast Hash Join	22 Smj Sort-Merge Join	23 Shj Shuffle Hash Join	24 Wsc Whole-Stage Codegen	25 Tu Tungsten	26 Ph Photon	27 Nne NEE (Native Execution Engine)	28 Sp Spill
29 Dss Data Skew and Salting	30 Rcr Repartition, Coalesce, and Repartition-ByRange	31 Bjf Broadcast join failure modes	32 Ohm On-Heap Memory	33 Oh Off-Heap Memory	34 Um Unified Memory	35 Gp GC Pressure	36 Ca cache() and persist()	37 Bv Broadcast Variables	38 Dci Disk Cache and Intelligent Cache	39 Rds Reading data and schemas	40 Odp On-disk Partitioning	41 Sfp Small-file Problem	42 Obp OPTIMIZE and Bin-packing
43 Ds Data Skipping	44 Zo Z-Ordering	45 Vo V-Order	46 Lc Liquid Clustering	47 Dcs Delta compaction strategies	48 Po Predictive Optimization	49 At ACID Transactions	50 Tl Transaction Log	51 Tt Time Travel	52 See Schema Enforcement and Evolution	53 Wdc Writing data correctly	54 Mu MERGE / Upsert	55 Dv Deletion Vectors	56 Oc Optimistic Concurrency
57 Vr VACUUM and Retention	58 Cdf Change Data Feed	59 Twr Type widening and row tracking	60 Cmt Catalog-managed tables	61 Tco Table and catalog object model	62 Mbm Micro-batch Model	63 Tr Triggers	64 Ws Watermarks and State	65 Ome Output Modes and Exactly-once	66 Ck Streaming checkpoints	67 Sj Streaming joins	68 Al Auto Loader	69 Rtm Real-time mode	70 Tsd transform-WithState and State Data Source
71 Sdp Spark Declarative Pipelines	72 Au Autoscaling	73 Apj All-purpose vs Job Clusters	74 Ips Instance Pools and Spot VMs	75 Cpd Cluster Policies and DBR	76 Am Access modes	77 Uc Unity Catalog	78 Lpf Lakeflow pipelines (formerly DLT)	79 Mv Materialised Views	80 St Streaming Tables	81 Lj Lakeflow Jobs	82 On OneLake	83 Os OneLake Shortcuts	84 Dpa Delta-Parquet as Native Format
85 Frr Fabric Runtime 2.0 and resource profiles	86 Nsj Notebooks and Spark Job Definitions	87 Hcm High concurrency mode	88 Fe Fabric Environments	89 MLv Materialized lake views	90 Scp Starter vs Custom Pools	91 Cub Capacity Units and Bursting	92 Odb On-demand billing for Spark	93 Csc Classic vs serverless compute	94 Nud Notebook utilities: dbutils and notebook-utils	95 Oip Operating in production	96 Db Diagnose by Symptom	97 Wtw Which Tool When	

01 How Spark thinks 02 How Spark runs 03 Data at rest and in motion 04 Azure Databricks 05 Microsoft Fabric Back matter

What this handbook is.

PySpark on the two platforms most data teams build on today: **Microsoft Fabric** and **Azure Databricks**. It is for engineers and architects who run PySpark there, from a first notebook to choosing between Photon and the Native Execution Engine, or between Z-Order, V-Order and Liquid Clustering.

THIS HANDBOOK

Concepts, drawn

How Spark and Delta actually work.
One concept per entry, each with a figure that shows the mechanism.

COMPANION

PySpark Cheat Sheet 4.0

Code patterns for common tasks.
Read the concept here once, keep the cheat sheet open while you code.

SCOPE AND VERSION

Concepts reflect PySpark on **Spark 4.1** and **Delta Lake 4.x** as they stand on **Databricks Runtime 18** and **Fabric Runtime 2.0**, as of September 2026. Where behaviour differs on older runtimes, the relevant page notes it. APIs evolve. Where a feature is platform-specific (Photon for Databricks, V-Order for Fabric), the page marks it explicitly.

HOW TO READ THIS HANDBOOK

Sequential reading works for learners: Chapter I (Foundational runtime) introduces the actors; Chapter II (DataFrames and execution) explains how code becomes a plan; Chapter III (Catalyst and query plans) covers the optimiser that owns most of what follows and how to read what it produced. Beyond Chapter III, chapters can be read independently.

Reference reading works for practitioners: open the Contents, navigate by topic. Each concept entry is self-contained: a short lead under the title, a figure, then sections named after what you do with the concept, such as checking it ran or where it bites, and the related pages to read next.

CONVENTIONS

The book has **five parts**, each with its own colour. The tabs on the right edge of every page show which part you are in.



Part 1 · Blue
How Spark thinks



Part 2 · Green
How Spark runs



Part 3 · Fuchsia
Data at rest and in motion



Part 4 · Teal
Azure Databricks



Part 5 · Blue grey
Microsoft Fabric



Red
Cost, failure, path to avoid

In Parts 1 and 2, the eight-stage **lifecycle strip** under each concept title shows where the concept lives in a query: *Your Code* through Catalyst (Unresolved, Analysed, Optimised, Physical) to Codegen, DAG and Tasks, and Execute. From Part 3 on, a **layer strip** shows what the concept acts on: Files, Table, Stream, Compute, Govern, Pipeline, Serve, Operate. Filled segments mark the concept's span. In the **figures**, the part colour marks the concept and its good path, grey is surrounding context, and red flags cost, failure, or the path to avoid.

Where each chapter acts.

Query lifecycle [05](#) Concept index [06](#) Where each feature lives [94](#)
 Glossary [96](#) Conclusion [98](#)

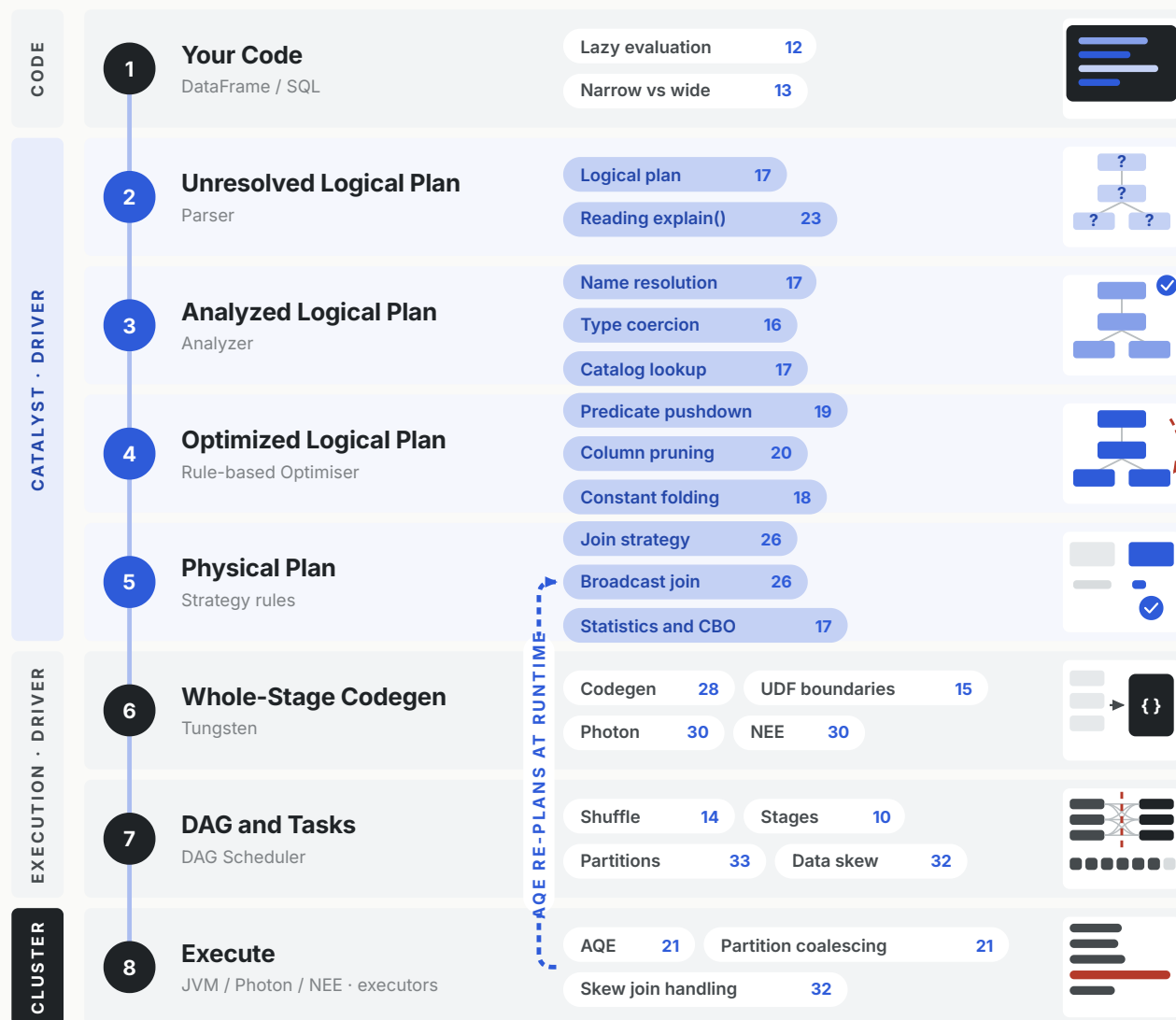
		QUERY STAGE · PARTS 1 AND 2	CODE	UNRESOLVED	ANALYSED	OPTIMISED	PHYSICAL	CODEGEN	DAG · TASKS	EXECUTE	CONCEPTS	PAGE
01 HOW SPARK THINKS												
I	Foundational runtime	1							1	3	4	09
II	DataFrames and execution	3		1				1	2	4	6	12
III	Catalyst and query plans		2	2	7	3	1	1	1	2	9	17
02 HOW SPARK RUNS												
IV	Physical execution						4	4		2	8	26
V	Shuffle, partitions, skew						1		1	3	4	31
VI	Memory and persistence	1								7	7	34
LAYER · PARTS 3 TO 5 AND BACK MATTER		FILES	TABLE	STREAM	COMPUTE	GOVERN	PIPELINE	SERVE	OPERATE			
03 DATA AT REST AND IN MOTION												
VII	Reading data and file layout	9	5						1	6	10	41
VIII	Delta Lake mechanics	4	13	1		2				1	13	48
IX	Structured Streaming	1	1	10	1		1				10	59
04 AZURE DATABRICKS												
X	Running PySpark on Databricks		1	2	5	3	4	1	1	4	10	68
05 MICROSOFT FABRIC												
XI	Running PySpark on Fabric	3	1		7	2	2	2	2	3	11	77
BACK MATTER												
XII	Compute choices, utilities and testing	1			1		1			3	3	87
XIII	Reference	2	2	1	2					1	2	90

Cell number = concepts in the chapter at that query stage (Parts 1 and 2) or layer (Parts 3 to 5 and back matter); darker = a larger share of the chapter.

The query lifecycle.

Every DataFrame or SQL query follows the same eight-step journey, and each stage has its own concept pages in this handbook.

FIG. 0.1 Stages 2 to 5 run inside Catalyst on the driver; stages 6 to 8 compile the chosen plan, split it into stages and tasks, and run them on the executors, while AQE feeds runtime statistics back into the physical plan. The small picture on each row shows what that stage produces; each chip gives the page of its concept.



Concept index: lifecycle and layer view.

PARTS 1 AND 2 · QUERY STAGE

1 Code 2 Unresolved 3 Analysed 4 Optimised 5 Physical 6 Codegen 7 DAG · Tasks 8 Execute

PARTS 3 TO 5, BACK MATTER · LAYER

A Files B Table C Stream D Compute E Govern F Pipeline G Serve H Operate

CONCEPT

I · FOUNDATIONAL RUNTIME

	1	2	3	4	5	6	7	8
Driver, Executor, Cores, Slots								
Cluster sizing heuristics								
Job, Stage, Task								
Spark Connect								

II · DATAFRAMES AND EXECUTION

	1	2	3	4	5	6	7	8
Lazy Evaluation								
Narrow vs Wide Transformations								
Shuffle								
PySpark UDF variants								
ANSI mode by default								
VARIANT type								

III · CATALYST AND QUERY PLANS

	1	2	3	4	5	6	7	8
Catalyst Optimizer								
Rule-Based Optimization								
Predicate Pushdown								
Column Pruning								
Partition Pruning								
Dynamic Partition Pruning								
Adaptive Query Execution (AQE)								
The Spark UI								
Reading the explain() output								

IV · PHYSICAL EXECUTION

	1	2	3	4	5	6	7	8
Physical Plan and Join Strategy								
Broadcast Hash Join								
Sort-Merge Join								
Shuffle Hash Join								
Whole-Stage Codegen								
Tungsten								
Photon								

CONCEPT

	1	2	3	4	5	6	7	8
NEE (Native Execution Engine)								

V · SHUFFLE, PARTITIONS, SKEW

	1	2	3	4	5	6	7	8
Spill								
Data Skew and Salting								
Repartition, Coalesce, and RepartitionByRange								
Broadcast join failure modes								

VI · MEMORY AND PERSISTENCE

	1	2	3	4	5	6	7	8
On-Heap Memory								
Off-Heap Memory								
Unified Memory								
GC Pressure								
cache() and persist()								
Broadcast Variables								
Disk Cache and Intelligent Cache								

VII · READING DATA AND FILE LAYOUT

	A	B	C	D	E	F	G	H
Reading data and schemas								
On-disk Partitioning								
Small-file Problem								
OPTIMIZE and Bin-packing								
Data Skipping								
Z-Ordering								
V-Order								
Liquid Clustering								
Delta compaction strategies								
Predictive Optimization								

VIII · DELTA LAKE MECHANICS

	A	B	C	D	E	F	G	H
ACID Transactions								
Transaction Log								
Time Travel								
Schema Enforcement and Evolution								

Concept index: lifecycle and layer view

CONTINUED

CONCEPT	A	B	C	D	E	F	G	H
Writing data correctly								
MERGE / Upsert								
Deletion Vectors								
Optimistic Concurrency								
VACUUM and Retention								
Change Data Feed								
Type widening and row tracking								
Catalog-managed tables								
Table and catalog object model								

IX · STRUCTURED STREAMING

	A	B	C	D	E	F	G	H
Micro-batch Model								
Triggers								
Watermarks and State								
Output Modes and Exactly-once								
Streaming checkpoints								
Streaming joins								
Auto Loader								
Real-time mode								
transformWithState and State Data Source								
Spark Declarative Pipelines								

X · RUNNING PYSPARK ON DATABRICKS

	A	B	C	D	E	F	G	H
Autoscaling								
All-purpose vs Job Clusters								
Instance Pools and Spot VMs								
Cluster Policies and DBR								
Access modes								
Unity Catalog								
Lakeflow pipelines (formerly DLT)								
Materialised Views								
Streaming Tables								
Lakeflow Jobs								

XI · RUNNING PYSPARK ON FABRIC

	A	B	C	D	E	F	G	H
OneLake								
OneLake Shortcuts								

CONCEPT	A	B	C	D	E	F	G	H
Delta-Parquet as Native Format								
Fabric Runtime 2.0 and resource profiles								
Notebooks and Spark Job Definitions								
High concurrency mode								
Fabric Environments								
Materialized lake views								
Starter vs Custom Pools								
Capacity Units and Bursting								
On-demand billing for Spark								

XII · COMPUTE CHOICES, UTILITIES AND TESTING

	A	B	C	D	E	F	G	H
Classic vs serverless compute								
Notebook utilities: dbutils and notebookutils								
Operating in production								

XIII · REFERENCE

	A	B	C	D	E	F	G	H
Diagnose by Symptom								
Which Tool When								

THINKS

RUNS

DATA

DATABRICKS

FABRIC

01

How Spark thinks.

Most tuning starts with the plan. The driver turns your DataFrame code into a logical plan, Catalyst rewrites it with rules and size estimates, and AQE adjusts it once real sizes come in. These chapters cover who does the work on the cluster, when your code actually runs, and how to read the plan Spark picked in `explain()` and the Spark UI. Read them first; the later chapters assume you can.

I	Foundational runtime		III	Catalyst and query plans	
01	Driver, Executor, Cores, Slots	09	11	Catalyst Optimizer	17
02	Cluster sizing heuristics	09	12	Rule-Based Optimization	18
03	Job, Stage, Task	10	13	Predicate Pushdown	19
04	Spark Connect	11	14	Column Pruning	20
II	DataFrames and execution		15	Partition Pruning	20
05	Lazy Evaluation	12	16	Dynamic Partition Pruning	21
06	Narrow vs Wide Transformations	13	17	Adaptive Query Execution (AQE)	21
07	Shuffle	14	18	The Spark UI	22
08	PySpark UDF variants	15	19	Reading the <code>explain()</code> output	23
09	ANSI mode by default	16			
10	VARIANT type	16			

READING PATH

Lifecycle heat: 1 2 3 4 5 6 7 8 = Code to Execute

I

Who does the work, and how is it divided up?



II

When does code actually run, and what does each call cost?



III

How does Spark rewrite my query, and where do I see what it did?



Driver, Executor, Cores, Slots.

QUERY LIFECYCLE CODE UNRESOLVED ANALYSED OPTIMISED PHYSICAL CODEGEN DAG · TASKS EXECUTE

Every Spark application has one **driver**, a JVM process running the application's `main()`, and many **executors**, JVM processes on worker nodes that run tasks. Each executor has a fixed number of **cores**, and each core runs one task at a time. Multiply executor count by cores per executor and you have your parallel **slots**.

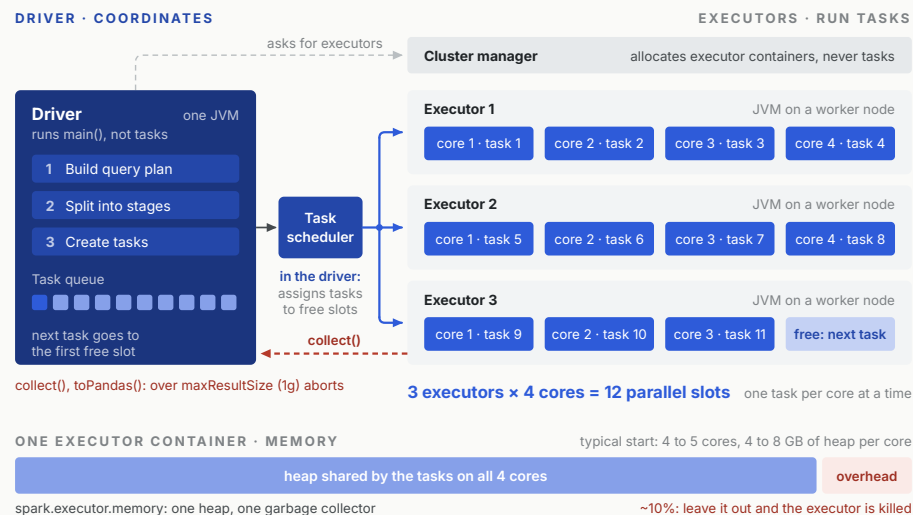


FIG. 1.1 One driver JVM plans the work and queues tasks; the cluster manager only allocates the executors, and the driver's task scheduler places each task in a free executor core, so parallel slots equal executors times cores per executor. All cores in an executor share one heap, and the container also needs room for overhead.

WHO DOES WHAT

The driver builds the query plan, splits it into stages and tasks, and asks the cluster manager for executors. The cluster manager only allocates executors; the driver's task scheduler puts each task on a free executor slot. A slot runs its task to completion, then picks up the next queued task. The driver does not process partitions. It coordinates, receives collected results and builds broadcast relations.

WHAT LANDS ON THE DRIVER

`collect()` and `toPandas()` bring data back to the driver. A result larger than `spark.driver.maxResultSize` (1g by default) aborts the job; one under that limit that still does not fit in the driver heap fails with an `OutOfMemoryError`. Driver memory only matters when you collect results or hold large broadcast variables. The other driver trap is running expensive Python loops there instead of pushing the work into DataFrame transformations, which execute on the executors.

SIZING EXECUTORS

More cores per executor gives you more parallel tasks, but they share one heap and one GC. Fewer cores gives each task more memory headroom and less parallelism per node. Start at 4 to 5 cores per executor, with executor memory sized at 4 to 8 GB per core. Budget for off-heap and JVM overhead: Spark reserves roughly 10% for overhead by default, so if you set `spark.executor.memory` to the full container size, the cluster manager kills the executor.

WATCH THE SLOTS FILL

Open the Executors tab in the Spark UI while a job runs. During busy stages, Active Tasks per executor should approach Cores per executor. If it stays well below, the stage has fewer tasks than slots and you are leaving parallelism on the floor. When that happens consistently, repartition upstream or raise `spark.sql.shuffle.partitions`.

DATABRICKS SERVERLESS

Not applicable. You cannot set executor count, cores or memory; Databricks sizes and scales the compute.

RELATED CONCEPTS

Cluster sizing heuristics translates these primitives into rules of thumb. Autoscaling adjusts executor count dynamically. GC Pressure is the failure mode when too many cores share too little heap.

Cluster sizing heuristics.

QUERY LIFECYCLE CODE UNRESOLVED ANALYSED OPTIMISED PHYSICAL CODEGEN DAG · TASKS EXECUTE

Pick executor count, executor memory and cores from a few rules of thumb, then adjust from what you observe: task durations, spill metrics and GC time. There is no formula; the numbers below are a starting point.

STARTING POINTS

4 to 5

cores per executor

4 to 8 GB

memory per core

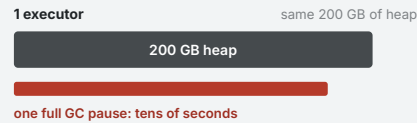
1 to 4 GB

input data per core

16 to 32 GB

driver memory

AVOID · ONE GIANT EXECUTOR



PREFER · MANY MEDIUM EXECUTORS



TUNE FROM THE STAGES TAB · MEDIAN TASK DURATION

< 1 second

too many partitions or cores
reduce `spark.sql.shuffle.partitions`

a few seconds

cluster is about right
keep it

> 30 seconds

partitions too large
more parallelism or executor memory

Heavy spill: more memory per core

High GC time: more executors, smaller heaps

Peak runs: size for the 95th percentile

FIG. 1.2 Sizing starts from rules of thumb, prefers many medium executors over one giant heap, and is then tuned from the median task duration and spill and GC metrics seen in the Stages tab.

STARTING NUMBERS

Cores per executor: 4 to 5. Go lower and JVM startup cost dominates; go higher and GC contention rises. **Memory per core:** 4 to 8 GB for typical analytical workloads. **Total cores:** aim for around 1 to 4 GB of input data per core for memory-bound work, and more data per core for CPU-bound work. **Driver size:** 16 to 32 GB, unless you collect large results.

MATCH IT TO THE WORKLOAD

ETL with wide tables and joins: more memory per core (8 GB+), moderate parallelism, AQE on. Aggregation-heavy SQL: more cores in total, a Photon-enabled instance, moderate memory per core. ML training: a large driver, fewer larger executors, GPU instances. Streaming: a small fixed cluster sized for steady-state throughput, plus headroom for catch-up.

TUNE FROM THE STAGES TAB

After a representative run, open the Stages tab in the Spark UI and look at task durations. If they cluster around a few seconds, the cluster is about right. A median task under 1 second means too many partitions or too many cores; reduce `spark.sql.shuffle.partitions`. A median over 30 seconds means the partitions are too large, so increase parallelism or executor memory. Heavy spill: add memory per core. High GC time: run more executors with smaller heaps.

WHERE SIZING GOES WRONG

One giant executor loses to many medium ones. A 200 GB heap GCs for tens of seconds; eight 25 GB heaps process the same data with no full-GC pause over a second or two. The other classic is sizing for the smallest dataset the pipeline ever sees, or for the median run, and watching month-end loads tip into spill, retries and OOMs. Size for the 95th percentile workload. Oversizing wastes money on idle slots, and too many small tasks inflate shuffle overhead. Undersizing shows up as executor OOMs, GC pauses over 10 seconds, or task retries that cascade into stage failures.

DATABRICKS SERVERLESS

Not applicable. There is no cluster to size; Databricks chooses and scales the compute.

Job, Stage, Task.

QUERY
LIFECYCLE

CODE

UNRESOLVED

ANALYSED

OPTIMISED

PHYSICAL

CODEGEN

DAG · TASKS

EXECUTE

Run an action (`write`, `count`, `collect`) and the scheduler submits a **job**; with AQE on, one action can submit several jobs. Spark expresses the job's physical execution plan as a Directed Acyclic Graph (DAG) of RDD operations and cuts it into **stages** at every shuffle boundary. Each stage is a set of **tasks**, one per input partition, all running the same code on different data, each on one CPU core on one executor.

WHERE STAGES COME FROM

This is the DAG and Tasks phase. The DAG Scheduler receives the physical plan after codegen and partitions it into stages; task scheduling then assigns tasks to executor cores to run in parallel. Wide transformations (`groupBy`, `join`, `distinct`) create shuffle boundaries. Everything between two boundaries is one stage and runs without network communication, because narrow transformations (`filter`, `map`, `select`) pipeline inside it. The number of tasks equals the number of partitions, and for post-shuffle stages `spark.sql.shuffle.partitions` controls that, defaulting to 200.

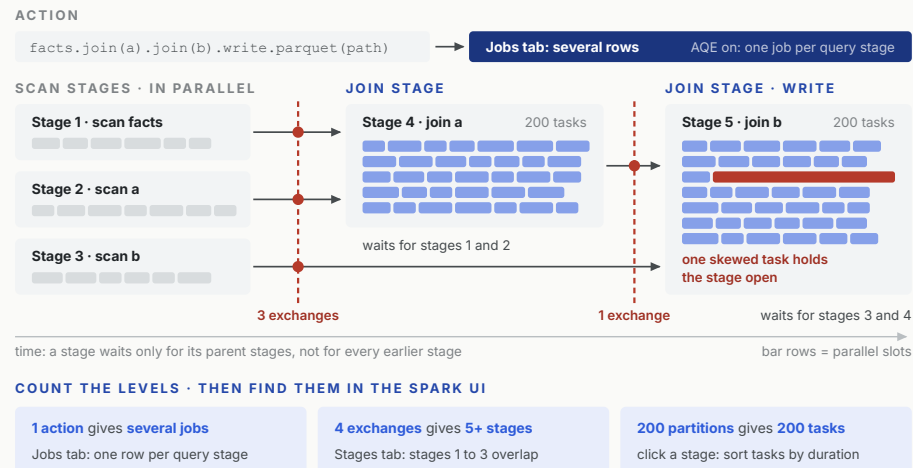


FIG. 1.3 One write action over two sort-merge joins: each input gets its own scan stage and the three scan stages run at the same time, then one stage per join, so at least 5 stages and 4 exchanges. A stage waits only for its parent stages, each shuffle stage runs one task per partition (200), and with AQE on each query stage is submitted as its own job.

COUNTING STAGES AND TASKS

Take `facts.join(a).join(b)` with sort-merge joins. Each input gets its own scan stage (facts, a, b), then there is one stage per join: at least 5 stages and 4 exchanges. A stage waits only for its parent stages, so the three scan stages run at the same time. A stage with M partitions has M tasks, and they run in parallel up to the cluster's slot capacity. With AQE on (the default), each query stage is submitted as its own job, so one action shows several rows in the Jobs tab. To keep jobs, stages and tasks apart, ask what runs in parallel. Tasks within a stage do. Stages within a job do when they do not depend on each other. Jobs within an application can overlap if the driver submits several actions concurrently from threads; most code submits one job at a time.

FINDING THE SLOW ONE

A slow job is a job with a slow stage, and a slow stage usually has one slow task (skew) or one slow shuffle. A high stage count points at unnecessary shuffles. In the Spark UI, the Jobs tab shows one row per job with its overall duration. Click a job to see its stages; each row has shuffle read/write sizes and skew indicators. Click a stage for its tasks: the task duration distribution exposes skew (one task much longer than the rest) and the spill column flags memory pressure. Most diagnoses start by sorting tasks by duration, descending. The SQL tab shows the DAG annotated with stage durations and shuffle sizes; that is where you spot stages that spill or hit Photon vs JVM fallback.

CHOOSING PARTITION COUNTS

More partitions mean more tasks and finer parallelism, and more scheduling overhead. Fewer partitions cut the overhead but can leave the cluster under-utilised or produce tasks too large for executor memory. Start at 128 MB to 256 MB of input data per task. Do not leave `spark.sql.shuffle.partitions` at the default of 200 for every workload: on a small dataset 200 partitions is excessive scheduling overhead, and on a large one the tasks get too big for memory. AQE coalesces small shuffle partitions automatically, but a sensible starting value still matters on paths where AQE is disabled.

ON DATABRICKS AND FABRIC

The model is identical on both. Databricks shows an enhanced DAG visualisation in the Spark UI. AQE changes the DAG at runtime: it coalesces small shuffle partitions, adds broadcast stages and splits skewed partitions. Expect the final DAG and stage count to differ from the pre-execution plan.

RELATED CONCEPTS

The Spark UI is the primary tool for navigating the hierarchy. Shuffle explains the boundaries that create stages. Adaptive Query Execution (AQE) rewrites the stage graph at runtime.

Spark Connect.

QUERY LIFECYCLE **CODE** UNRESOLVED ANALYSED OPTIMISED PHYSICAL CODEGEN DAG · TASKS EXECUTE

Spark Connect, introduced in Spark 3.4, splits the client from the driver. Your client code is a thin layer that sends an unresolved logical plan to a remote Spark cluster, and the cluster's driver handles all execution. **Databricks Connect** (versions 13+) is built on Spark Connect and is the modern way to use a remote Databricks cluster from local Python.

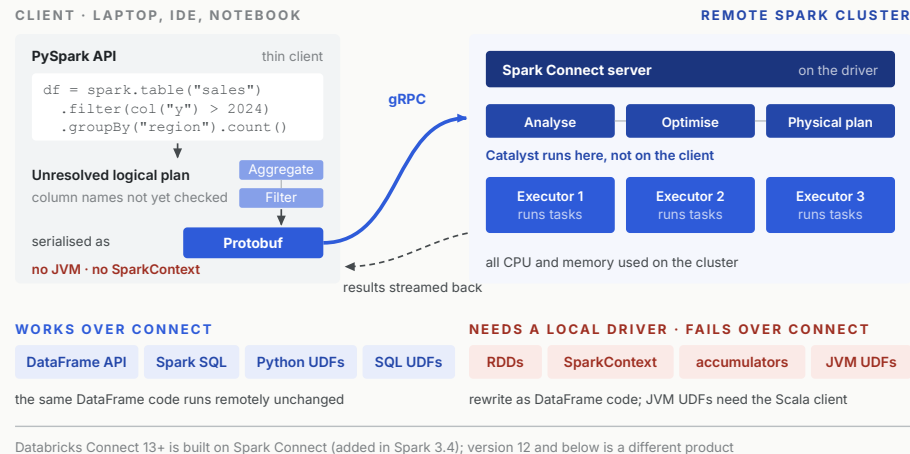


FIG. 1.4 With Spark Connect the client only builds an unresolved logical plan and sends it as Protobuf over gRPC; the remote driver runs Catalyst and the executors, then streams results back. DataFrame code works over the wire; RDD-era APIs that need a local SparkContext do not.

WHAT GOES OVER THE WIRE

You build a DataFrame with the PySpark API as usual. The client serialises the unresolved logical plan to Protobuf and sends it over gRPC to the Spark Connect server on the cluster. The server runs Catalyst, executes the plan and streams results back. The client never starts a JVM, never holds a `SparkContext`, and never executes user code on the driver.

CLIENTS AND TOOLS

PySpark and Scala are the first-class clients. Go, Swift and Rust clients are official Apache Spark subprojects; R and .NET clients are community projects. Notebook IDEs and local tools such as VS Code, PyCharm and Jupyter can drive a remote cluster without packaging logic into a JAR or running everything in a notebook. Your laptop's memory and CPU stay free because the work happens server-side.

WHAT BREAKS OVER CONNECT

It is not a drop-in replacement for local PySpark in every case. Not every Spark API is supported on the client. `SparkContext` is unavailable, so RDDs, accumulators and broadcast variables in their RDD-era forms cannot be used over Connect. Python and SQL UDFs work from the Python client, and Scala UDFs work from the Scala client (Spark 3.5 and above); what fails is defining a JVM UDF from the Python client. Lower-level operations such as file system access via `SparkContext.hadoopConfiguration` need a different approach. RDD APIs, SparkContext and certain UDF registration paths need the driver to be local, so they will not work over Spark Connect. Move that code to DataFrame APIs or run it on the server side.

ON DATABRICKS

`databricks-connect`, the PyPI package, replaces the older Databricks Connect for DBR 12 and below; from DBR 13 onward it runs on Spark Connect. Keep the two apart: v12 and below is a different product from v13+. Authenticate with a Databricks personal access token or OAuth. The remote cluster runs your code and your laptop only orchestrates, so most PySpark code works as long as it does not reach for SparkContext directly. Databricks Connect supports serverless compute (Databricks Connect 15.1 and above) and clusters in dedicated or standard access mode (DBR 13.3 LTS and above). Serverless connects faster; a cluster can be reused across sessions.

SPARK 4 CHANGES

Spark 4.x ships a lightweight `pyspark-client` package that contains only the Connect client, so a laptop or service no longer needs the full PySpark install to drive a remote cluster. Spark 4.1 adds a JDBC driver for Spark Connect. On Databricks, standard access mode compute runs on Spark Connect. Analysis and name resolution are deferred to execution time, so a misspelled column can surface at the action instead of at the transformation, and RDD APIs and `sparkContext` are not available. Use dedicated compute when a workload needs them.

Lazy Evaluation.

QUERY LIFECYCLE

CODE UNRESOLVED ANALYSED OPTIMISED PHYSICAL CODEGEN DAG · TASKS EXECUTE

Every Spark operation is a **transformation** or an **action**. Transformations (`select`, `filter`, `join`, `groupBy`) return a new DataFrame and extend the logical plan without executing anything. Actions (`count`, `collect`, `write`, `show`) submit the accumulated plan to Catalyst and trigger execution. Learn this split, and the deferred execution behind it, before anything else in Spark.

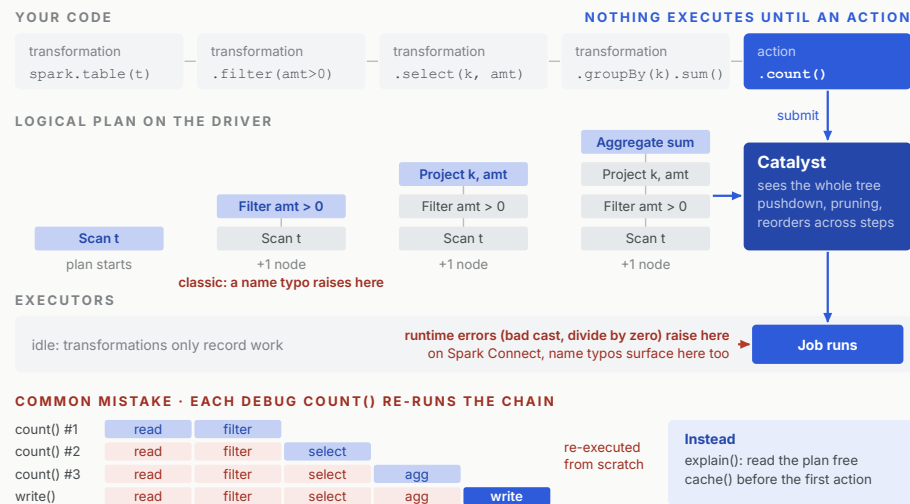


FIG. 2.1 Each transformation only adds a node to the logical plan on the driver; nothing runs until the action submits the whole tree to Catalyst. Every action then re-executes the full chain, so debug `count()` calls repeat work. Runtime errors surface at the action; a misspelled column raises at the transformation on classic compute and at the action on Spark Connect.

WHAT CATALYST GETS TO SEE

Each transformation returns a new DataFrame backed by a new logical plan node attached to the previous one, so a chain of them builds a tree of unexecuted operations. When an action runs, Spark hands the whole tree to Catalyst, which sees the full query and can reorder, combine and push down operations across step boundaries. Predicate pushdown, column pruning and join reordering all need that complete picture before execution begins. With eager transformations each step would run immediately in isolation, with no chance to optimise across them. Laziness is the architectural prerequisite for everything the optimiser does, far more than an implementation detail.

WHEN ERRORS SURFACE

Errors in transformations wait for an action. Runtime errors such as a bad cast or a division by zero surface only when `write()` or `count()` runs. Name errors depend on the client. On classic compute a misspelled column raises at the transformation, because PySpark analyses eagerly; on Spark Connect (standard access mode, serverless) it surfaces at the action. The further the action sits from the bad transformation, the harder the error is to find.

DEBUG COUNTS COST A RERUN

Calling `count()` mid-pipeline to look at intermediate data is expensive. Each call re-executes every transformation built so far, and the next action starts again from scratch. A pipeline with three debug `count()` calls can run the same transformations four times. Use `explain()` to inspect the plan at zero cost, or cache the DataFrame before the first action if several actions genuinely need the same intermediate result.

FORCING EXECUTION

When you do need execution at a specific point, use `df.count()`, `df.cache().count()` or `df.collect()`. For diagnosis only, `df.foreach(lambda _: None)` walks every row without bringing data back. Each of these runs a job, so use them sparingly in loops, where the cost compounds.

RELATED CONCEPTS

Catalyst Optimizer receives the plan built by lazy transformations. Narrow vs Wide Transformations distinguishes those that can pipeline within a stage from those forcing a shuffle. Job, Stage, Task is the physical form of the lazy chain once Catalyst compiles it.

Narrow vs Wide Transformations.

QUERY LIFECYCLE

CODE UNRESOLVED ANALYSED OPTIMISED PHYSICAL CODEGEN DAG · TASKS EXECUTE

Narrow transformations compute each output partition from exactly one input partition: `filter`, `select`, `withColumn`, `union`. **Wide** transformations need data from multiple partitions and force a shuffle: `groupBy`, `join` (sort-merge), `distinct`, `repartition`.

WHAT A WIDE ONE COSTS

Each wide transformation adds a stage boundary and a full shuffle cycle: serialise to disk, transfer over the network, deserialise on the other side. Narrow transformations pipeline within a stage with no network I/O.

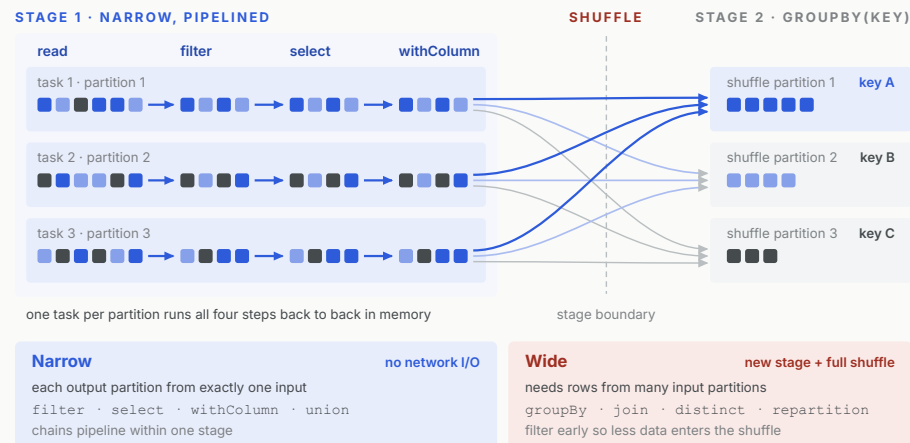


FIG. 2.2 Narrow steps (filter, select, withColumn) turn each partition into exactly one output partition, so one task runs them back to back inside a stage. The wide groupBy needs rows from every partition, which ends the stage and forces a shuffle.

SHRINKING THE SHUFFLE

Filter early and broadcast small tables so less data enters each shuffle. That is usually the biggest structural saving available.

RELATED CONCEPTS

Whole-Stage Codegen covers how narrow chains execute within a stage. Shuffle covers the write, transfer, and read cycle that wide transformations trigger.

Shuffle.

QUERY LIFECYCLE

CODE UNRESOLVED ANALYSED OPTIMISED PHYSICAL CODEGEN DAG · TASKS EXECUTE

Wide transformations such as groupBy, join and distinct force Spark to redistribute data across executors. Each executor writes intermediate results to disk, the data crosses the network, and the next stage reads it back. That makes shuffle the most expensive operation in Spark and the main source of stage boundaries.

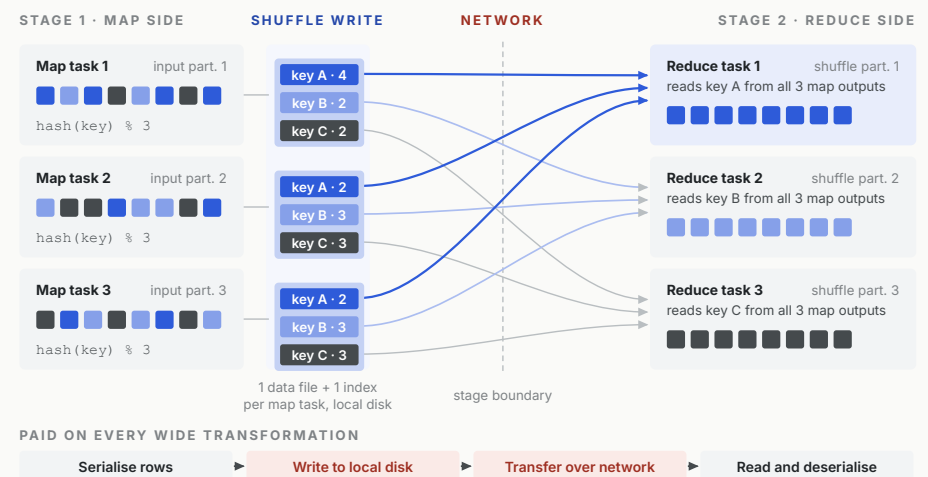


FIG. 2.3 Every map task hash-partitions its output, sorts it by target partition and writes one data file plus one index file to local disk; the index records where each downstream partition starts, and every reduce task pulls its key range from every map output across the network.

WRITE, THEN READ

In the shuffle write phase, each task partitions its output by a hash of the join or group key, sorts it by target partition, and writes one data file plus one index file per map task. The index records where each downstream partition starts. In the read phase, each downstream task reads its assigned partition from every upstream executor.

WHAT DRIVES THE COST

Total data moved is the product of output rows and record size, so cutting either cuts shuffle cost. Because a shuffle writes to disk and reads from disk across the network, it dominates runtime on large datasets.

CUTTING IT AT THE SOURCE

Use broadcast joins, pre-partitioned writes or bucketing, or repartition on the join key before a series of joins. Each of these reduces shuffle before it happens.

PySpark UDF variants.

QUERY LIFECYCLE CODE UNRESOLVED ANALYSED OPTIMISED PHYSICAL CODEGEN DAG · TASKS EXECUTE

Spark has several Python UDF APIs, and picking the right one changes Python UDF performance by 10x to 100x. The naive `@udf` is slowest, Pandas UDFs batch the JVM-Python boundary, and the Pandas Function API works on entire partitions. As a concrete case, a 100M-row column transformation often runs 10x slower with `@udf` than with a native Spark function, and Pandas UDFs typically close 70 to 90% of that gap.

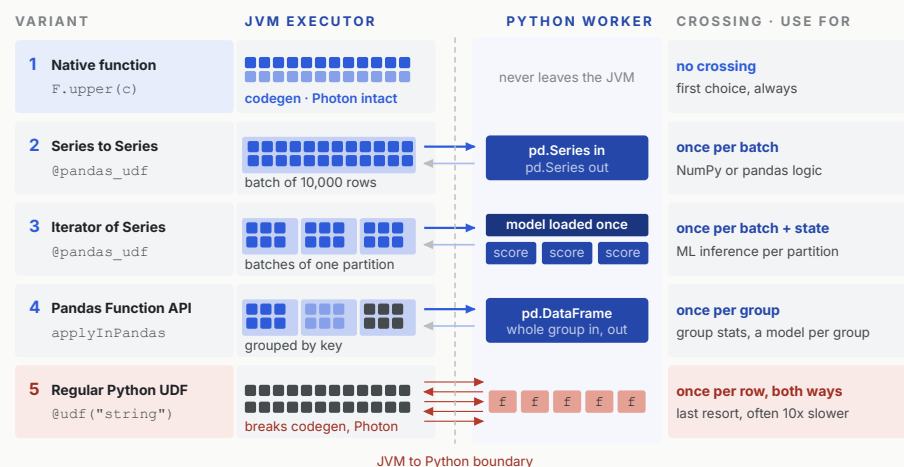


FIG. 2.4 The five ways to run logic on a column, in the order the decision rule suggests. The fewer times rows cross the JVM to Python boundary, the faster the variant: native functions never cross, Pandas UDFs cross once per batch, plain `@udf` crosses once per row unless `useArrow=True` moves its rows in Arrow batches, the default from Spark 4.2.

PLAIN @UDF

Put `@udf("string")` on a function and Spark serialises each row to Python, runs it and serialises the result back: one JVM-Python crossing per row. Whole-stage codegen splits at the UDF operator and Photon makes a row round trip out there; Fabric NEE now executes Python UDFs but gains little on scalar ones. The same logic in native Spark functions is almost always faster, so keep it for logic that truly cannot be vectorised. Passing `useArrow=True` (Spark 3.5 and above) keeps the function but moves rows in Arrow batches, cutting serialisation cost; Spark 4.2 turns this on by default. Spark 4.1 adds `@arrow_udf`, which hands the function PyArrow arrays with no pandas conversion.

PANDAS UDF, SERIES TO SERIES

Put `@pandas_udf("string")` on a function that takes a `pd.Series` and returns a `pd.Series`. Spark batches rows (default 10000 per batch), the function works on each whole Series in Python, and the result Series goes back to the JVM, so the boundary cost is amortised across the batch. It still breaks codegen, at much lower cost per row. Use it for NumPy or pandas-based logic.

PANDAS UDF, ITERATOR FORM

Same decorator, `@pandas_udf("string")`, but the function takes `Iterator[pd.Series]` and yields Series. That lets you keep state across batches in the same partition: load a model once, score many batches. Use it for ML inference, where loading the model per row is prohibitive.

WHOLE GROUPS AND PARTITIONS

`df.groupBy("key").applyInPandas(f, schema)` takes a function that receives a full pandas DataFrame per group and returns a DataFrame. `df.mapInPandas(f, schema)` processes partitions iteratively as DataFrames. Use these for arbitrary Python logic that needs the whole group or partition, such as custom group-level statistics or one scikit-learn model per group.

WHICH ONE TO REACH FOR

Native Spark functions first. Logic that needs Python: a Series-to-Series Pandas UDF. State across batches: Iterator-of-Series. Full groups or partitions: the Pandas Function API. Plain `@udf` is the last resort and should be rare in modern PySpark code.

ANSI mode by default.

QUERY LIFECYCLE

CODE UNRESOLVED ANALYSED OPTIMISED PHYSICAL CODEGEN DAG · TASKS EXECUTE

Since Spark 4.0, `spark.sql.ansi.enabled` is `true` by default. Spark SQL and DataFrame expressions follow ANSI SQL semantics, so an operation with no valid result raises an error instead of silently returning `NULL` or a wrapped value. Before 4.0 the default was `false`, the lenient Hive-compatible behaviour.

EXPRESSION	ANSI OFF Spark 3.5 default	ANSI ON · SPARK 4 DEFAULT raised inside the task, at the action	TOLERATE ONE COLUMN returns NULL again
<code>CAST('a' AS INT)</code>	NULL	<code>CAST_INVALID_INPUT</code>	<code>try_cast</code>
<code>2147483647 + 1</code>	<code>-2147483648</code>	<code>ARITHMETIC_OVERFLOW</code>	<code>try_add</code>
<code>10 / 0</code>	NULL	<code>DIVIDE_BY_ZERO</code>	<code>try_divide</code>
<code>element_at(arr, 10)</code>	NULL	<code>INVALID_ARRAY_INDEX_IN_ELEMENT_AT</code>	<code>try_element_at</code>

WHERE IT SURFACES · LAZY EXECUTION

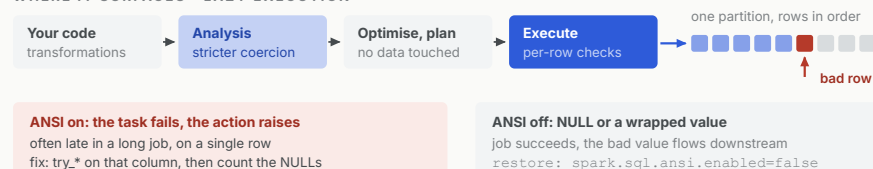


FIG. 2.5 The same four expressions under the old default and the Spark 4 default: with ANSI off a bad value quietly becomes NULL or wraps around, with ANSI on the task raises a named error at the action. Each `try_*` function restores NULL for one expression only.

WHAT NOW RAISES

Two layers change. At analysis, stricter type coercion rules apply, and `spark.sql.storeAssignmentPolicy=ANSI` (the default since Spark 3.0) rejects unsafe inserts such as `STRING` into an `INT` column. At execution, expressions check their input row by row. `CAST('a' AS INT)` fails with `CAST_INVALID_INPUT`, `2147483647 + 1` with `ARITHMETIC_OVERFLOW`, `x / 0` with `DIVIDE_BY_ZERO`, and `element_at(arr, 10)` on a 3-element array with `INVALID_ARRAY_INDEX_IN_ELEMENT_AT`. Invalid input to `to_date`, `to_timestamp` and `make_date` raises too. Because execution is lazy, you see the error at the action, often hours into a job, on the one bad row. In return, bad data no longer turns into `NULL` unnoticed and flows downstream into Delta tables.

GETTING NULL BACK ON PURPOSE

The `try_*` functions keep the old contract for one expression: `try_cast`, `try_add`, `try_subtract`, `try_multiply`, `try_divide`, `try_mod`, `try_sum`, `try_avg`, `try_element_at`, `try_to_timestamp` return `NULL` on bad input. When a migrated job fails, wrap the known-dirty column in `try_cast` or `try_divide` and count the resulting `NULL`s. Turning ANSI off globally to get the job through hides the bad rows again. Watch the test data too: a migration tested on a clean sample passes because no row triggers the error, then fails on full production data. Check the effective value with `spark.conf.get("spark.sql.ansi.enabled")` instead of assuming the runtime default.

RUNTIMES AND THE OFF SWITCH

Databricks Runtime 17.0 and above enable ANSI mode by default, matching Spark 4.0; Databricks SQL warehouses already defaulted `ANSI_MODE` to `TRUE` for accounts created from October 2022. Fabric Runtime 2.0 is built on Spark 4.1 and Microsoft points to the Apache Spark migration guide, so the Spark default applies; Runtime 1.3 (Spark 3.5) predates the change. Spark 4.1 also turns ANSI on for the pandas API on Spark and stores the ANSI setting with each view definition. To get the old behaviour back, set `spark.sql.ansi.enabled=false` or the `SPARK_ANSI_SQL_MODE=false` environment variable.

RELATED CONCEPTS

Lazy Evaluation, Catalyst Optimizer, Schema Enforcement and Evolution, VARIANT type, The Spark UI.

VARIANT type.

QUERY LIFECYCLE

CODE UNRESOLVED ANALYSED OPTIMISED PHYSICAL CODEGEN DAG · TASKS EXECUTE

Use `VARIANT` for semi-structured data such as JSON, where the schema is unknown or changes. Values are stored in an open binary encoding that can be navigated without re-parsing text. The functions arrived in Spark 4.0, and Spark 4.1 made `VARIANT` generally available and enabled by default. Databricks supports it from Runtime 15.3 (15.4 LTS for reading and writing Delta tables), and Fabric Runtime 2.0 lists it among its Spark 4.1 features.

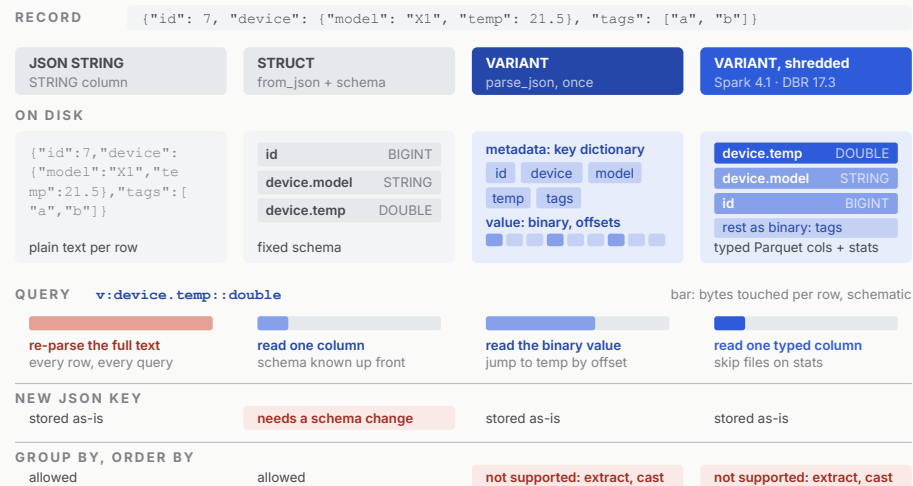


FIG. 2.6 One JSON record stored four ways. A string is re-parsed on every query and a STRUCT needs a fixed schema; VARIANT is parsed once into a binary value with a key dictionary, and shredding also writes common fields as typed Parquet columns, so a path query reads one column.

WRITING AND READING

Convert a JSON string once, at write time, with `parse_json(col)`; `try_parse_json` returns `NULL` for malformed input instead of failing. Read by path: `variant_get(v, '$.store.price', 'double')` returns `NULL` when the path is missing but raises if the cast fails, while `try_variant_get` returns `NULL` in both cases. Spark 4.1 added the colon syntax, `v.store.price::double`. `schema_of_variant_agg` reports the merged schema of a column, and `variant_explode` turns an object or array into rows. Path matching is case-sensitive, unlike the JSON string functions.

SHREDDING

Shredding writes commonly occurring fields as separate typed Parquet columns, so a query reads only the fields it asks for and can skip files on their statistics. Spark 4.1 infers the shredding schema in the Parquet writer. On Databricks, shredding is on by default for new tables from Runtime 17.3 (`delta.enableVariantShredding`); shred existing data with `REORG TABLE t APPLY (SHRED VARIANT)`. Runtime 18.1 adds variant statistics for data skipping.

PICKING THE COLUMN TYPE

A JSON `STRING` column is re-parsed on every query. A `STRUCT` is fastest but needs a fixed schema and a process to evolve it. `VARIANT` keeps the flexibility of the string with most of the read speed. Put stable, well-known fields in a `STRUCT` and the parts that drift in `VARIANT`.

LIMITS TO PLAN FOR

A Delta table with a `VARIANT` column carries the `variantType` table feature, which upgrades the writer protocol, so older external Delta clients may not write to it; reading and writing needs Runtime 15.4 LTS or above. A single value may be up to 128 MiB (16 MiB on Runtime 17.1 and below). You cannot use a `VARIANT` column in `GROUP BY`, `ORDER BY`, `DISTINCT`, set operators, partitioning or clustering keys; extract the field and cast it first. Under ANSI mode, `variant_get` on dirty data fails on one bad cast, so use `try_variant_get` when that is what you meant.

RELATED CONCEPTS

ANSI mode by default, Schema Enforcement and Evolution, Data Skipping, Column Pruning, Liquid Clustering, Auto Loader.

Catalyst Optimizer.

QUERY LIFECYCLE

CODE UNRESOLVED ANALYSED OPTIMISED PHYSICAL CODEGEN DAG · TASKS EXECUTE

SQL and the DataFrame API feed the same query optimiser, Catalyst, which is why equivalent queries produce identical plans however they were written. It owns every step of the planning pipeline, from the unresolved logical plan through to the physical plan.

THE FOUR PHASES

The **Parser** turns code into an unresolved logical plan, a tree where column names are still plain strings. The **Analyzer** resolves every column name and type against the catalog. A misspelled column fails here, before any job runs (on Spark Connect, analysis waits until the action). The **Rule-based optimiser** applies transformation rules repeatedly until the plan stops changing. Rules run in a fixed order inside ordered batches, and a batch repeats until it reaches a fixed point. Predicate pushdown, column pruning and constant folding all run here. The **Physical planner** turns the optimised plan into physical operators using strategy rules (such as `JoinSelection`) and size thresholds, and takes the first plan the rules produce.

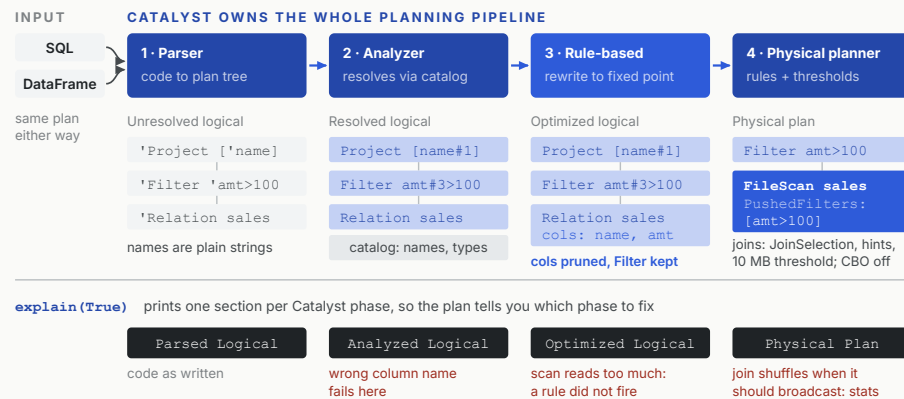


FIG. 3.1 SQL and DataFrame code enter the same Catalyst pipeline: the parser builds an unresolved plan, the analyser resolves names against the catalog, rules rewrite the plan to a fixed point, and the physical planner applies strategy rules (such as JoinSelection) with hints, size estimates and thresholds, keeping the first plan they produce. The Filter stays in the optimised logical plan; the pushed predicate appears only on the physical scan as PushedFilters. Each phase is one section of `explain(True)`.

WHICH PHASE TO BLAME

Before tuning anything, run `explain(True)`; its four sections map to Catalyst's phases. A wrong column name is an Analyzer problem. A scan reading more data than expected is an Optimized Logical Plan problem; a rule did not fire. A join shuffling when it should broadcast is a Physical Plan problem, usually a size estimate above the broadcast threshold. People file predicate pushdown, column pruning and join strategy under separate topics. They are all Catalyst rules or cost decisions, read from the same output. The pre-execution plan can also change, because AQE may revise the physical plan at runtime, so check the SQL tab in the Spark UI for the final plan.

STATISTICS AND CBO

The cost-based optimiser (`spark.sql.cbo.enabled`) and join reordering are off by default; turned on, they only add statistics-driven estimates. Table statistics (row counts, column cardinalities) feed size estimates, which decide, for example, whether a join side fits under the broadcast threshold. With `spark.sql.cbo.enabled` and join reordering on, they also reorder joins in the logical plan. The physical planner never costs alternative plans against each other. Collect statistics explicitly with `ANALYZE TABLE t COMPUTE STATISTICS FOR ALL COLUMNS`; without them Spark estimates sizes from file sizes. In practice, modern Databricks and Fabric pipelines rely on AQE to correct join planning at runtime and skip maintaining ANALYZE statistics. CBO matters most when AQE is disabled or the initial plan triggers an expensive cascading rewrite.

RELATED CONCEPTS

AQE replans at runtime where Catalyst cannot. Rule-Based Optimization is the phase where rules fire. Physical Plan and Join Strategy is the output of the physical planning phase.

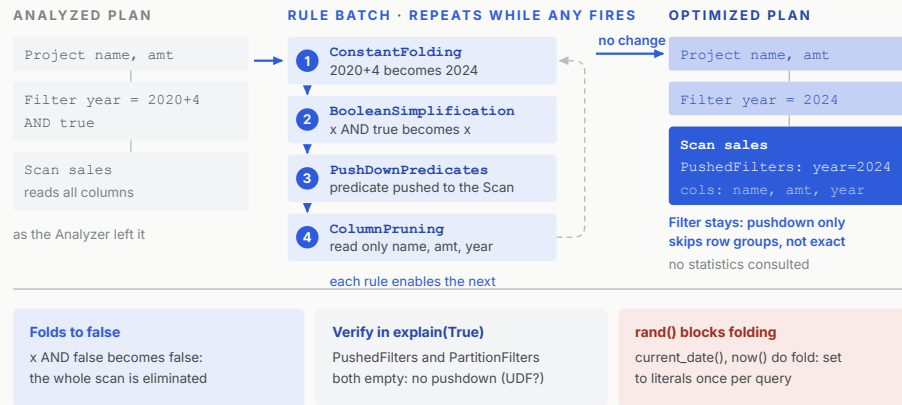
Rule-Based Optimization.



In the third phase of Catalyst, Spark applies a fixed set of rewrite rules to the analysed logical plan, over and over, until the plan stops changing. Each rule pattern-matches a plan subtree and replaces it with a cheaper equivalent. No statistics are consulted; the rules fire purely on plan structure.

HOW THE RULES RUN

Rules run in ordered batches. Each rule walks the plan tree top-down or bottom-up, as it needs, and if any rule fires, the batch runs again. The main ones: `PushDownPredicates` moves filters toward the scan, `ColumnPruning` removes unreferenced columns, `ConstantFolding` evaluates literal expressions at planning time and `BooleanSimplification` collapses constant predicates. They compose. Folding a constant may simplify a predicate, which enables pushdown, which shrinks the scan. Dozens more run at the optimised logical plan stage. `SimplifyConditionals` reduces always-true or always-false branches in CASE expressions, and `ConstantPropagation` substitutes literals through expressions. Each is a small, targeted transformation; together they cut data movement by orders of magnitude.



Rules are idempotent and side-effect-free, so the batch can run until the plan stops changing

FIG. 3.2 A batch of rewrite rules runs over the analysed plan and repeats while any rule fires: folding the constant simplifies the predicate, which lets the predicate be pushed to the scan and unused columns be pruned. For Parquet and Delta the Filter stays above the scan; the pushed predicate appears in PushedFilters. No statistics are consulted.

CONSTANT FOLDING

Anything whose value is known without reading data gets evaluated at planning time: `1+1` becomes `2`, `upper("hello")` becomes `"HELLO"`, `datediff("2024-12-31", "2024-01-01")` becomes `365`. Boolean simplification follows. `x AND true` collapses to `x`, and `x AND false` collapses to `false`. That last one pays off most. A filter that always resolves to false lets Catalyst eliminate the whole scan beneath it, so a query that would have scanned gigabytes returns empty at planning time.

DATES, TIMES AND RAND()

The optimiser rule `ComputeCurrentTime`, in the Finish Analysis batch, replaces `current_date()` and `now()` with literals once per query. They then fold and push down like any constant, and every reference in the query sees the same value. Truly non-deterministic functions such as `rand()` do block folding and pushdown. Precompute the value in Python and pass it in as a literal; that enables folding and any pushdown that depends on it.

CHECK THE PLAN

Run `explain(True)` and read the Optimized Logical Plan. Constants in the plan tree mean folding fired. For Parquet and Delta, a `Filter` on a data column stays above the `FileScan`, because pushdown only skips row groups and is not exact, so seeing it there proves nothing. A predicate on a partition column behaves differently: Spark drops the Filter, lists the predicate under `PartitionFilters` and leaves `PushedFilters` empty. Read both lists on the scan. When both are empty, pushdown did not fire, most often because of a UDF in the predicate.

Predicate Pushdown.

QUERY LIFECYCLE: CODE UNRESOLVED ANALYSED **OPTIMISED** PHYSICAL CODEGEN DAG · TASKS EXECUTE

Spark moves each filter as close to the data source as possible so it reads far less data. It is one optimiser rule inside Catalyst, not a separate system before or after it.



FIG. 3.3 The PushDownPredicates rule moves the filter down to the scan, and the physical scan lists the predicate under PushedFilters; the Filter node stays above the scan because row-group skipping is not exact. Delta then skips whole files using per-file stats in the transaction log, and Parquet skips row groups whose min/max cannot match.

WHERE ROWS GET SKIPPED

The `PushDownPredicates` rule moves Filter nodes toward the scan. Parquet then skips row groups whose min/max stats cannot match the predicate, and Delta eliminates whole files via per-file stats in the transaction log.

CHECK IT FIRED

To confirm the rule fired, look for `PushedFilters` on the `FileScan` line in `explain()`; a predicate on a partition column shows under `PartitionFilters` instead. A Python UDF in the predicate breaks it. The rule cannot inspect opaque functions, so the full table is scanned.

WHAT IT BUYS YOU

On the same query it is often the difference between scanning 2 TB and 20 GB. It works best with data clustered on the filter column via Z-Order, Liquid Clustering or partition layout.

Column Pruning.

Spark reads only the columns referenced anywhere in the query and skips the rest at the byte level. With columnar formats like Parquet, unreferenced columns are never decompressed or deserialised.

Spark reads only the columns referenced anywhere in the query and skips the rest at the byte level. With columnar formats like Parquet, unreferenced columns are never decompressed or deserialised.

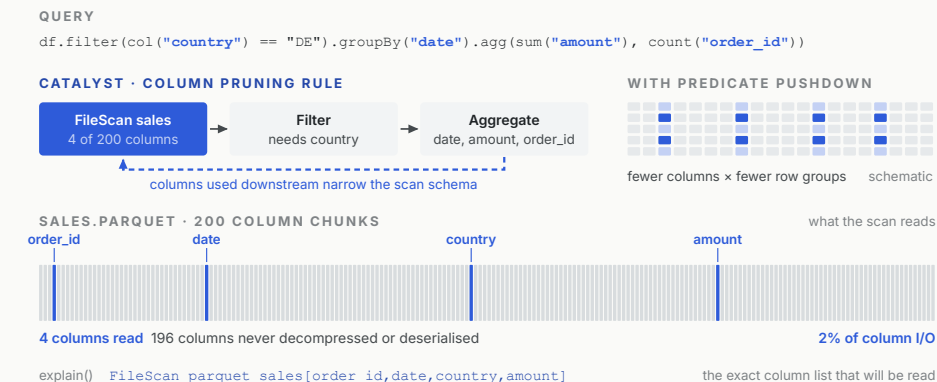


FIG. 3.4 The ColumnPruning rule traces which columns the query references and narrows the scan schema, so only 4 of the table's 200 Parquet column chunks are ever read, decompressed or deserialised.

CHECK THE SCAN

The `ColumnPruning` rule traces which columns are used downstream and narrows the schema passed to the scan node. Run `explain()`: the `FileScan` line shows the exact column list that will be read.

WHAT YOU SAVE

Query 4 columns of a 200-column table and you skip 196 columns of I/O automatically. With predicate pushdown on top, you read fewer rows as well as fewer columns.

RELATED CONCEPTS

Predicate Pushdown is the row-level sibling. Both are Catalyst rules at the Optimized Logical Plan phase.

Partition Pruning.

QUERY LIFECYCLE CODE UNRESOLVED ANALYSED **OPTIMISED** PHYSICAL CODEGEN DAG · TASKS EXECUTE

When a query filters on a partition column, a Catalyst rule removes whole directory paths from the scan at planning time. Only matching directory subtrees are listed and scanned; the rest are skipped before any file is opened.

WHAT GETS LISTED

Partition a table by `date`, query it with `WHERE date = '2024-01-01'`, and Catalyst resolves the filter to a partition path for the file system, which lists only `date=2024-01-01/`. On three years of daily partitions, a single-day query skips over a thousand partition directories entirely.

FILTERS THAT STILL PRUNE

The common mistake is filtering on a derived expression instead of the raw partition column, though not every derived filter fails. A deterministic function of the partition column itself, such as `WHERE year(date) = 2024` on a table partitioned by `date`, still prunes, because Spark evaluates it against each partition value.

WHERE IT BITES

What fails is filtering on a source column when the partition column is derived from it, for example filtering `event_ts` when the partition column is `event_date`. Filter on the partition column, or declare it as a Delta generated column (for example `CAST(event_ts AS DATE)`) so Delta derives the partition filter automatically.

PLANNING TIME · BEFORE ANY FILE IS OPENED

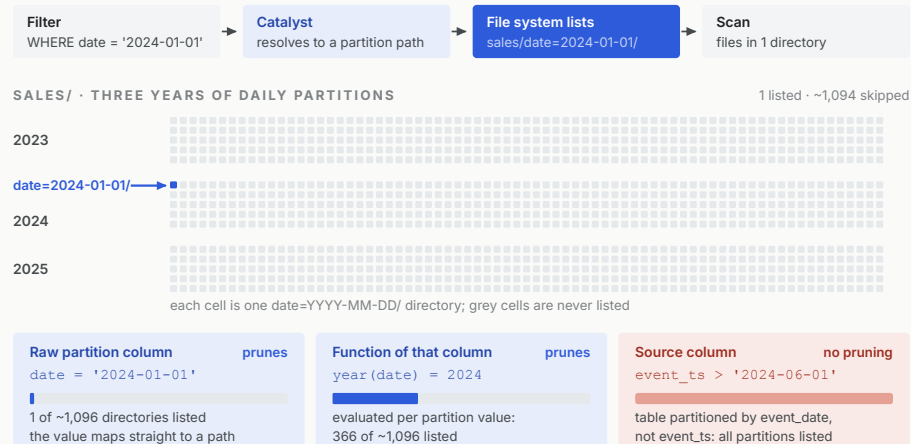


FIG. 3.5 At planning time Catalyst turns a filter on the raw partition column into a directory path, so only date=2024-01-01/ is listed and the other daily partitions are skipped before any file is opened. A function of the partition column still prunes; a filter on the source column of a derived partition column does not, unless the partition column is a Delta generated column.

Dynamic Partition Pruning.

QUERY LIFECYCLE: CODE UNRESOLVED ANALYSED **OPTIMISED** PHYSICAL CODEGEN DAG · TASKS EXECUTE

Partition pruning fires at planning time; DPP extends it to runtime. When the partition filter value comes from a join, Spark builds a runtime filter from the join's build side and pushes it to the probe-side scan.

IN A STAR SCHEMA

Take a star-schema join. Spark evaluates the dimension filter, builds a dynamic subquery filter (an in-list) from the matching dimension rows and pushes it to the fact-table scan to prune partitions.

WHAT YOU SAVE

Without DPP the whole fact table is scanned, whatever the dimension filter. With it, only the relevant partitions open. On large fact tables that is commonly 10x to 100x less I/O.

fact_sales JOIN dim_store USING (store_id) WHERE dim_store.region = 'EU'

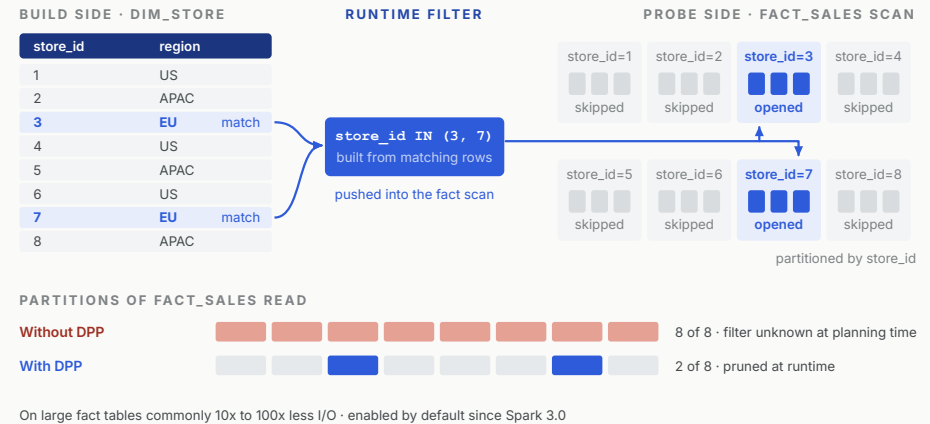


FIG. 3.6 At runtime Spark filters the dimension (build side), collects the matching keys into an in-list and pushes it to the fact-table scan (probe side), so only the partitions for those keys are opened.

DEFAULTS AND NEIGHBOURS

Enabled by default since Spark 3.0. Runtime Bloom Filter Join is a separate feature, for non-partition filtering.

Adaptive Query Execution (AQE).

QUERY LIFECYCLE: CODE UNRESOLVED ANALYSED OPTIMISED **PHYSICAL** CODEGEN DAG · TASKS EXECUTE

Catalyst builds its plan before any data is read and cannot revise it. AQE, the runtime re-optimisation framework, replans the query mid-flight, stage by stage, from the real shuffle sizes, row counts and skew patterns that execution produces.

WHERE IT STEPS IN

AQE acts during execution, at the shuffle boundaries between stages, since those are the first points where real output statistics exist. Within a stage it can do nothing. After each shuffle stage completes, Spark reads the real output statistics and applies three optimisations. **Coalescing shuffle partitions** merges tiny outputs into right-sized partitions. **Switching join strategies** turns a sort-merge join into a broadcast hash join when the build side measures small enough. **Skew join handling** splits oversized partitions and replicates the matching rows, so no single straggler holds up the stage.

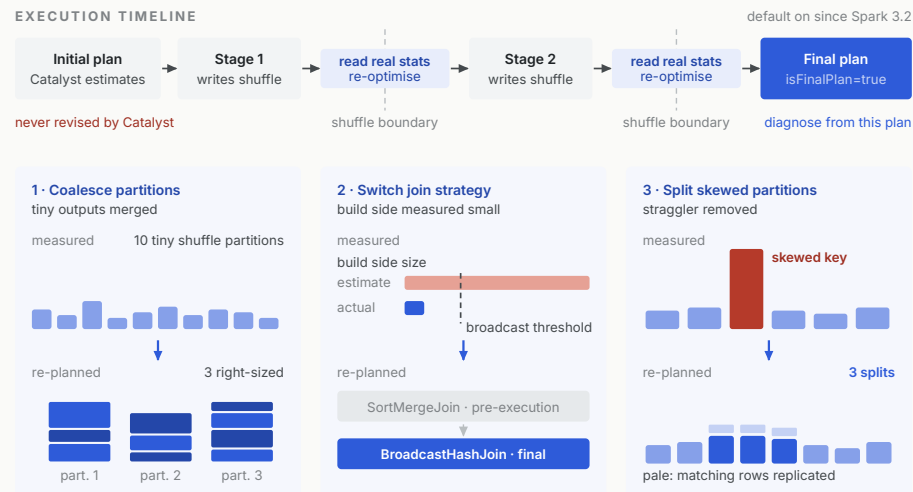


FIG. 3.7 Catalyst's plan is only an estimate; at every shuffle boundary AQE reads the real output statistics and re-optimises the rest of the query: coalescing tiny partitions, switching to a broadcast join, and splitting skewed partitions.

CHECK IT RAN

After the job completes, open the SQL tab in the Spark UI and look for `AdaptiveSparkPlan isFinalPlan=true`. A converted join shows `BroadcastHashJoin` in that final plan even though the pre-execution estimate said `SortMergeJoin`. The usual mistake is treating the plan from `explain()` as what will run. With AQE on it is always an estimate; the final plan, marked `isFinalPlan=true`, is what executed. Diagnose from that one.

WHAT IT COSTS YOU

AQE adds a small planning overhead at each shuffle boundary to collect and evaluate statistics, and on very short queries with few shuffles you can measure it against the runtime. In pathological cases it can pick a worse plan when mid-execution data is anomalous, or attempt a broadcast upgrade under tight executor memory. Both are uncommon and rarely a reason to disable AQE globally.

WHAT YOU STOP TUNING

Pipelines that used to need hand-tuned shuffle partitions, broadcast hints and skew salt keys now adapt on their own, and the gap between Catalyst's estimate and what actually ran shrinks without you intervening.

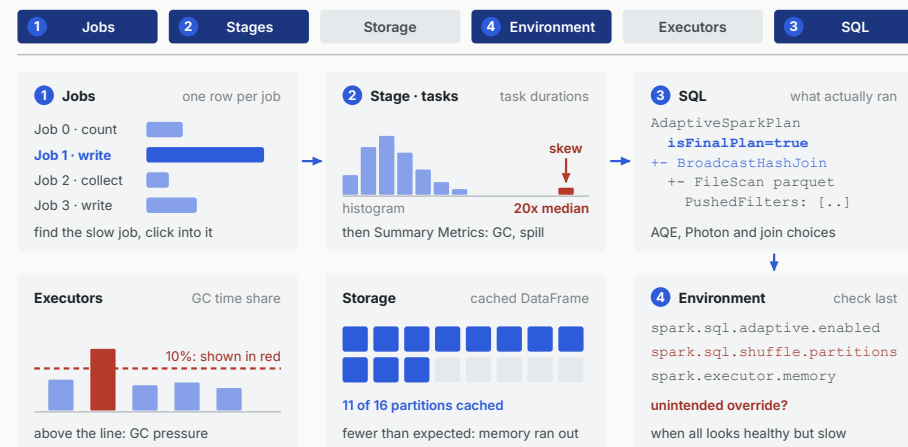
VERSION NOTES

On by default since Spark 3.2, and active on Databricks and Fabric with no configuration. Skew join handling is controlled by `spark.sql.adaptive.skewJoin.enabled`, which defaults to true.

The Spark UI.

QUERY LIFECYCLE CODE UNRESOLVED ANALYSED OPTIMISED PHYSICAL CODEGEN DAG · TASKS EXECUTE

Every running Spark application gets its own web UI with execution metrics: jobs, stages, tasks, SQL queries, executor health, storage and environment. It persists only while the application runs; for completed jobs use the Spark History Server, which Databricks and Fabric host automatically.



The live UI exists only while the application runs (port 4040 on open-source Spark); finished runs: Spark History Server

FIG. 3.8 The Spark UI tabs and the order to read them: find the slow job, drill into its stage and task histogram, confirm the final plan in the SQL tab, and check Environment last; Executors and Storage show cluster health at any point.

OPENING IT

On Databricks classic compute, open it from the compute page or a cell's job link; on Fabric, from the notebook or the Monitoring hub. On open-source Spark it runs on port 4040. Databricks serverless has no Spark UI, so use the query profile there.

JOBS AND STAGES

Jobs: one row per job, with duration, stages completed and status; with AQE on, one action can show several rows. **Stages:** click a job to see its stages. Per stage, the useful views are the task-duration histogram, where skew shows as a long tail, and the Summary Metrics table: median, 75th and max task duration, GC time, shuffle read/write sizes and spill metrics. A task running 20x the median is a skewed partition. Sort tasks by duration descending to surface stragglers first.

SQL TAB

For DataFrame and SQL workloads, this is where you see what actually executed. Each query appears with its physical plan; `AdaptiveSparkPlan isFinalPlan=true` confirms AQE has finalised it after runtime adjustments, and Photon-supported operators are annotated as such. Check here whether a join converted from sort-merge to broadcast at runtime. `PartitionFilters` and `PushedFilters` on the FileScan node confirm partition and predicate pruning fired.

EXECUTORS AND STORAGE

Executors: per-executor JVM heap usage, GC time, task counts and shuffle volume. High GC time signals GC pressure; the tab turns it red once it passes 10% of task time, and over 30% is serious. A sudden drop in executor count mid-run signals OOM kills or spot interruption, not a planned scale-down. **Storage:** cached RDDs and DataFrames with size, partition count and eviction status. If a cached DataFrame shows fewer partitions cached than expected, executor memory ran out and the rest was either evicted or never stored.

WHERE TO START

Start at Jobs to find the slow job, click into it for the stage, then click the stage for the task histogram (skew check) and Summary Metrics (memory and shuffle health). If the problem is plan-related, go to the SQL tab and read the final plan; Photon, AQE and join strategy decisions all appear there. If everything looks healthy but the job is still slow, check the Environment tab for unintended config overrides.

Reading the explain() output.

QUERY LIFECYCLE

CODE UNRESOLVED ANALYSED OPTIMISED PHYSICAL CODEGEN DAG · TASKS EXECUTE

`df.explain(True)` prints the query's Catalyst plan as text in four sections: parsed logical plan, analysed logical plan, optimised logical plan and physical plan. Reading it is the most useful diagnostic skill you can have for Spark performance.

DF.EXPLAIN(TRUE) · FOUR SECTIONS, TOP TO BOTTOM

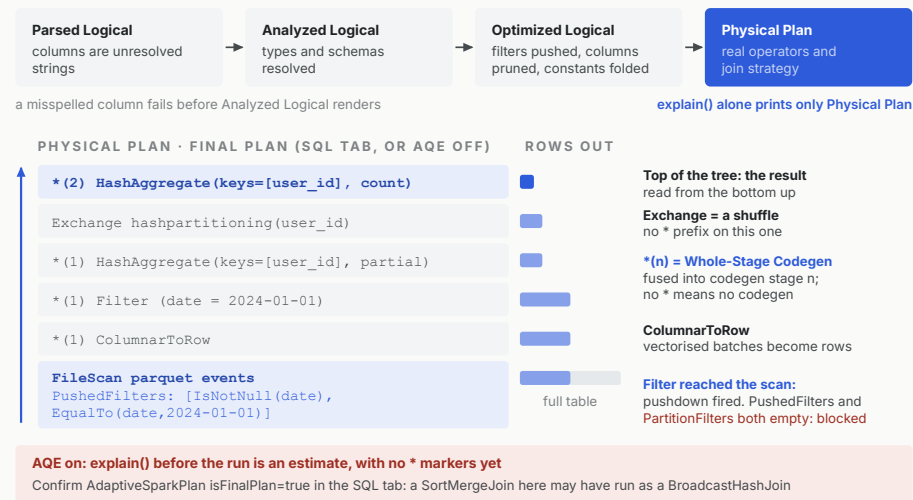


FIG. 3.9 `explain(True)` prints four plans in the order Catalyst builds them; the physical plan is read bottom to top, checking PushedFilters on the scan, the *(n) codegen prefix, and where each operator shrinks the data. The plan shown is the final one: with AQE on, the plan printed before the run has no * markers.

THE FOUR SECTIONS

Parsed Logical Plan: column references are still unresolved strings; useful only to confirm the query parsed. **Analyzed Logical Plan:** column types and table schemas are resolved. A misspelled column fails before this section ever renders. **Optimized Logical Plan:** Catalyst rules have fired. Look for Filter nodes pushed below Project, columns pruned from the scan schema and folded constants. **Physical Plan:** the actual operators and join strategies the planner's rules and size thresholds chose. Read it bottom up: the scan sits at the bottom, and each operator consumes the output of the one below.

LINES TO FIND

On the physical plan's FileScan line, `PushedFilters: [IsNotNull(amount), GreaterThan(amount,100)]` means predicate pushdown fired on a data column. A predicate on a partition column appears under `PartitionFilters` instead, with `PushedFilters: []` and no Filter above the scan; that is pruning working, not failing. When both lists are empty, a UDF or expression blocked it. Check that the scan's column list holds only the columns you need. Join nodes name their strategy: `BroadcastHashJoin`, `SortMergeJoin`, `ShuffleHashJoin`. A `*(n)` prefix marks operators Whole-Stage Codegen fused into codegen stage `n`; an operator without it runs without codegen.

WITH AQE ON

AQE is on by default, and then the pre-execution `explain()` is an estimate that AQE can revise at runtime. After the query runs, check the final plan in the SQL tab of the Spark UI for `AdaptiveSparkPlan isFinalPlan=true`. A `SortMergeJoin` in `explain()` may have executed as `BroadcastHashJoin` because AQE upgraded it based on the actual build-side size. The pre-run plan also shows no `*` at all, because codegen stages are added per query stage at runtime; look for the markers in the final plan in the SQL tab, or with AQE off.

PICKING A VARIANT

`explain()` with no argument shows only the physical plan; `explain(True)` shows all four sections. `explain("formatted")` prints a short operator outline, then a details block per numbered node (inputs, pushed filters, join keys), with no runtime metrics. `explain("cost")` prints the optimised logical plan with each node's statistics: `sizeInBytes` always, `rowCount` only when CBO or ANALYZE statistics exist. Use it to work out why a join went sort-merge when you expected broadcast.

A WORKED EXAMPLE

For `df = spark.read.parquet('events').filter(col('date') == '2024-01-01').groupBy('user_id').count()`, the bottom of `explain()` shows a FileScan with `PushedFilters: [IsNotNull(date), EqualTo(date,2024-01-01)]`, so the filter reached the scan. Had `events` been partitioned by `date`, the predicate would appear under `PartitionFilters` instead, with no Filter node above the scan. Above it, a HashAggregate handles the groupBy. Read bottom to top and you watch the data shrink at each operator and can pinpoint where the largest reductions happen.

02

How Spark runs.

This part follows the plan onto the cluster, where the cost is paid: the join Spark picked, the engine running each operator (codegen, Photon, NEE), the shuffles between stages, the skewed key that keeps one task running long after the rest, and how each executor splits its memory between caching and execution.

IV Physical execution

20	Physical Plan and Join Strategy	26
21	Broadcast Hash Join	26
22	Sort-Merge Join	27
23	Shuffle Hash Join	28
24	Whole-Stage Codegen	28
25	Tungsten	29
26	Photon	30
27	NEE (Native Execution Engine)	30
V Shuffle, partitions, skew		
28	Spill	31
29	Data Skew and Salting	32

V Shuffle, partitions, skew cont.

30	Repartition, Coalesce, and RepartitionByRange	33
31	Broadcast join failure modes	34
VI Memory and persistence		
32	On-Heap Memory	34
33	Off-Heap Memory	35
34	Unified Memory	35
35	GC Pressure	36
36	cache() and persist()	37
37	Broadcast Variables	38
38	Disk Cache and Intelligent Cache	38

READING PATH

Lifecycle heat: 1 2 3 4 5 6 7 8 = Code to Execute

IV

Which join and which engine execute the plan?



V

Where does data move between machines, and why is one task slow?



VI

How is executor memory split, reclaimed and reused?



Physical Plan and Join Strategy.

QUERY LIFECYCLE CODE UNRESOLVED ANALYSED OPTIMISED **PHYSICAL** CODEGEN DAG · TASKS EXECUTE

The physical plan is what Catalyst actually hands to the cluster: which concrete operators run, which join algorithm applies and in what order data is scanned. The logical plan says what to compute; the physical plan says how to compute it on the cluster you have.

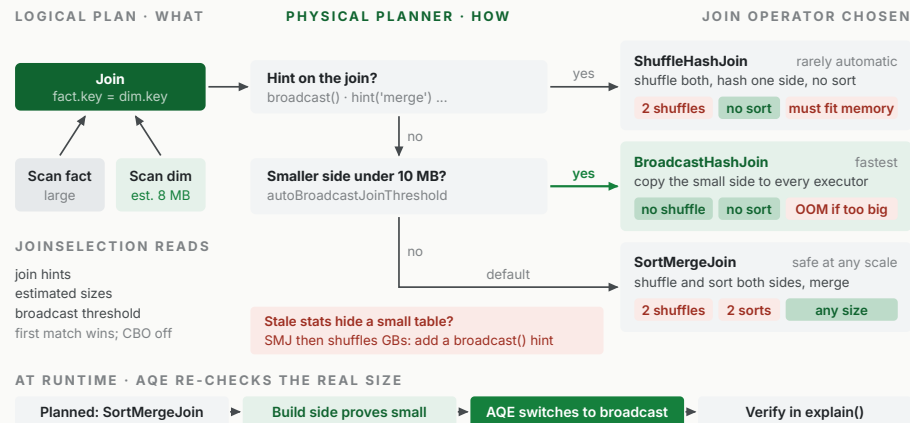


FIG. 4.1 The physical planner turns the logical Join into one concrete operator: a hint wins if present, otherwise the JoinSelection rule compares the estimated smaller side with the 10 MB broadcast threshold and the first matching strategy wins (the CBO, `spark.sql.cbo.enabled`, is off by default), and AQE may still switch the choice at runtime.

HOW SPARK CHOOSES

This happens in the physical planning phase. Catalyst applies strategy rules to the optimised logical plan, maps each logical operator to a concrete implementation, and keeps the first physical plan the rules produce. The `JoinSelection` rule picks the join from hints, estimated sizes and configuration thresholds, including the `autoBroadcastJoinThreshold` setting (default 10 MB); the cost-based optimiser (`spark.sql.cbo.enabled`) is off by default. It weighs three strategies. **Broadcast hash join** sends the smaller side to every executor and avoids a shuffle entirely. **Sort-merge join** shuffles and sorts both sides on the join key and is the default for large tables. **Shuffle hash join** shuffles both sides and builds a hash table without sorting.

OVERRIDING IT

When the planner picks wrong, override it with a hint. `broadcast(df)` or `.hint("broadcast")` forces a broadcast join; `.hint("merge")` forces sort-merge. Check the final strategy in `explain()` before you assume the hint took. The usual miss is a dimension table that is clearly small enough but not broadcast. Spark's size estimate can be wrong when statistics are stale, and the threshold check happens before execution, so a correct hint is the reliable fix.

WHAT A WRONG PICK COSTS

Join strategy is often the largest single driver of query performance, and the wrong strategy on a large table can be the biggest problem in the whole pipeline. A sort-merge join on a 10 MB dimension table shuffles gigabytes for nothing. Go the other way and a broadcast fails with OOM once the broadcast table exceeds executor memory; sort-merge is safe at any scale but costs two full shuffles. Read the physical plan. Running the query and hoping will not tell you which one you got.

SETTINGS AND VERSIONS

`spark.sql.autoBroadcastJoinThreshold` defaults to 10 MB; many Databricks workloads benefit from raising it to 30 MB or higher. AQE can switch join strategy at runtime when the actual build-side size differs from the planner's estimate.

Broadcast Hash Join.

QUERY LIFECYCLE CODE UNRESOLVED ANALYSED OPTIMISED **PHYSICAL** CODEGEN DAG · TASKS EXECUTE

Spark serialises the smaller table on the driver, broadcasts a copy to every executor and builds a hash table from it in executor memory. Each executor then probes that local hash table for every row of its own partition of the larger table. Neither side is shuffled.

WHEN SPARK PICKS IT

Catalyst picks it when the estimated size of the smaller side is under `spark.sql.autoBroadcastJoinThreshold` (default 10 MB). The driver turns the small table into a byte array and broadcasts it with a BitTorrent-style protocol; each executor deserialises it into a hash map keyed on the join column. The large table is then scanned locally, partition by partition, and every row is looked up in that map. The large side never crosses the network and nothing is sorted, which makes it the fastest join whenever it is valid: it removes the two full shuffles of sort-merge join and the two shuffles of shuffle hash join. If a pipeline keeps joining a large fact table to a small dimension table, making sure the dimension is broadcast is often the biggest win you have.

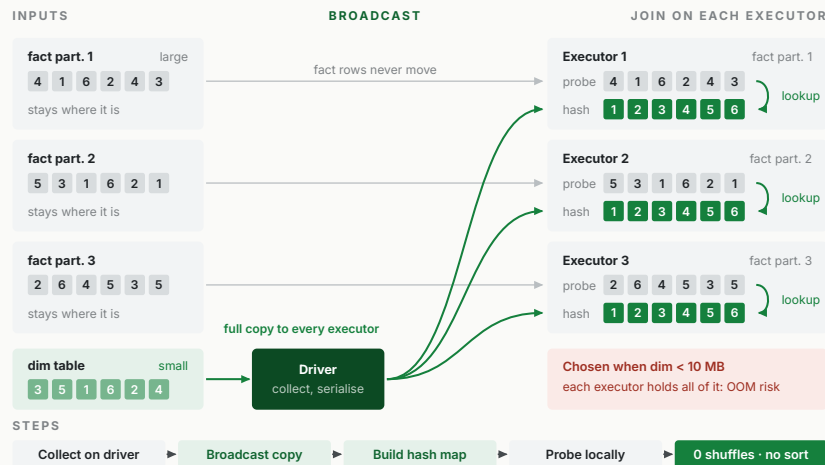


FIG. 4.2 The driver collects the small dim table and broadcasts a full copy to every executor, where it becomes an in-memory hash map; each fact partition probes that map where it already sits, so neither side is shuffled or sorted.

FORCING IT

When stale statistics make Catalyst overestimate the smaller table's size, force the broadcast with `df.join(broadcast(dim), 'key')` or `df.join(dim.hint('broadcast'), 'key')`. Then run `explain()` and check that `BroadcastHashJoin` is in the physical plan. If the broadcast fails with OOM, the table is too large; go back to sort-merge.

WHERE IT BITES

Broadcast a table that is too big for executor memory and every executor hits OOM at the same moment. The safe ceiling depends on how executor memory is configured. As a rule of thumb, prefer sort-merge above about 1 GB, and broadcast up to 2 GB only with very large executor heaps. AQE can also switch a planned sort-merge join to broadcast at runtime if the actual build-side size turns out smaller than estimated.

CHECK IT RAN

In `explain()` or the SQL tab, the join operator should read `BroadcastHashJoin`, with the small table as the build side. If you see `SortMergeJoin` there, the planner rejected the broadcast: either the smaller side was over the threshold, or AQE downgraded the choice based on runtime statistics.

RELATED CONCEPTS

Sort-Merge Join is the safe fallback for large tables. AQE can upgrade a sort-merge to broadcast at runtime. Broadcast Variables are the underlying mechanism used to distribute the hash table.

Sort-Merge Join.

QUERY LIFECYCLE CODE UNRESOLVED ANALYSED OPTIMISED **PHYSICAL** CODEGEN DAG · TASKS EXECUTE

Both sides are shuffled by a hash of the join key so matching rows land on the same executor, each side is sorted on the key, and the sorted streams are merged. It is the default join for large tables, safe at any scale, and it costs two full shuffles and two sorts.

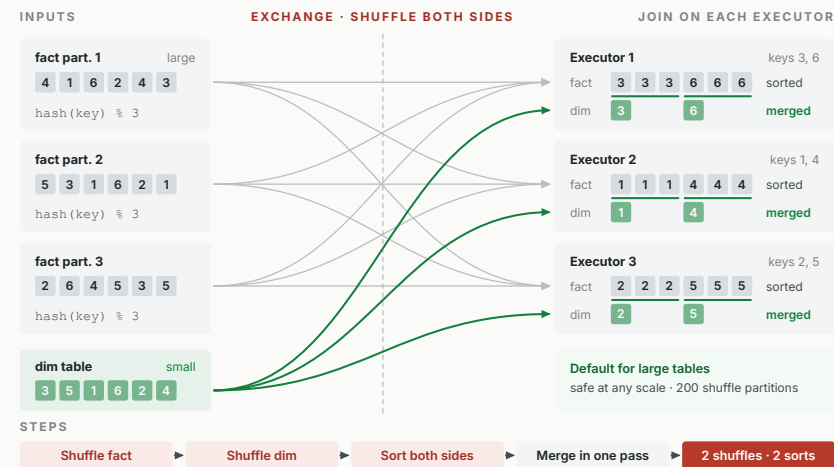


FIG. 4.3 Both sides are hash-partitioned on the join key so matching keys meet on one executor, then each side is sorted and the two sorted streams are merged in one pass: safe at any size, but it pays two shuffles and two sorts.

STAGE BY STAGE

Stage 1 partitions both sides by `hash(key) % numPartitions` into shuffle files. In Stage 2 each executor reads its partition of both sides, sorts each independently and merge-joins the sorted streams, which is $O(n)$ once sorted. `spark.sql.shuffle.partitions` (default 200) sets the partition count.

WHAT IT COSTS YOU

All that network and disk I/O makes it expensive, but on large tables where a broadcast would OOM it is the only safe choice. AQE can upgrade it to broadcast at runtime if the build side proves small enough.

JOINS WITH NO EQUALITY

For non-equi-joins (cross, inequality predicates), Spark uses **Broadcast Nested Loop Join** when one side is small enough to broadcast, otherwise a cartesian product for inner joins, with broadcast nested loop as the last resort for other join types. All are $O(n*m)$ and usually accidental; add an equi-join condition.

Shuffle Hash Join.

Both sides are shuffled by the join key, then Spark builds a hash table from the smaller side and probes it with the larger one. Unlike sort-merge, there is no sort. Unlike broadcast hash, both sides go through a shuffle instead of one side being sent to every executor.

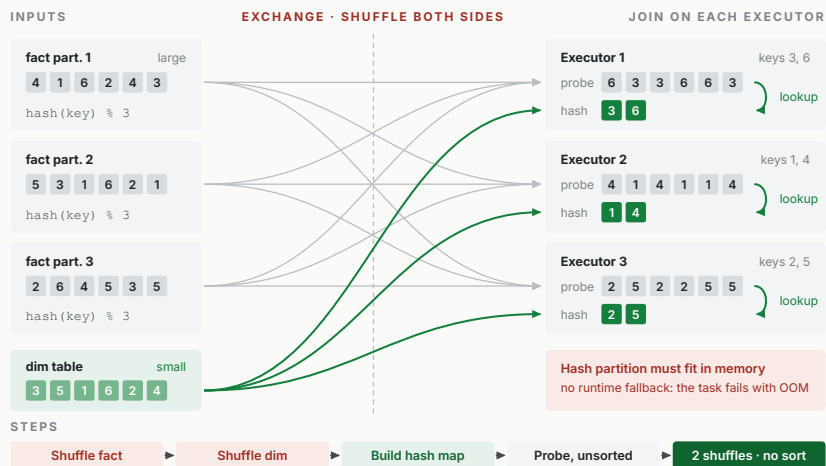


FIG. 4.4 Both sides are shuffled by join key exactly as in sort-merge, but each executor then builds a hash map from its dim partition and probes it with fact rows in arrival order, skipping the sort. Each build partition must fit in memory: open-source Spark has no runtime fallback, so an oversized partition fails the task with an OOM.

INSIDE EACH TASK

After the shuffle, each executor holds matching partitions from both sides. It loads the smaller partition into a hash map in executor memory and scans the larger one row by row, probing the map for each row. Skipping the sort saves CPU time compared with sort-merge, but the build-side partition has to fit in executor memory.

WHEN TO REACH FOR IT

Catalyst rarely picks it on its own. It earns its place when the build side is too large to broadcast but small enough to fit in executor memory once shuffled. Force it with `.hint('shuffle_hash')`. AQE chooses shuffle hash join by itself only when `spark.sql.adaptive.maxShuffledHashJoinLocalMapThreshold` is raised above 0.

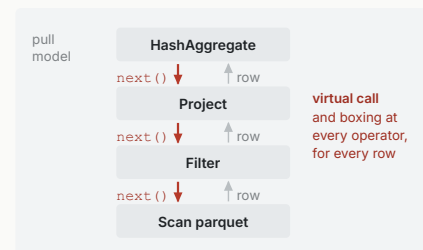
IF A PARTITION IS TOO BIG

Open-source Spark has no runtime fallback here. If a build partition does not fit in memory, the task fails with an OOM, and SPARK-32634 is still unresolved.

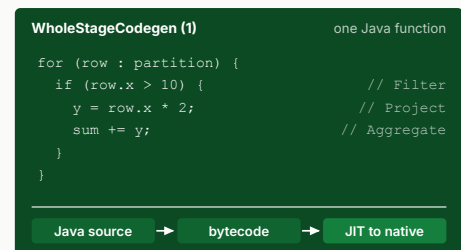
Whole-Stage Codegen.

Tungsten takes a chain of operators (scan, filter, project, aggregate), generates a single Java function for it and JIT-compiles that at runtime. Row-at-a-time interpretation becomes a tight loop in which the JVM can inline method calls, drop virtual dispatch and use registers efficiently.

INTERPRETED · ONE ROW AT A TIME



WHOLE-STAGE CODEGEN · FUSED



IN EXPLAIN(): * MARKS FUSED OPERATORS

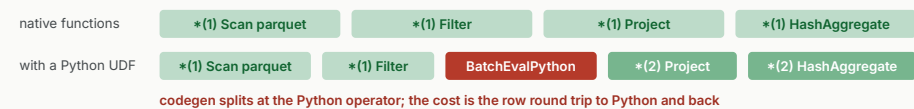


FIG. 4.5 Without codegen every operator pulls one row at a time from its child through a virtual next() call; whole-stage codegen fuses the chain into one generated Java loop that the JIT compiles, and a Python UDF splits the fused block in two at the BatchEvalPython operator, *(1) below and *(2) above.

WHAT GETS FUSED

After Catalyst picks the physical plan, Tungsten's code generator walks the operator chains and emits Java source that fuses scan, filter, project and aggregate into one function. Each partition flows through it once, top to bottom, with no intermediate materialisation between operators inside a stage. In `explain()` output the fused operators carry a `*` prefix.

WHY THE LOOP IS FASTER

Without codegen, each operator calls `next()` on its child to pull one row, processes it and hands it back up the chain, paying virtual dispatch at every step plus boxing for primitive types. With codegen the whole chain is one JVM-compiled function with no virtual dispatch and direct primitive access. On CPU-intensive workloads that is several times the throughput of interpreted execution. It arrived in Spark 2.0 as the second phase of Tungsten, has been on by default since via `spark.sql.codegen.wholeStage`, and rarely needs configuring.

WHERE THE FUSION SPLITS

Python UDFs break the codegen boundary, because the JVM cannot inline Python logic. Codegen splits at the Python operator (`BatchEvalPython` or `ArrowEvalPython`), and the operators below and above it get their own fused blocks, `*(1)` and `*(2)`. What you pay for is moving rows to Python and back; the rest of the stage keeps its codegen. Pandas UDFs at least batch the crossing instead of paying it per row. Operators without codegen support also create a boundary, shown by a missing `*` in explain output. The usual way this bites is a Python UDF slipped into a fully fused stage, splitting the block in two and adding a round trip of rows to Python. To find these boundaries, look in `explain()` for a new `*(n)` number either side of the UDF operator. Swap in native Spark functions where you can to recover the full codegen path.

PHOTON AND NEE GO FURTHER

Codegen compiles to JVM bytecode, and the JIT then compiles that to native code. Photon on Databricks and NEE on Fabric skip the JVM entirely for supported operators and run native C++ vectorised code with full SIMD parallelism. Both keep pipelining within a stage, but apply it to wider column batches with hardware SIMD instructions.

Tungsten.

QUERY LIFECYCLE

CODE UNRESOLVED ANALYSED OPTIMISED PHYSICAL **CODEGEN** DAG · TASKS EXECUTE

Tungsten is where Spark manages its own memory and optimises execution, introduced in Spark 1.4 (June 2015). Phase 1 (explicit memory management, the binary `UnsafeRow` format, cache-aware sort and hash) shipped across 1.4 to 1.6; phase 2, Whole-Stage Codegen, came with Spark 2.0. JVM execution acceleration is built on it, and it is the layer Photon and NEE replace.

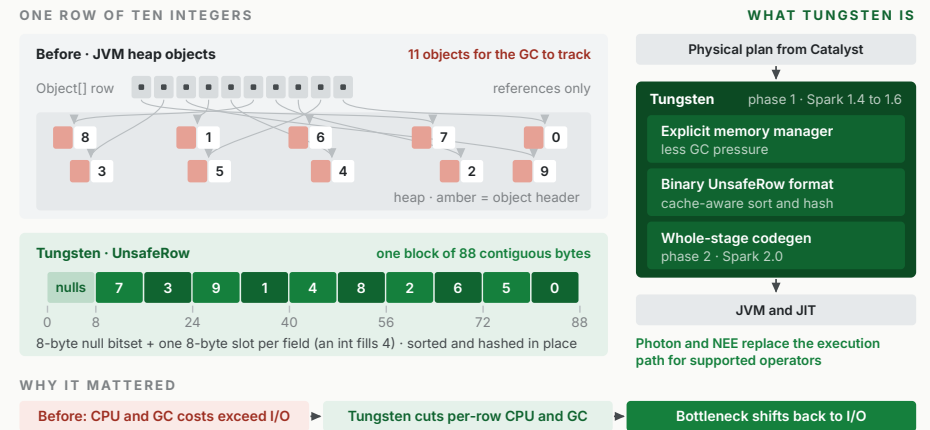


FIG. 4.6 The same row of ten integers as JVM objects (a reference array pointing at ten scattered boxed values the GC must track) and as a Tungsten UnsafeRow (88 contiguous bytes: an 8-byte null bitset plus ten 8-byte slots, managed outside the object model). Tungsten phase 1 (Spark 1.4 to 1.6) built this memory layer; phase 2 (Spark 2.0) added whole-stage codegen.

ROWS OR BATCHES

Row-at-a-time: each operator pushes one row through the whole chain before the next row is touched. The CPU hits a different code path per operator per row, which costs branch mispredictions and virtual dispatch overhead. **Vectorised:** each operator works on a batch of rows from a single column at once, so SIMD instructions handle many values per CPU cycle and memory is read sequentially. JVM Tungsten gets part of the way there through codegen's tight loops; Photon and NEE go fully vectorised in native code.

THE BINARY ROW FORMAT

Tungsten allocates memory directly with `sun.misc.Unsafe` and bypasses the JVM object model. Data sits in `UnsafeRow`, a compact binary format with a null bitset and an 8-byte slot per fixed-width field. A row of ten integers takes 88 contiguous bytes ($8 + 10 \times 8$) instead of ten heap objects with reference overhead. Sort and hash walk this layout sequentially, which keeps CPU cache hit rates high and GC interaction low. The code generator emits operator chains that read and write these binary rows directly.

WHAT IT CHANGED

Before Tungsten, Spark's CPU and GC costs often exceeded its I/O costs. Tungsten cut per-row CPU overhead and largely removed GC pressure from execution buffers, which pushed the typical bottleneck back to I/O. Add codegen in 2.0, and the typical CPU-bound job went from object churn to tight loops over binary rows.

WHERE PHOTON AND NEE TAKE OVER

Photon (Databricks) and NEE (Fabric) replace the JVM Tungsten execution path for supported operators with native C++ vectorised code at full SIMD width. Tungsten still runs the non-Photon and non-NEE operators and the underlying memory management. You rarely tune Tungsten directly now; you enable Photon or NEE, which carry its ideas in a faster implementation.

SEEING IT IN THE PLAN

Open the SQL tab in the Spark UI and look at the physical plan. Operators inside `WholeStageCodegen` blocks are running Tungsten-generated code. Operators outside them (Python UDFs, certain joins with skew handling, or unsupported expressions) fall back to interpreted execution. Each block carries a codegen ID number, so you can match task duration improvements to a codegen stage.

Photon.

QUERY LIFECYCLE

CODE UNRESOLVED ANALYSED OPTIMISED PHYSICAL CODEGEN DAG · TASKS EXECUTE

On Databricks, Photon runs supported operators in native C++ over column batches with SIMD instructions and none of the JVM overhead. You enable it per cluster type or SQL warehouse, and on SQL warehouses it is the only execution mode.

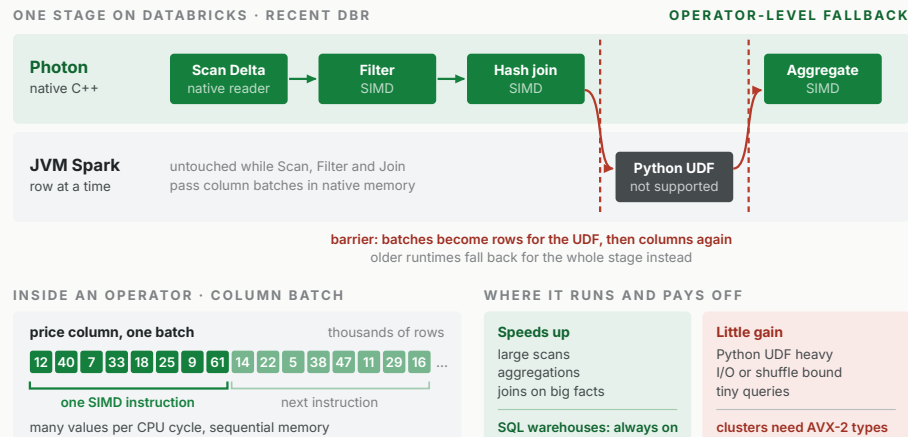


FIG. 4.7 Within one stage on Databricks, supported operators run in Photon on column batches in native memory; an unsupported Python UDF drops to the JVM behind a row-materialisation barrier, and later operators return to Photon.

WHAT RUNS ON IT

At execution time the Databricks runtime routes supported operators to Photon. Photon reads Parquet and Delta files in native code, processes column batches of thousands of rows using SIMD where available, and hands batches from operator to operator without leaving native memory. Results go back to the JVM only at non-Photon boundaries. Most SQL operators are covered: scans, filters, projections, joins, aggregations, sorts, common window functions, and most arithmetic and string functions. UDFs, the RDD and Dataset APIs, and stateful streaming are not; they fall back to the Spark runtime and still return correct results. Coverage grows with each DBR release.

SPOTTING A FALLBACK

An unsupported operator in a stage gets a row-materialisation barrier at that operator. In recent DBR versions the Photon operators upstream and downstream keep running on Photon; older runtimes fall back for the whole stage. The usual culprit is a Python UDF dropped into an otherwise Photon-accelerated stage, which quietly adds that barrier (the whole stage on older runtimes). Rewrite it with native Spark functions or Pandas UDFs. To check, open the SQL tab, where Photon-accelerated operators are annotated.

PICKING THE HARDWARE

Photon needs Photon-enabled cluster types: specific instance families with CPU support for AVX-2 at minimum. On Azure Databricks the cluster creation UI marks the Photon-enabled instance types clearly. SQL warehouses run on Photon-capable hardware by default. On other deployments Photon may not be available on small or memory-optimised instance types.

WHERE IT PAYS OFF

It does best on scan-heavy SELECT queries, aggregations over large tables and joins on large fact tables. It helps less when Python UDFs dominate the workload (they get no acceleration), when queries are small and startup cost dominates, or when the job spends most of its time in I/O wait. Photon optimises compute, not I/O, so if you compare runs with and without it on a workload dominated by I/O or shuffle, the speedup will be smaller than benchmarks suggest. Nor will it rescue a badly written query: it accelerates correct operators, not bad joins.

NEE (Native Execution Engine).

QUERY LIFECYCLE

CODE UNRESOLVED ANALYSED OPTIMISED PHYSICAL CODEGEN DAG · TASKS EXECUTE

On Microsoft Fabric, NEE runs supported operators in native C++ over column vectors with SIMD instructions, in place of JVM Spark. It is the Fabric counterpart of Photon. You change nothing in your code: existing PySpark and SQL runs on NEE wherever its operators are supported.

OPT IN: ENVIRONMENT OR SPARK.NATIVE.ENABLED · OPERATOR-LEVEL FALLBACK

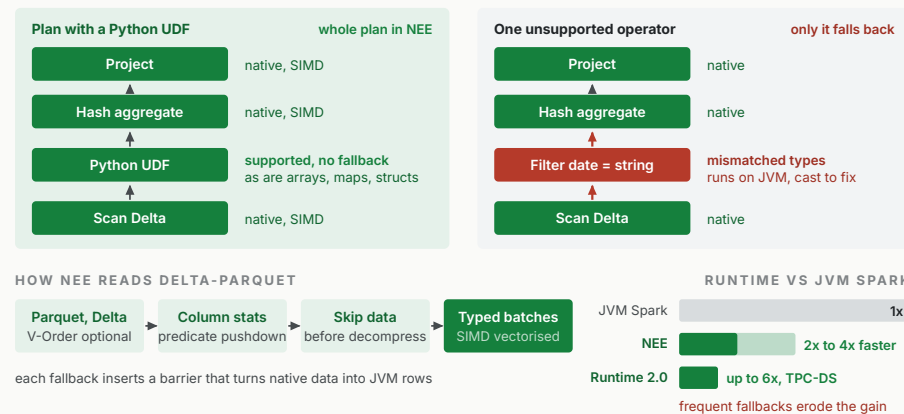


FIG. 4.8 NEE is opt in, per Environment or with `spark.native.enabled`. Python UDFs no longer force a fallback, and an unsupported operator falls back to JVM Spark on its own while the operators around it stay native; supported queries run 2x to 4x faster, up to 6x on TPC-DS with Runtime 2.0.

TURNING IT ON

NEE plugs into the Spark physical plan as an alternative backend. It covers scans, filters, projections, hash aggregations, sorts and hash joins on Parquet and Delta data. It reads columnar data natively, applies column-level statistics and predicate pushdown before decompression, and works through typed column batches with SIMD vectorisation. It is off by default. Enable it in the Environment (Spark compute, Acceleration), or set `spark.native.enabled` to `true` for a session.

WHEN IT FALLS BACK

When NEE meets an operator it does not support, that operator or plan segment runs on the standard JVM Spark path and the supported operators around it stay native. Fabric's runtime inserts a materialisation barrier where data converts from native format back to JVM rows, and each switch costs a format conversion, so frequent fallbacks eat the gain. The usual triggers are Structured Streaming, JSON and XML scans, date comparisons between mismatched types (cast explicitly), deeply nested complex types in some operations, and Python libraries that need arbitrary object serialisation. Python UDFs, Scala UDFs, and arrays, maps and structs now run on NEE and no longer force fallback, though native Spark functions are still faster. The Fabric Spark Advisor flags fallbacks in the notebook cell output; the Spark UI confirms which stages ran native and which fell back.

WHAT YOU GAIN

On supported queries and clusters, expect 2x-4x over JVM execution, because the bytecode interpretation overhead is gone and the code is vectorised and SIMD-friendly. Wide aggregations, joins with selective filters and Parquet scans gain most. Workloads dominated by scalar Python UDFs or unsupported expressions barely move, because the Python worker round trip becomes the bottleneck, not the engine.

ON RUNTIME 2.0

Fabric Runtime 2.0 (Spark 4.1, Delta 4.2) supports NEE, and Microsoft cites up to 6x faster than open-source Spark on TPC-DS workloads. It adds vectorised CSV parsing, so CSV scans run natively while JSON and XML still fall back, and parallel Delta snapshot loading to shorten query startup on large tables. ANSI mode, on by default in Spark 4, runs natively on Runtime 2.0 with the same error semantics as JVM Spark; on Runtime 1.3 it forced fallback. Several Runtime 1.3 result mismatches (rounding, decimal to float casts, duplicate map keys) are fixed.

RELATED CONCEPTS

Photon is the Databricks equivalent. V-Order is a separate, write-time Parquet optimisation aimed mainly at Power BI Direct Lake reads, not at NEE. Tungsten and Whole-Stage Codegen describe the JVM execution path NEE bypasses for supported operators.

Spill.

QUERY LIFECYCLE

CODE

UNRESOLVED

ANALYSED

OPTIMISED

PHYSICAL

CODEGEN

DAG · TASKS

EXECUTE

When execution memory runs out, Spark writes intermediate data to local executor disk and reads it back when the operation continues, instead of failing straight away. That avoids an OOM but costs at least two disk passes per spill event.

WHERE IT HAPPENS

Shuffle sort, external sort and hash aggregation spill. A full in-memory buffer is serialised to a temporary file on local executor disk, and the spilled files are merged with the remaining data at the end. The metrics are in the Spark UI Stages tab.

WHEN SPILL IS A PROBLEM

A job can succeed and still show non-zero spill; do not wave it through. Heavy spill on every run can make a job 5x to 20x slower than it would be with enough executor memory. Increase executor memory, raise `spark.memory.fraction`, or reduce partition size so buffers stay smaller.

EXECUTION MEMORY · ONE TASK

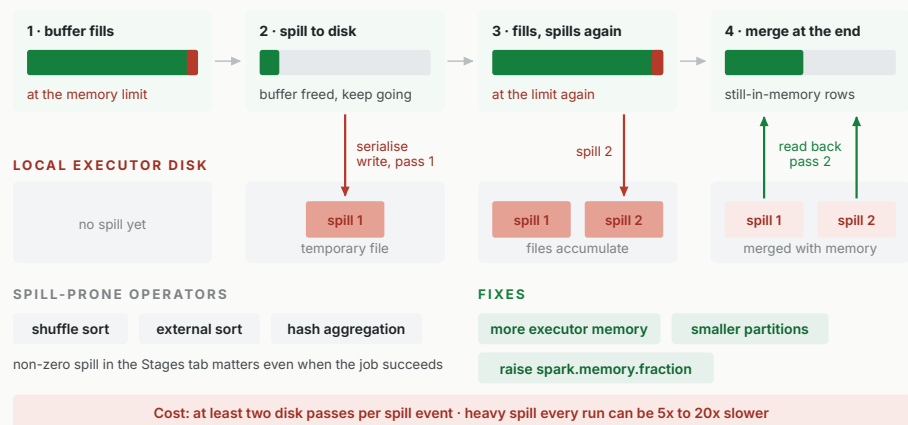


FIG. 5.1 When a sort or aggregation buffer fills execution memory, Spark serialises it to a spill file on local executor disk and keeps going; at the end it reads every spill file back and merges it with what is still in memory.

DATABRICKS SERVERLESS

Memory settings cannot be set. Of the tuning settings only

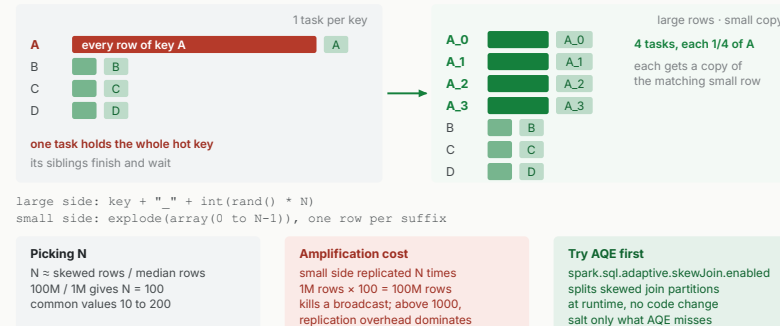
`spark.sql.shuffle.partitions` and `spark.sql.files.maxPartitionBytes` can, so make partitions smaller or cut the data per task.

Data Skew and Salting.

QUERY LIFECYCLE CODE UNRESOLVED ANALYSED OPTIMISED PHYSICAL CODEGEN DAG · TASKS EXECUTE

Data skew means one or a few values of a join or group key hold a disproportionate share of the rows, so partitions come out uneven. The task with the oversized partition runs far longer than its siblings, and the whole stage waits, because a stage cannot complete until every task finishes. Salting is the manual fix: append a random suffix to the skewed key so it becomes N virtual keys and the load spreads across N tasks.

BEFORE · JOIN ON KEY



AGGREGATES THAT STAY LARGE · TWO-STEP GROUPBY, AQE CANNOT HELP

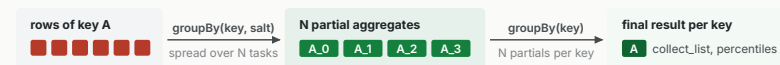


FIG. 5.2 Salting appends a random suffix 0 to N-1 to the skewed key on the large side and replicates each small-side row N times, so one oversized task becomes N balanced ones; aggregates whose partial result does not shrink, such as `collect_list` and `percentiles`, use a two-step `groupBy` instead (`sum`, `count`, `min` and `max` are already pre-aggregated on the map side).

SPOTTING IT

Skew comes from the key distribution in the data. Join on a key where one value appears in 80% of rows and 80% of that side goes to one task. A `groupBy` with `sum` or `count` suffers far less, because Spark pre-aggregates on the map side and ships about one row per key per map task. In the Spark UI skew shows up as a long tail in the task duration distribution within a stage; a median task time of 5 seconds with a max of 10 minutes is the classic signature. One skewed task makes the whole stage as slow as that task, so fixing it often cuts stage time by an order of magnitude; it is consistently one of the best-paying fixes in production pipelines. Treat it as a query design problem as well as a data problem: filtering the skewed key separately, salting the join key or enabling AQE skew join handling often makes it go away.

LET AQE TRY FIRST

Since Spark 3.0, AQE skew join handling (`spark.sql.adaptive.skewJoin.enabled`, default true with AQE) detects skewed partitions at runtime, splits the oversized ones and replicates the matching rows from the other side. You change no code. Look for skew-handling annotations on the join in the SQL tab. Salt by hand only for what AQE misses. It handles joins only, so `groupBy` skew (one key holding most rows in an aggregation) is out of reach. So is skew severe enough that AQE's split bucket size is still too large, and a skewed key known in advance where you want determinism instead of runtime detection.

SALTING A JOIN

Add a random integer suffix to the join key in the larger table:

```
df_large.withColumn("salted_key", concat(col("key"), lit("_"), (rand() * N).cast("int"))). Replicate every row of the smaller table N times, once per possible suffix: df_small.withColumn("salt", explode(array([lit(i) for i in range(N)]))).withColumn("salted_key", concat(col("key"), lit("_"), col("salt"))). Join on salted_key. The skewed value now maps to N partitions instead of one.
```

CHOOSING N AND PAYING FOR IT

As a rough guide, N is the ratio of the skewed key's row count to the median key's row count. If one key has 100M rows and the median key has 1M, N=100 spreads the skewed key over roughly 100 tasks of 1M rows each. Smaller N means less overhead but leftover skew; larger N inflates the smaller table more than necessary. Common values are 10 to 200, and above 1000 the overhead of replicating the smaller table dominates. That replication is the price of salting: at N=100 a 1M-row dimension table becomes 100M rows. For a broadcast join this kills the broadcast, since the table is now too large; for sort-merge it adds shuffle cost. Check the alternatives first: filter the skewed key out separately, broadcast the dimension if it is small, or rely on AQE.

SALTING A GROUPBY

For aggregation skew, aggregate twice. First group by `(key, salt)` to get N partial aggregates per key. Then group by `key` alone and re-aggregate the partials. The first step spreads the work across N tasks; for sums and counts the second is small because it works on per-key partial results. For `collect_list` the N partial lists still hold every value of the hot key, so salting spreads the collection work but the final task still receives all of it. Spark already does this for `sum`, `count`, `min` and `max` through its partial aggregate, so salting pays off only for aggregates whose partial result does not shrink, such as `collect_list`, percentiles and some distinct aggregates.

Repartition, Coalesce, and RepartitionByRange.

QUERY LIFECYCLE CODE UNRESOLVED ANALYSED OPTIMISED PHYSICAL CODEGEN **DAG · TASKS** EXECUTE

These three calls control partition count and distribution. `repartition(n)` shuffles the data round-robin into n evenly sized partitions, and `repartition(n, cols)` hash-partitions on the columns. `coalesce(n)` merges existing partitions on the same executor without a shuffle. `repartitionByRange(n, col)` shuffles into n partitions that each hold a contiguous range of the column's values, so the output is sorted across partitions.

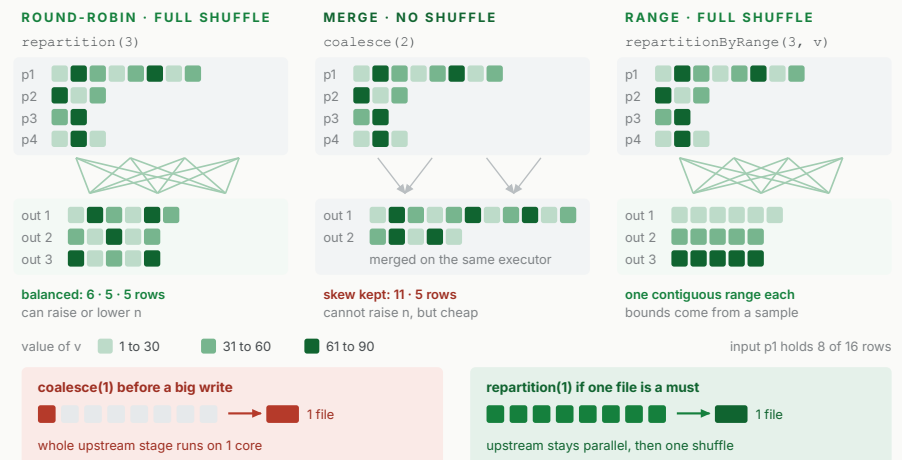


FIG. 5.3 repartition and repartitionByRange shuffle every row, giving evenly sized round-robin partitions or contiguous value ranges; coalesce merges neighbouring partitions without a shuffle, so it is cheap but keeps any skew.

PICKING ONE

repartition always costs a full shuffle. Use it to fix skew (the shuffle redistributes evenly), set the partition count before a wide transformation, or raise parallelism. **coalesce** has no shuffle, so it is cheap, but it cannot increase the partition count. Use it after a filter that leaves most partitions empty, or before a write to consolidate output files. **repartitionByRange** is for when you need sorted output: downstream range queries, file pruning by range, or stable sort-merge join partitioning.

SHUFFLE OR NO SHUFFLE

Coalesce skips the shuffle but cannot rebalance, so skewed upstream partitions stay skewed. Repartition pays for a shuffle and gives balanced output. After a heavily filtered scan, use coalesce; to fix existing skew before a join, repartition is worth the shuffle.

WHERE PEOPLE GET BURNED

`coalesce(1)` before writing a multi-GB result to get one file forces all the data through one executor and one core and kills parallelism for the entire upstream computation. If you really need one file, use `repartition(1)`, which at least shuffles in parallel before consolidating, or accept multiple output files. `repartitionByRange` on a column with very skewed values also gives uneven partitions, whatever the name suggests, because the boundaries are chosen from sampled data.

Broadcast join failure modes.

QUERY LIFECYCLE

CODE UNRESOLVED ANALYSED OPTIMISED **PHYSICAL** CODEGEN DAG · TASKS EXECUTE

A broadcast hash join can fail in three ways: **broadcast timeout** (the broadcast itself did not finish in time), **driver OOM** (the driver could not hold the broadcast in memory while serialising it) and **executor OOM** (the broadcast hash table was too large for executor memory once deserialised). The error messages look alike, but each needs a different fix.

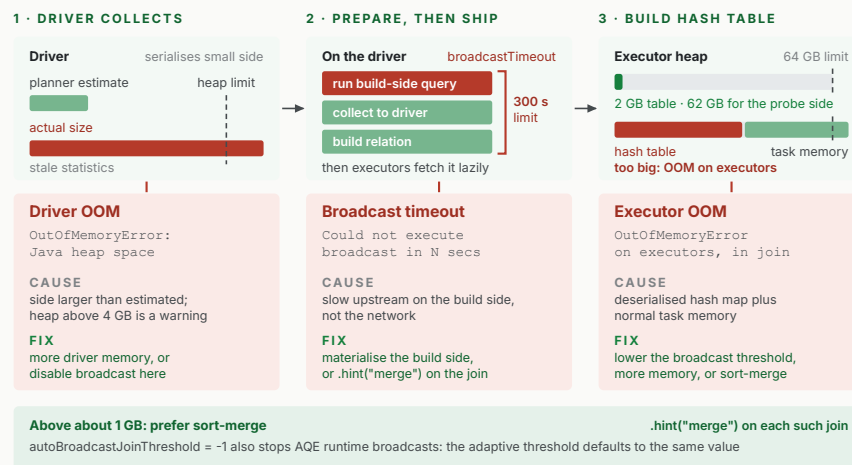


FIG. 5.4 A broadcast join can fail at three points: the driver runs out of heap while collecting and serialising the small side, the driver does not finish running the build-side query, collecting it and building the relation within `spark.sql.broadcastTimeout` (executors fetch it lazily afterwards), or an executor runs out of memory building the hash table.

WHEN THE BROADCAST TIMES OUT

You see `Could not execute broadcast in N secs`. The driver did not finish preparing the broadcast within `spark.sql.broadcastTimeout` (default 300 seconds). That clock covers running the build-side query, collecting it and building the relation; executors fetch the result lazily afterwards. The usual cause is a slow build-side upstream, not the network. Speed up or materialise the build side (write it out, or cache and count it first), or stop the broadcast with `.hint("merge")` or `.hint("shuffle_hash")` on the join. Raise the timeout only when the build side is small but genuinely slow.

DRIVER OUT OF HEAP

The driver logs show `OutOfMemoryError: Java heap space` during a broadcast. The smaller side proved larger than estimated and the driver could not hold the serialised form. Stale statistics are the most common cause, because the planner thought the table was small. Watch the driver row in the Executors tab: heap usage above 4 GB during broadcast collection is the warning sign. Increase driver memory, or disable the broadcast for this join.

EXECUTORS OUT OF MEMORY

Executors throw `OutOfMemoryError` during the join itself. The broadcast arrived, but deserialising it into a hash map on top of normal task memory was too much for each executor. Lower `spark.sql.autoBroadcastJoinThreshold` so this join no longer qualifies as a broadcast, increase executor memory, or rewrite the join as sort-merge. For scale: on a 64 GB heap, a 2 GB broadcast leaves 62 GB for probe-side processing and other operator memory.

SWITCHING BROADCAST OFF

Above about 1 GB, a broadcast is risky even when the planner picks it. Put `.hint("merge")` on that specific join. Setting `spark.sql.autoBroadcastJoinThreshold = -1` forces sort-merge for the whole session, and because `spark.sql.adaptive.autoBroadcastJoinThreshold` defaults to the same value, it also stops AQE converting joins to broadcast at runtime. If you do switch it off at session level, set the adaptive threshold separately to keep runtime broadcasts.

DATABRICKS SERVERLESS

You cannot set `spark.sql.autoBroadcastJoinThreshold` or `spark.sql.broadcastTimeout` here. Join hints such as `.hint("merge")` still work.

On-Heap Memory.

QUERY LIFECYCLE

CODE UNRESOLVED ANALYSED OPTIMISED **PHYSICAL** CODEGEN DAG · TASKS EXECUTE

By default Spark keeps its working memory on the JVM heap, where the garbage collector manages it. All standard execution lives there: the unified memory pool with its storage and execution regions, user memory for UDFs and Spark internal bookkeeping, and the reserved JVM system overhead of approximately 300 MB.

WHAT IT COSTS YOU

You configure nothing and it is safe by default. You pay in GC pressure: when the heap fills, the JVM pauses all threads to reclaim memory. On executor heaps above 32 GB, a full GC pause can stop every task for several seconds. If the pause runs past the executor heartbeat timeout, the driver declares the executor dead and the task fails.

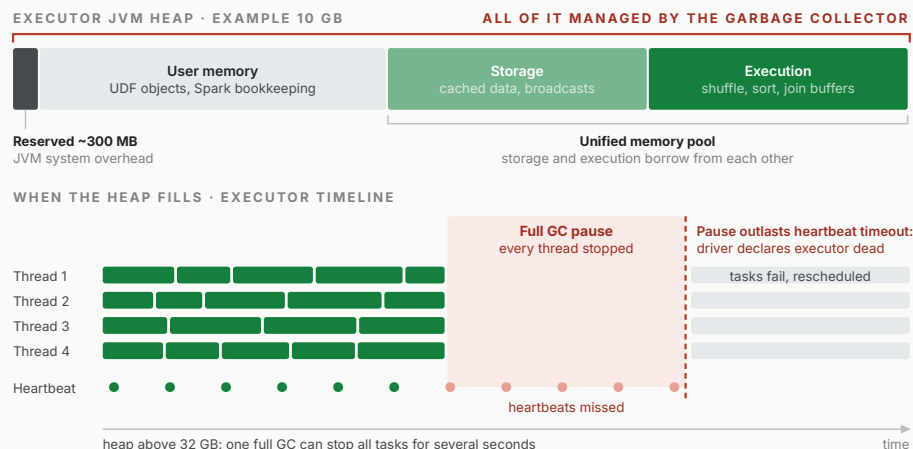


FIG. 6.1 Everything Spark allocates by default lives inside the JVM heap, which the garbage collector scans; when the heap fills, a full GC pauses every task thread and, if the pause outlasts the heartbeat timeout, the driver declares the executor dead.

DATABRICKS SERVERLESS

Not applicable: you cannot set heap size or garbage collector settings.

RELATED CONCEPTS

Off-Heap Memory bypasses GC. Unified Memory is the pool within the heap. GC Pressure covers the failure mode in detail.

Off-Heap Memory.

QUERY LIFECYCLE CODE UNRESOLVED ANALYSED OPTIMISED PHYSICAL CODEGEN DAG · TASKS **EXECUTE**

Spark allocates this memory outside the JVM heap with `sun.misc.Unsafe`, so the garbage collector never tracks it. When you enable it, Tungsten keeps its binary row format there. The on-heap footprint shrinks, and the data stored off-heap causes no GC pauses.

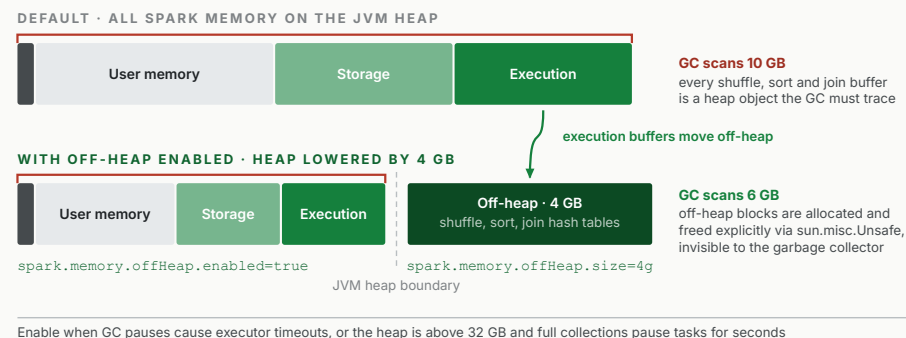


FIG. 6.2 Enabling off-heap memory moves execution buffers out of the JVM heap into blocks Spark allocates and frees itself, so the garbage collector has far fewer objects to trace. The off-heap size is added on top of the heap, so the heap here is lowered from 10 GB to 6 GB (`spark.executor.memory`) to keep the container the same size.

TURNING IT ON

Set `spark.memory.offHeap.enabled=true` and give it a size with `spark.memory.offHeap.size`. Spark allocates and frees the blocks explicitly through `sun.misc.Unsafe`. The execution region then keeps shuffle buffers, sort buffers and join hash tables off-heap, which cuts the on-heap object count and shortens GC pauses. The off-heap size is added on top of the heap, not carved out of it. If the container size must stay the same, lower `spark.executor.memory` by the same amount.

WHEN IT EARNS ITS KEEP

Turn it on when GC pauses are causing executor timeouts, or when the JVM heap is over 32 GB and full collections pause tasks for multiple seconds. It also helps memory-intensive aggregations or joins where the execution region keeps filling up.

DATABRICKS SERVERLESS

Not applicable: you cannot set `spark.memory.offHeap.*`.

Unified Memory.

QUERY LIFECYCLE CODE UNRESOLVED ANALYSED OPTIMISED PHYSICAL CODEGEN DAG · TASKS **EXECUTE**

Since Spark 1.6, **storage memory** (cached DataFrames, broadcast variables) and **execution memory** (shuffle buffers, sort buffers, join hash tables) share one pool and borrow from each other as needed. Before that, each had a fixed region of its own, and an imbalanced workload would OOM in one while the other sat idle. The unified pool replaces both with a single boundary that shifts on demand.

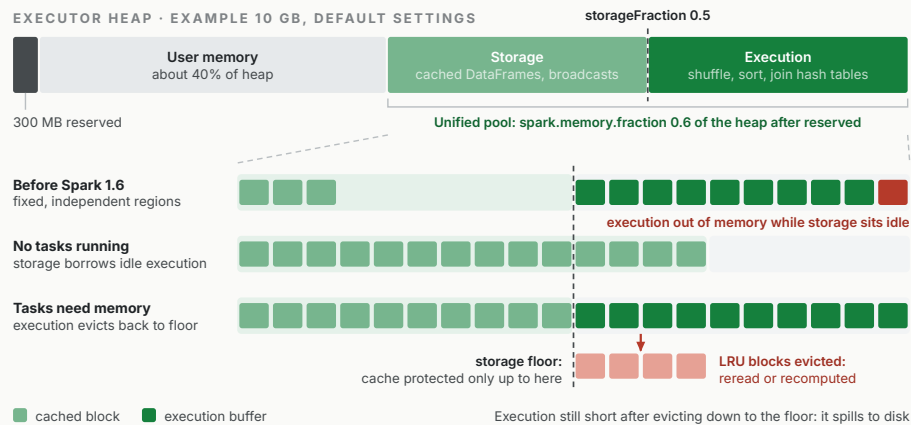


FIG. 6.3 Storage and execution share one pool with a movable boundary: storage may borrow idle execution memory, and execution evicts least recently used cached blocks when it needs space, but only down to the storage floor.

HOW THE HEAP IS SPLIT

Spark first holds back 300 MB of reserved memory. `spark.memory.fraction` (default 0.6) sets the unified pool as a fraction of the heap that remains. Inside the pool, `spark.memory.storageFraction` (default 0.5) is the minimum guaranteed to storage; anything above it is fluid. The rest of the heap, about 40% by default, holds user objects, JVM internals and the Spark task code itself. The boundary moves on every memory request. Execution can evict storage blocks when it needs room, down to the `storageFraction` minimum. Storage can use idle execution memory when no tasks are running, but must give it back on demand. Storage eviction is LRU: least recently used cached blocks go first. Spilled cached DataFrames go to disk (MEMORY_AND_DISK) or are recomputed (MEMORY_ONLY).

TUNING THE SPLIT

Three settings matter in practice. `spark.memory.fraction` grows or shrinks the unified pool. Raising it gives caching and joins more room but leaves less user memory inside the JVM heap (user data structures, UDF buffers, internal metadata), so OOMs there get more likely. The usual mistake is pushing it to 0.8 or higher to "give Spark more memory"; user data structures, UDF buffers and large broadcast objects then OOM unexpectedly. Python workers run outside the JVM heap, so this setting does not squeeze them.

`spark.memory.storageFraction` sets the cache floor; lower it when execution matters more than caching. `spark.memory.offHeap.enabled` with `spark.memory.offHeap.size` moves execution memory off-heap, out of GC pressure. Do not assume cached DataFrames are safe: they are protected only up to the storage floor, and under heavy execution pressure cached blocks evict and recompute on next access.

SPOTTING A MISMATCH

The Executors tab in the Spark UI shows storage memory used against allocated. If a cached DataFrame should be 10 GB but only 4 GB shows as stored, execution memory borrowed the rest. Frequent recomputation of cached data is the symptom you notice. Either shrink the cache footprint or raise total executor memory.

DATABRICKS SERVERLESS

Not applicable: you cannot set `spark.memory.fraction` or `spark.memory.storageFraction`.

RELATED CONCEPTS

Spill is what happens when execution cannot claim enough space even after evicting cached blocks down to the `storageFraction` floor. On-Heap Memory and Off-Heap Memory describe where this pool lives. GC Pressure is the failure mode when the pool is too small relative to allocation rate.

GC Pressure.

QUERY
LIFECYCLE

CODE

UNRESOLVED

ANALYSED

OPTIMISED

PHYSICAL

CODEGEN

DAG · TASKS

EXECUTE

The JVM garbage collector spends so much wall-clock time reclaiming heap memory that its pauses slow or fail Spark tasks. When it gets severe you see executor heartbeat timeouts, task failures with lost executor messages, and jobs that run far slower than their I/O profile would predict.

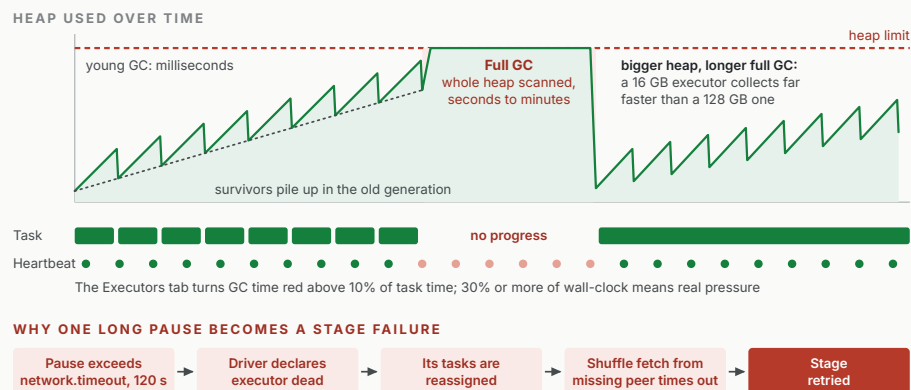


FIG. 6.4 Short-lived task objects trigger brief young-generation collections while survivors pile up in the old generation; when the heap hits its limit a full GC freezes the executor, and a pause longer than `spark.network.timeout` turns one slow executor into a stage retry.

WHY THE PAUSES HAPPEN

Spark tasks churn out short-lived Java objects: row iterators, buffers, intermediate aggregation maps. As heap usage nears the limit, the JVM pauses all threads to reclaim memory. A young-generation GC is brief (milliseconds). A full GC scans the entire heap and can pause for seconds, even minutes, on large heaps. While it runs the task makes no progress and the executor cannot heartbeat. If a pause runs past `spark.network.timeout` (default 120 seconds), the driver declares the executor dead and reassigns its tasks. Other executors then fetch shuffle data from the now-missing peer, time out and trigger a stage retry. One sustained GC episode turns into a stage-level failure instead of a slow task.

SPOTTING IT

Open the Stages tab in the Spark UI, then the Executors tab, which shows GC time in red once it passes 10% of task time. Treat that as an early warning. A task spending 30% or more of its wall-clock time in GC is under real pressure. The executor logs record each GC event with its duration. Other signs: task durations vary widely across runs, executors are periodically lost and restarted, and throughput drops sharply after the job has run for several minutes as the old generation fills.

WHAT TO CHANGE

Do not give an executor a very large heap and leave the GC settings alone. A 200 GB heap can pause for tens of seconds on a full collection, even with G1, the default collector on the Java 17 and 21 runtimes Spark 4 runs on. Prefer several smaller executors over one large one: a 16 GB executor collects far faster than a 128 GB executor processing the same total data. Tuning G1 (already the JVM default on Java 17 and 21) with larger region sizes is often the least disruptive fix for large-heap GC problems. A smaller heap shortens pauses but may spill more to disk. Off-heap memory removes GC from execution buffers, at the cost of explicit size configuration and no GC safety net. For heartbeat timeouts, raising `heartbeatInterval` does NOT fix them; it makes them more likely. If the timeouts are misdiagnosed GC pauses, increase `spark.network.timeout` instead.

DATABRICKS SERVERLESS

Not applicable: you cannot set heap size or garbage collector settings.

RELATED CONCEPTS

Off-Heap Memory bypasses GC for execution buffers. Spill is the other memory pressure relief valve. On-Heap Memory is where GC operates.

`cache()` and `persist()`.

QUERY
LIFECYCLE

CODE

UNRESOLVED

ANALYSED

OPTIMISED

PHYSICAL

CODEGEN

DAG · TASKS

EXECUTE

`cache()` and `persist()` materialise a DataFrame into executor memory so later actions skip recomputation. Without them, every action re-executes the full plan from source.

WHAT GETS STORED

In PySpark, `cache()` equals `persist(StorageLevel.MEMORY_AND_DISK_DESER)`, which is also the default level of `persist()`. Watch the constant names:

`StorageLevel.MEMORY_AND_DISK` is the serialised level, so passing it is not the same as `cache()`. On the first action Spark computes and stores each partition; later actions read the cached partitions. If unified memory pressure evicts a partition, it is read back from disk (MEMORY_AND_DISK) or recomputed (MEMORY_ONLY). Call `.unpersist()` to release it.

ON SERVERLESS

On Databricks serverless compute, `cache()` and `persist()` raise an error. Disk Cache still applies there.

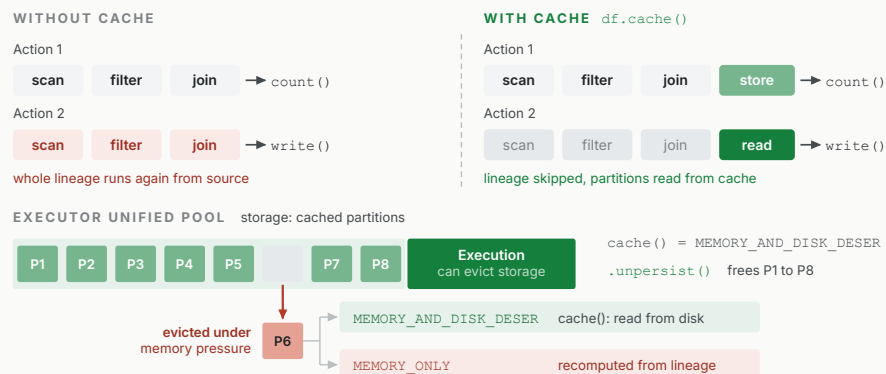


FIG. 6.5 Without caching, every action reruns the whole lineage from source; after `cache()`, the first action stores each partition in executor storage memory and later actions read it back, falling back to disk or recomputation if a partition is evicted. In PySpark, `DataFrame.cache()` uses `MEMORY_AND_DISK_DESER`.

WHEN IT PAYS OFF

Cache a `DataFrame` only when two or more actions use it. Caching one used once adds overhead and buys nothing. In long notebooks, call `unpersist()` when you are done; otherwise executor storage fills up and evicts other cached data.

Broadcast Variables.

QUERY LIFECYCLE CODE UNRESOLVED ANALYSED OPTIMISED PHYSICAL CODEGEN DAG · TASKS EXECUTE

The driver serialises a read-only value once and sends it to every executor, so tasks do not each carry their own copy. Broadcast joins run on this mechanism, and you can use it directly for lookup tables, configuration maps or any data every task needs to read.

CREATING AND READING ONE

Call `spark.sparkContext.broadcast(value)`. Spark serialises the value on the driver, distributes it to each executor once with a BitTorrent-style protocol and caches it in executor storage memory. Tasks read it through `.value`. It stays there until you call `.unpersist()` explicitly.

CHANGING THE VALUE

You cannot. A broadcast variable is immutable: `.unpersist()` only drops the executor copies, and the next use re-sends the same original value. To change it, create a new broadcast and, if you want the old one gone, `destroy()` it.

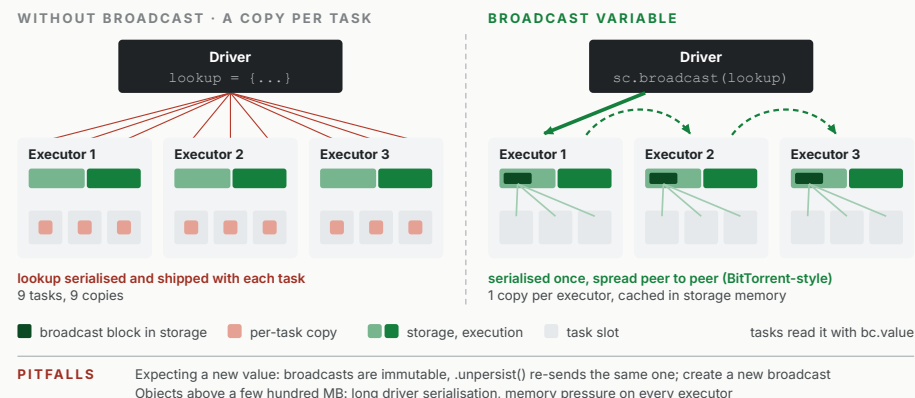


FIG. 6.6 Without broadcasting, the driver ships a copy of the lookup data inside every task; a broadcast variable is serialised once, spread to executors peer to peer, and cached once per executor in storage memory, where every task reads it through `.value`.

KEEP IT SMALL

Once an object goes above a few hundred MB, the driver spends significant time serialising it and every executor feels the memory pressure.

Disk Cache and Intelligent Cache.

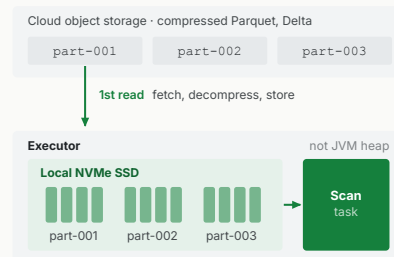
QUERY LIFECYCLE CODE UNRESOLVED ANALYSED OPTIMISED PHYSICAL CODEGEN DAG · TASKS EXECUTE

Both platforms keep data read from remote storage on the local NVMe SSD of the compute nodes, so a repeated scan skips the remote read and you never write a cache call. On Databricks this is **Disk Cache**: it stores decompressed Parquet and Delta data on executor local SSD, outside the JVM heap. On Microsoft Fabric it is **Intelligent Cache**: it caches frequently accessed OneLake data with no explicit configuration, watching access patterns and promoting hot data to local storage.

ON DATABRICKS

After the first read from cloud storage, files sit on the executor local SSDs and later reads bypass cloud storage. The cache key is file path plus modification time, so rewriting a file invalidates its entry. The cache is per cluster: it survives across queries but not restarts. It works particularly well on Delta tables, because the decompressed columnar layout is cached in a format Photon reads directly.

DATABRICKS · DISK CACHE



Stores	decompressed columnar Parquet, Delta
Stale check	key = file path + modification time
Scope	per cluster; survives queries, not restarts
Photon	reads the cached layout directly
Control	transparent, no <code>cache()</code> call

BOTH REACTIVE

first read pays
remote latency

Query 1: miss

remote read, fill SSD

Query 2: hit

read local SSD

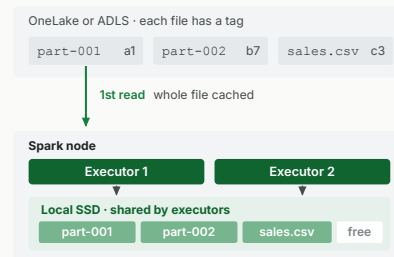
File rewritten

stale, fetched again

Random access

never repeats, no gain

FABRIC · INTELLIGENT CACHE



Stores	whole files: Parquet, Delta, CSV
Stale check	remote tag compared on every read
Scope	per node; LRU eviction when full
Default	on, 50% of node disk
Toggle	<code>spark.synapse.vegas.useCache</code>

RELATED CONCEPTS

`cache()` and `persist()` are explicit alternatives for pinning specific DataFrames. Photon reads the Disk Cache format directly. OneLake is the storage Intelligent Cache sits in front of on Fabric.

FIG. 6.7 Databricks Disk Cache and Fabric Intelligent Cache side by side: both copy data read from remote storage onto local NVMe SSD on first read so repeat scans stay local. Disk Cache stores decompressed columnar data per cluster, keyed by path and modification time; Intelligent Cache stores whole files per node, shared by executors, checks the remote tag on every read and evicts the least recently used file.

ON FABRIC

Files read from OneLake or ADLS (Parquet, Delta, CSV) are cached on the local SSD of each Spark node, and executors on the same node share that cache. On every read the file's tag in remote storage is compared with the cached copy, so stale files are fetched again. When the cache is full, the least recently accessed file is evicted. It is on by default with 50% of node disk; toggle it with `spark.synapse.vegas.useCache`.

WHERE THEY DIFFER

Disk Cache keeps the decompressed columnar layout of Parquet and Delta files; Intelligent Cache keeps Parquet, Delta and CSV files. For staleness, Disk Cache keys on path plus modification time, while Intelligent Cache checks the remote file tag on every read. Disk Cache is per cluster and empties on restart; Intelligent Cache is per node, shared by its executors, with least recently accessed eviction. Both are transparent. On Fabric the visible knobs are the default cache size and the `spark.synapse.vegas.useCache` toggle.

WHEN IT DOES NOT HELP

Both speed up repeated scans without explicit cache calls, but both are reactive and only cache after the first access. The first query in a session reads from remote storage at full latency, and workloads with random, non-repeating access patterns get no benefit. When you know in advance which DataFrames you will reuse, explicit `cache()` is still useful.

03

Data at rest and in motion.

Many slow queries turn out to be layout problems. This part covers how file layout decides what a scan reads, how the Delta log turns those files into a table you can time travel, MERGE into and write to concurrently, and how Structured Streaming keeps a table current one micro-batch at a time.

VII Reading data and file layout

39	Reading data and schemas	41
40	On-disk Partitioning	42
41	Small-file Problem	42
42	OPTIMIZE and Bin-packing	43
43	Data Skipping	43
44	Z-Ordering	44
45	V-Order	44
46	Liquid Clustering	45
47	Delta compaction strategies	46
48	Predictive Optimization	47

VIII Delta Lake mechanics

49	ACID Transactions	48
50	Transaction Log	49
51	Time Travel	49
52	Schema Enforcement and Evolution	50
53	Writing data correctly	51
54	MERGE / Upsert	52
55	Deletion Vectors	53

VIII Delta Lake mechanics cont.

56	Optimistic Concurrency	54
57	VACUUM and Retention	55
58	Change Data Feed	55
59	Type widening and row tracking	56
60	Catalog-managed tables	57
61	Table and catalog object model	58

IX Structured Streaming

62	Micro-batch Model	59
63	Triggers	59
64	Watermarks and State	60
65	Output Modes and Exactly-once	61
66	Streaming checkpoints	61
67	Streaming joins	62
68	Auto Loader	63
69	Real-time mode	64
70	transformWithState and State Data Source	65
71	Spark Declarative Pipelines	66

READING PATH

Layer heat: **A B C D E F G H** = Files to Operate

VII

How do files on disk decide what a query reads?



VIII

How does the log turn files into a reliable table?



IX

How does a table stay current, exactly once?

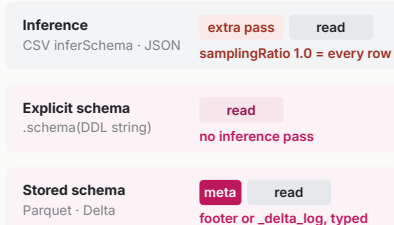


Reading data and schemas.

LAYER FILES TABLE STREAM COMPUTE GOVERN PIPELINE SERVE OPERATE

Spark needs a schema before it can plan a read. You pass one with `.schema()`, Spark infers it from the data, or it reads metadata stored with the data (the Parquet footer, the Delta transaction log). Reader options also decide what happens to rows that do not fit and, for JDBC, how many queries run in parallel.

SCHEMA SOURCE · WORK BEFORE THE JOB



MALFORMED LINE · READ MODE



DATABRICKS ONLY

badRecordsPath: bad lines to JSON files

_rescued_data: mismatched fields as JSON

PARTITIONED JDBC READ

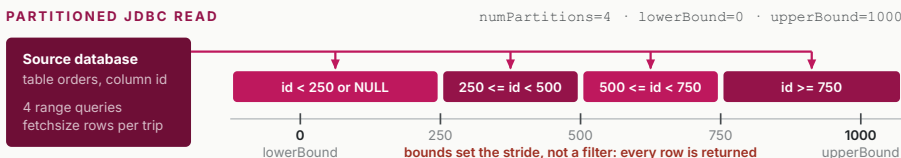


FIG. 7.1 Where the schema comes from decides the cost before the read: inference adds a full pass, an explicit or stored schema does not. The read mode decides what happens to a malformed line, and a partitioned JDBC read splits one table into range queries whose bounds set the stride, not a filter.

INFERENCE AND BAD ROWS

CSV `inferSchema` defaults to `false`, so every column comes back as a string; `true` costs one extra pass over the data. JSON infers whenever you give no schema, and reads every object to do it because `samplingRatio` defaults to 1.0. Lower it and inference is faster but can miss rare fields. Rows that fail to parse follow `mode`. **PERMISSIVE** (default) sets bad fields to `NULL` and keeps the raw text in the column named by `columnNameOfCorruptRecord` (`_corrupt_record`); for CSV you only get that text if you declare the column as a string in your schema. **DROPMALFORMED** drops the whole record. **FAILFAST** throws on the first bad record. Keep **DROPMALFORMED** out of production: it loses rows without a count.

WHY TEXT FORMATS COST MORE

CSV and JSON are portable but untyped. Every read parses text, inference scans it, and nothing can be skipped. Parquet and Delta store typed columns, so the schema is a metadata lookup and column pruning and predicate pushdown apply. For text formats in production, pass an explicit schema, a `StructType` or a DDL string (`"id BIGINT, ts TIMESTAMP"`). You avoid the inference pass and the contract lives in code. Land the raw text once, then read Delta downstream.

PARALLEL JDBC READS

A JDBC read with no partitioning options is one query on one connection. Set `partitionColumn` (numeric, date or timestamp), `lowerBound`, `upperBound` and `numPartitions` together and Spark issues one range query per partition. The bounds only set the stride. They do not filter, every row still comes back, and a skewed column gives you skewed tasks. `numPartitions` also caps concurrent connections; set it too high and you overwhelm the source. Raise `fetchsize` for drivers with small defaults (Oracle fetches 10 rows at a time).

ON DATABRICKS AND FABRIC

On Databricks, `badRecordsPath` writes bad records and unreadable files as JSON to a timestamped directory and drops them from the result; it is not transactional. The `rescuedDataColumn` reader option captures fields with type, case or name mismatches as JSON instead of nulling them. Auto Loader (`cloudFiles`) adds `_rescued_data` automatically when it infers a schema, sampling the first 50 GB or 1000 files. Where Lakehouse Federation is supported, Databricks prefers it to raw JDBC. On Fabric the Apache Spark options apply, but `badRecordsPath` and rescued data are Databricks features, so use **PERMISSIVE** with a corrupt record column. Read Lakehouse files by the relative path `Files/...` (default lakehouse) or an `abfss://` OneLake path.

RELATED CONCEPTS

Schema Enforcement and Evolution, Auto Loader, Delta-Parquet as Native Format, Predicate Pushdown, Column Pruning, Data Skew and Salting, VARIANT type.

On-disk Partitioning.

LAYER FILES TABLE STREAM COMPUTE GOVERN PIPELINE SERVE OPERATE

Partitioning splits a Delta or Parquet table into separate directory paths, one per unique value combination of the partition columns: `date=2024-01-01/`, `date=2024-01-02/`. A query that filters on the partition column skips whole directories without opening any files.

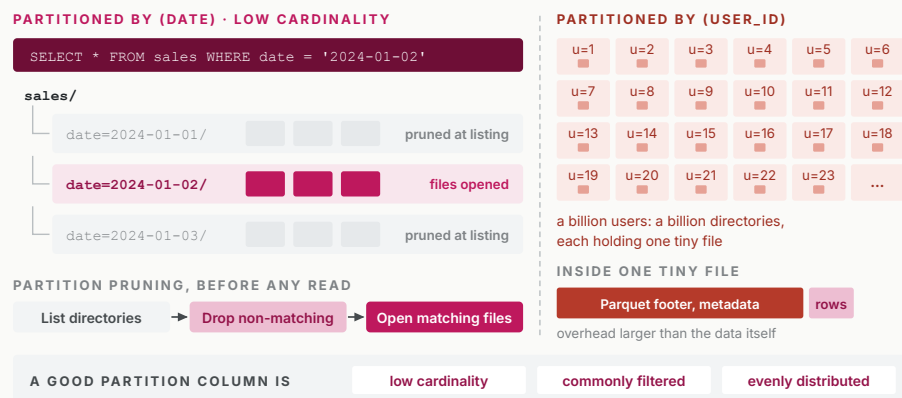


FIG. 7.2 Each value of the partition column becomes a directory. A filter on that column prunes non-matching directories while listing, before any file is opened; a high-cardinality column instead explodes into countless directories of tiny files.

DECLARING PARTITIONS

Set it when you create the table, with `PARTITIONED BY (col)` in SQL or `partitionBy('col')` on a DataFrame write. Each write appends files to the matching partition directory. Partition pruning then drops the non-matching directories while the file system is being listed, before any data is read. It works best on low-cardinality columns with an even distribution that filters use often: date, region, status.

WHEN IT BACKFIRES

Partition on a high-cardinality column and you get thousands of directories, each holding tiny files. Partition a billion-row table by user ID and you get a billion directories. File system metadata operations then dominate the runtime, and each file carries Parquet footer overhead bigger than the data inside it. Pick partition columns that are low cardinality, commonly filtered and evenly distributed.

RELATED CONCEPTS

Small-file Problem is the failure mode of over-partitioning. Liquid Clustering is the modern alternative that avoids fixed partition columns entirely.

Small-file Problem.

LAYER FILES TABLE STREAM COMPUTE GOVERN PIPELINE SERVE OPERATE

A table has a small-file problem when it holds many files too small to read efficiently. Every file carries fixed overhead: Parquet footer parsing, object-store metadata, task scheduling. When files are kilobytes in size instead of hundreds of MB, that overhead dominates.

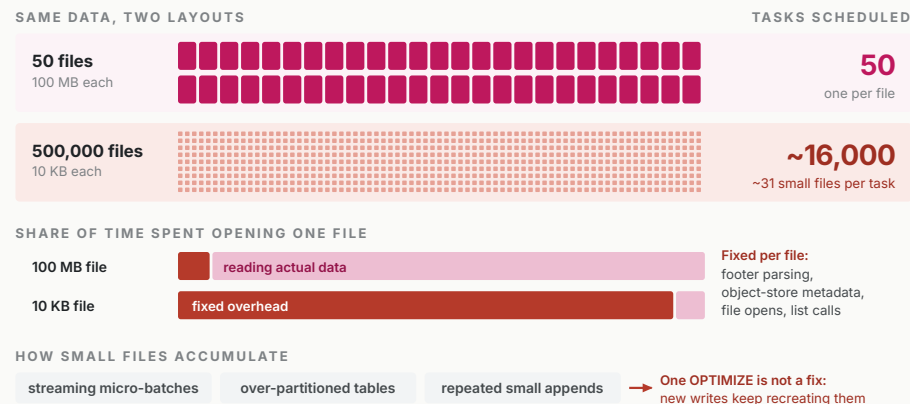


FIG. 7.3 The same data stored as 50 files of 100 MB or as 500,000 files of 10 KB. Every file pays a fixed cost for footer parsing and object-store metadata, 10000x more of it here. Spark packs small files into read tasks (each counted as at least `openCostInBytes`, 4 MB, up to `maxPartitionBytes`, 128 MB), yet about 16,000 tasks still replace 50, so overhead, not the data, sets the runtime.

WHERE THEY COME FROM

They accumulate from streaming ingestion writing frequently in small batches, over-partitioned tables receiving few rows per write, and repeated small appends.

WHAT SPARK DOES WITH THEM

Compare 500K files of 10 KB with 50 files of 100 MB. The data is the same, but the first table opens 10000x more files and footers. Spark packs small files into read tasks, counting each as at least `spark.sql.files.openCostInBytes` (4 MB) up to `spark.sql.files.maxPartitionBytes` (128 MB). About 31 files share a task, and Spark still schedules around 16,000 tasks instead of 50.

KEEPING IT FIXED

Running OPTIMIZE once does not solve it. Streaming or frequent writes keep recreating small files. Schedule OPTIMIZE as maintenance, or use Auto Compaction, or Liquid Clustering, which compacts incrementally.

OPTIMIZE and Bin-packing.

LAYER **FILES** TABLE STREAM COMPUTE GOVERN PIPELINE SERVE OPERATE

`OPTIMIZE table_name` reads a Delta table's small files, merges their contents and writes new, right-sized files. It then records the file additions and removals atomically in the transaction log.

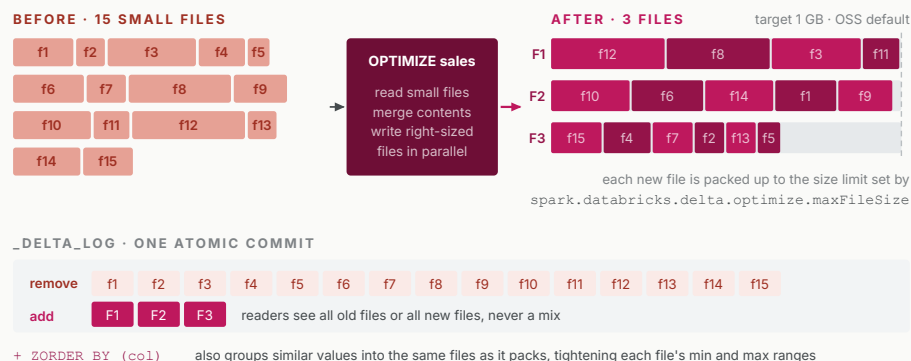


FIG. 7.4 OPTIMIZE packs many small files into a few files near the target size (1 GB by default in open-source Delta; Databricks and Fabric Runtime 2.0 size it from the table, up to 1 GB), rewriting them in parallel, then records every removal and addition in a single atomic commit to the transaction log.

TARGET FILE SIZE

The target is configurable: 1 GB by default in open-source Delta, set with `spark.databricks.delta.optimize.maxFileSize`. Databricks and Fabric Runtime 2.0 size it from the table instead, Databricks from 256 MB up to 1 GB at 10 TB, Fabric from 128 MB up to 1 GB. The rewrite runs in parallel across the cluster. Add `ZORDER BY` and it also groups rows with similar values into the same files while it consolidates, which tightens each file's min and max ranges and improves data skipping in the same pass.

WHAT DATABRICKS DOES FOR YOU

Databricks always runs Optimised Writes and Auto Compaction for MERGE, UPDATE and DELETE. It turns Optimised Writes on for CTAS and INSERT into partitioned Unity Catalog tables and on SQL warehouses, and it tunes file sizes for Unity Catalog managed tables. Any other write needs them switched on.

WHAT YOU GAIN

Fewer files means less task scheduling overhead, less Parquet footer parsing and fewer object-store metadata requests on every scan. On a table with hundreds of thousands of small files, OPTIMIZE can cut query planning time by an order of magnitude and take a large bite out of total scan time before any skipping happens.

RELATED CONCEPTS

Data Skipping is amplified by the sorted layout OPTIMIZE with ZORDER creates. Liquid Clustering clusters incrementally, on large writes and on each `OPTIMIZE`, and Predictive Optimization can schedule that `OPTIMIZE` for you.

Data Skipping.

LAYER **FILES** **TABLE** STREAM COMPUTE GOVERN PIPELINE SERVE OPERATE

Delta skips whole files during a scan when the per-file column statistics in the transaction log show the filter cannot match any row in them. It works at the file level, below partition pruning and above row-group skipping inside Parquet.

WHAT DELTA RECORDS

Each file Delta writes gets its min and max values recorded in `_delta_log`. By default it does this only for the first 32 columns (`delta.dataSkippingNumIndexedCols`), or for the columns you list in `delta.dataSkippingStatsColumns`. Filters on later columns get no file skipping. The exception is Unity Catalog managed tables with Predictive Optimization, where Databricks picks the statistics columns from the columns used in query filters, with no 32-column limit.



FIG. 7.5 Delta records each file's min and max for its indexed columns (the first 32 by default, except on Unity Catalog managed tables with Predictive Optimization) in the transaction log. The reader compares the filter to those ranges and opens only files that could match, so tight, clustered ranges skip far more files than wide, unsorted ones.

AT QUERY TIME

The reader checks your filter against each file's ranges and drops every file where the predicate cannot match; those files are never opened. Z-Order or Liquid Clustering tightens the ranges, so more files drop out.

WHAT YOU GET

On a well-clustered table, a filter on a non-partition column can skip 90% or more of the files. It is the main reason Delta is fast on filters where partitioning was impractical.

Z-Ordering.

LAYER **FILES** TABLE STREAM COMPUTE GOVERN PIPELINE SERVE OPERATE

`OPTIMIZE ... ZORDER BY (col1, col2)` sorts the data on several columns at once so that rows with similar values in those columns land in the same files. It maximises data skipping for queries that filter on those columns.

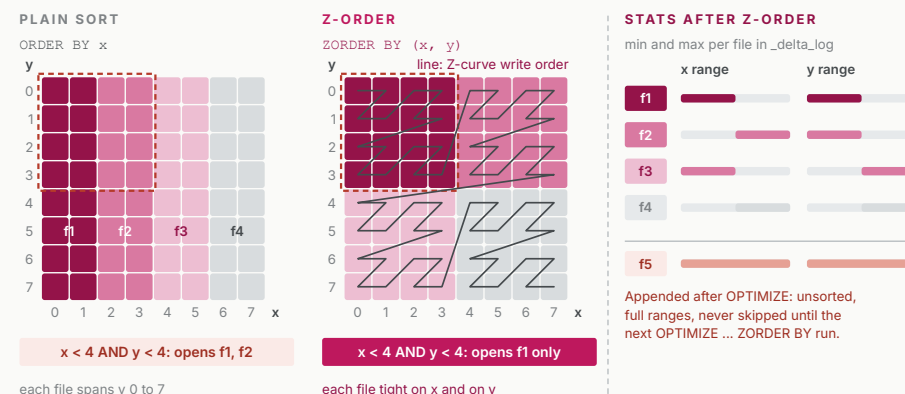


FIG. 7.6 An 8 by 8 grid of (x, y) values split into four files. Sorting by x alone gives files that are tight on x but span all of y; the Z-curve keeps both columns tight, so a filter on x and y opens one file instead of two.

WHAT THE SORT DOES

Z-Order maps multi-dimensional values onto a one-dimensional curve (the Z-curve or Morton curve) that preserves locality, so rows with similar values on both columns end up physically adjacent in the sorted output. Files written after Z-Ordering carry tighter min/max ranges per column in the transaction log statistics, and more of them can be skipped at query time.

RUNNING IT

Z-Ordering is a one-time sort applied at OPTIMIZE time. You run `OPTIMIZE` explicitly and re-run it as new data arrives.

BETWEEN RUNS

Files written since the last OPTIMIZE are unsorted and get no skipping benefit. On frequently updated tables, skipping effectiveness degrades between OPTIMIZE runs.

V-Order.

LAYER **FILES** TABLE STREAM COMPUTE GOVERN PIPELINE **SERVE** OPERATE

Fabric can apply an extra sort to Parquet files as Spark writes them, producing a layout that Power BI Direct Lake and other columnar readers can consume at maximum speed. V-Order is a Parquet write optimisation specific to Microsoft Fabric, not a Delta feature, and it is transparent to standard Parquet readers.

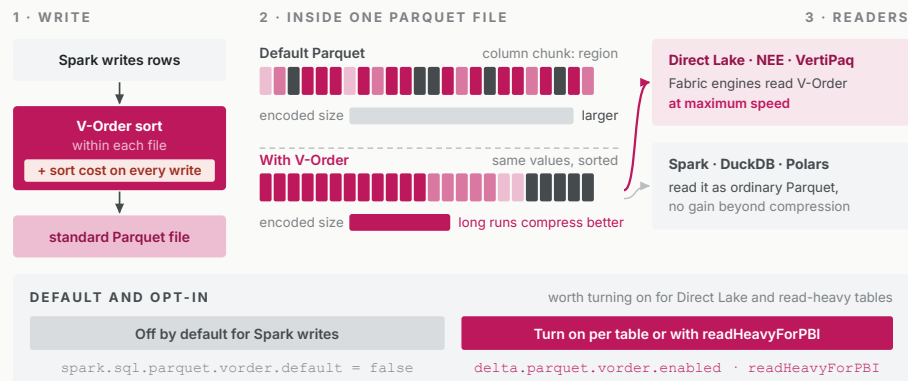


FIG. 7.7 With V-Order enabled, Fabric sorts rows inside each Parquet file as it writes, producing long runs of equal values that compress well. The file stays standard Parquet. V-Order is off by default for Spark writes in new workspaces (Warehouse writes V-Order by default) and is turned on per table or through the readHeavyForPBI resource profile.

WHAT IT WRITES

It is off by default in new Fabric workspaces, where `spark.sql.parquet.vorder.default` is `false`. With it on, Spark sorts the data within each Parquet file during the write, using a special ordering that optimises column-level compression. The files are valid Parquet that any reader can open.

WHO GETS THE SPEED-UP

The speed advantage only shows up on the Fabric Direct Lake / NEE / VertiPaq path. Spark, DuckDB, Polars and other readers get nothing from V-Order beyond ordinary compression.

TURNING IT ON

You pay for the sort at write time, and writes often take around 15% longer. New workspaces default to the `writeHeavy` resource profile, which leaves V-Order off. For read-heavy tables, especially those served through Direct Lake, turn it on per table with `delta.parquet.vorder.enabled` or switch to the `readHeavyForPBI` resource profile.

Liquid Clustering.

LAYER FILES TABLE STREAM COMPUTE GOVERN PIPELINE SERVE OPERATE

Declare cluster columns and Delta keeps rows with similar values in those columns in the same files, reorganising through incremental `OPTIMIZE` runs with no full rewrite. Liquid Clustering replaces static partitioning and Z-Ordering in Delta Lake, and you are not locked into a fixed choice of partition columns.

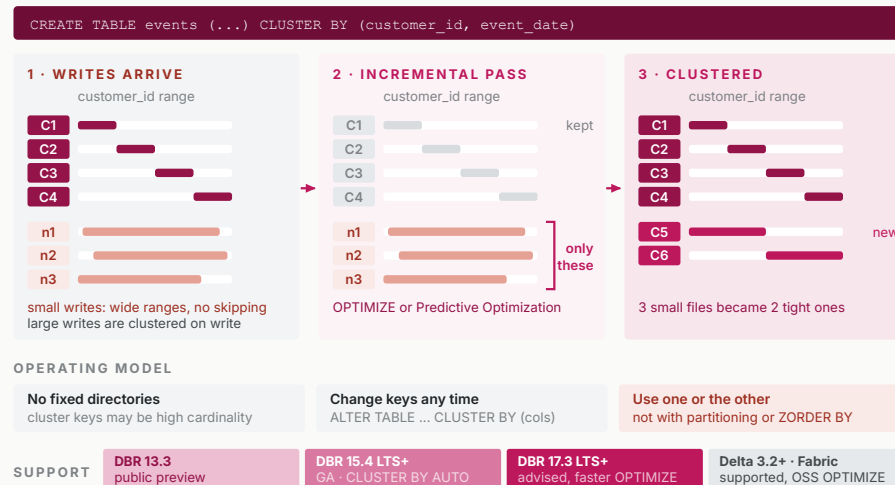


FIG. 7.8 With CLUSTER BY, small writes land as unclustered files with wide ranges (writes above a size threshold are clustered on write). OPTIMIZE, run on a schedule or by Predictive Optimization, rewrites only those files into tight, clustered ones and leaves already-clustered files alone, so no full-table rewrite or fixed partition directories are needed.

SETTING IT UP

Declare the columns when you create the table: `CREATE TABLE ... CLUSTER BY (col1, col2)`. As writes arrive, Delta tracks which files need clustering. It clusters on write only when a write exceeds a size threshold (for example 64 MB for one clustering column on a Unity Catalog managed table); everything else waits for `OPTIMIZE` or Predictive Optimization, and Databricks recommends running `OPTIMIZE` regularly. Each run is idempotent and incremental: it reorganises only newly written or unclustered files, not the full table. To change the keys, run `ALTER TABLE ... CLUSTER BY`; files the change does not affect are not rewritten.

CHOOSING COLUMNS

Pick the columns queries filter or join on most often. High-cardinality columns such as timestamps and user IDs work well, and Liquid Clustering handles them better than Z-Order does. Leave out columns nobody filters on. Two to four cluster columns is typical; beyond that, returns diminish. Cluster columns do not need to be partition columns, and a table using Liquid Clustering should not also use partitioning.

LETTING DATABRICKS PICK KEYS

With `CLUSTER BY AUTO`, Databricks picks the clustering keys and adapts them over time. Predictive Optimization analyses the table's historical query workload, picks candidate columns and runs incremental clustering through `OPTIMIZE`. It changes keys only when the predicted data skipping savings outweigh the cost of reclustering, and it skips selection when the table is too small, already well clustered or rarely queried. You get it in Databricks SQL and DBR 15.4 LTS and above for Unity Catalog managed Delta tables, materialised views and streaming tables, and on DBR 18.0 and above for managed Iceberg v3 tables (not Iceberg v2). Turn it on with `CREATE TABLE t (...) CLUSTER BY AUTO` or `ALTER TABLE t CLUSTER BY AUTO`. Setting `CLUSTER BY (c1, c2)` first gives the selector a starting hint, `CLUSTER BY NONE` turns clustering off, and `CLUSTER BY (cols)` switches back to manual keys. From PySpark, pass `.option("clusterByAuto", "true")` on `saveAsTable`, `writeTo` or `writeStream`; with the batch DataFrame writer it works only in `overwrite` mode. To check it, run `DESCRIBE TABLE` or `SHOW TBLPROPERTIES` and look for `clusterByAuto = true` and the current `clusteringColumns`. If no keys were chosen, the Catalog Explorer History tab shows an `AUTO LIQUID` row marked Not applied, with the skip reason.

Versions and moving off Z-Order

Databricks Runtime 13.3 has it in preview, with manual `OPTIMIZE` required; 15.4 LTS and later has it in general availability. Open-source Delta Lake 3.1 needs a preview flag; 3.2 and later support it, with clustering run by explicit `OPTIMIZE`. Microsoft Fabric supports it. Below DBR 13.3 or Delta 3.1 the feature is unavailable and tables must use partitioning plus Z-Order. To migrate a Z-Ordered table, run `ALTER TABLE ... CLUSTER BY (cols)`. New writes are clustered; existing files stay Z-Ordered until `OPTIMIZE FULL` rewrites them. On heavily written tables the migration is usually free over time, as new data comes to dominate. On static tables you may need an `OPTIMIZE FULL` to see the benefit. You cannot combine Liquid Clustering with `ZORDER BY` in the same `OPTIMIZE` call. Z-Order with periodic `OPTIMIZE` is still valid on DBR below 13.3, on tables whose stable query patterns, infrequent writes and existing Z-Order layout do not justify a migration, and on tables with file counts so small that no clustering gains much.

WHERE IT GOES WRONG

Cluster on columns no query filters on and the clustering effort is wasted. Combining Liquid Clustering with partitioning is not supported; the table uses one or the other. Only the declared cluster columns benefit, so queries filtering on other columns rely on file-level stats alone. `CLUSTER BY AUTO` has its own traps. Running `CREATE OR REPLACE` without it turns automatic clustering off and drops the selected columns. Enabling it with Predictive Optimization off means nothing selects keys or clusters. Key selection and clustering run as serverless jobs billed under the serverless jobs SKU. And `ALTER TABLE` cannot change keys on materialised views or streaming tables.

RELATED CONCEPTS

Predictive Optimization, Z-Ordering, Data Skipping, `OPTIMIZE` and Bin-packing, Unity Catalog.

Delta compaction strategies.

LAYER **FILES** TABLE STREAM COMPUTE GOVERN PIPELINE SERVE OPERATE

Streaming writes, frequent batch appends and over-partitioning all leave Delta tables full of small files. Several mechanisms compact them, and they differ in what triggers them and what they cost. Knowing which fires when keeps tables fast without manual maintenance.

SPOTTING SMALL FILES

Look for queries with a high task count and short tasks (under 1 second), long directory listing times, and metadata overhead dominating reads. A table whose files average under 16 MB is a candidate for compaction. Streaming ingest is the most common source of small-file proliferation.

RUNNING OPTIMIZE YOURSELF

Explicit and on demand: `OPTIMIZE table_name [WHERE predicate] [ZORDER BY (cols)]`. It reads the small files, merges them into files near the target size and commits atomically to the transaction log. The target is 1 GB in open-source Delta (configurable with `spark.databricks.delta.optimize.maxFileSize`); Databricks and Fabric Runtime 2.0 size it from the table. Run it as scheduled maintenance or after a large batch load. On big tables, a WHERE clause narrowing it to recent partitions cuts the cost sharply.



FIG. 7.9 Four compaction mechanisms on the same series of writes. Optimised Writes prevents small files during each write, Auto Compaction cleans up after a commit once a partition passes 50 files, OPTIMIZE runs on a schedule to a target file size (1 GB in open-source Delta, sized from the table on Databricks and Fabric Runtime 2.0), and Liquid Clustering reclusters incrementally: large writes are clustered on write, the rest by OPTIMIZE on a schedule or through Predictive Optimization.

COMPACTING AFTER EACH WRITE

Auto Compaction runs automatically after the write. Set

`delta.autoOptimize.autoCompact = true` on the table. After every successful write commit, Spark inspects the partitions that commit touched and runs a small compaction pass if the file count exceeds a threshold (default 50 files in a partition). It is cheaper than a full OPTIMIZE but targets smaller files, and suits streaming sinks with frequent small writes.

WRITING FEWER FILES

Optimised Writes act during the write. Set `delta.autoOptimize.optimizeWrite = true` on the table and Spark repartitions the output first, producing fewer, larger files. Each write pays for a small shuffle, but no small files get created. Pair it with Auto Compaction on streaming pipelines.

CLUSTERING AS YOU GO

Liquid Clustering is incremental, on large writes or through OPTIMIZE. Declaring `CLUSTER BY (cols)` on the table replaces partition-and-Z-Order with incremental reclustering. Writes above a size threshold are clustered on write; `OPTIMIZE` clusters the rest, run on a schedule or by Predictive Optimization. It is the recommended layout for new tables.

PICKING ONE

For streaming or frequent small writes, enable Optimised Writes and Auto Compaction when you create the table. For batch ETL with large daily loads, schedule OPTIMIZE after the load with a WHERE clause narrowed to that day. For analytical tables with stable query patterns, use Liquid Clustering on the columns most queries filter by. Manual Z-Order is still valid for tables that cannot move to Liquid yet.

Predictive Optimization.

LAYER FILES **TABLE** STREAM COMPUTE GOVERN PIPELINE SERVE OPERATE

Predictive Optimization runs `OPTIMIZE`, `VACUUM` and statistics maintenance on your Delta tables automatically, replacing manually scheduled maintenance jobs. It is a Databricks Unity Catalog feature that you enable at the catalog, schema or table level, and it needs Unity Catalog and a supported Databricks Runtime.

WHAT IT RUNS

Once enabled on a table, Databricks watches query history and write patterns. It schedules `OPTIMIZE` when small files accumulate or when compaction would help query latency, runs `VACUUM` at safe intervals that respect the retention setting, and refreshes statistics for the cost-based optimiser. That covers small-file compaction, incremental Liquid Clustering on clustered tables (it does not run `ZORDER`), statistics collection for CBO and stale-file cleanup via `VACUUM`. Everything runs as managed background jobs on the catalog's default serverless compute, billed to the workspace. It does NOT change cluster columns on Liquid tables (except those declared with `CLUSTER BY AUTO`), alter the schema or modify table properties; those stay explicit operations.

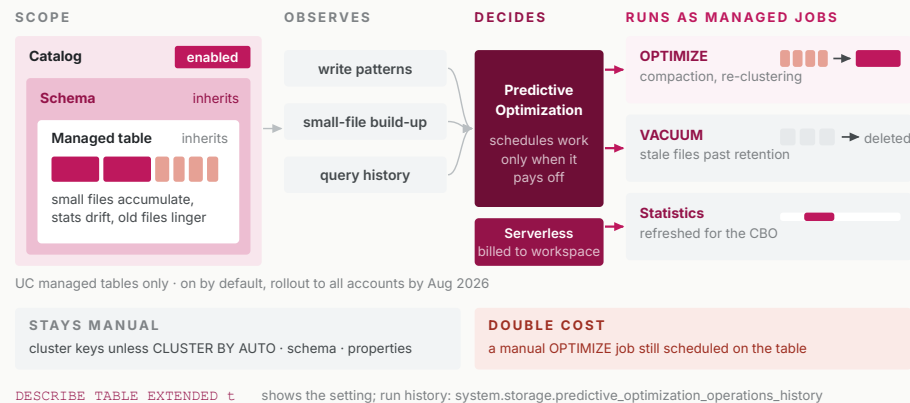


FIG. 7.10 Enabled at catalog, schema or table level, Predictive Optimization observes write patterns and query history on Unity Catalog managed tables (on by default for accounts created from 11 November 2024, with existing accounts enabled in a rollout due to finish by August 2026), then runs OPTIMIZE, VACUUM and statistics refresh as serverless background jobs within the layout you declared.

WHO HAS IT BY DEFAULT

Accounts created on or after 11 November 2024 have it on by default. Databricks has been enabling existing accounts in a gradual rollout it expected to complete by August 2026, so check the account console for yours. It applies only to Unity Catalog managed tables (Delta and Iceberg), never to external tables. On tables declared with `CLUSTER BY AUTO` (automatic Liquid Clustering, DBR 15.4 LTS and above) it also selects the clustering keys from query history, and changes them only when the expected savings outweigh the cost of reclustering. Tables that must keep hand-picked keys or custom maintenance can opt out at the catalog, schema or table level.

CHECKING IT RUNS

Run `DESCRIBE TABLE EXTENDED` (or `DESCRIBE SCHEMA EXTENDED`, `DESCRIBE CATALOG EXTENDED`) and look at the Predictive Optimization setting. Then query `system.storage.predictive_optimization_operations_history`; it lists each automatic `OPTIMIZE`, `VACUUM` and `ANALYZE` run with its cost.

WHERE TO TURN IT ON

Make it the default for analytical tables in production: enable it at catalog level for the Gold and Silver layers of a medallion architecture. You drop the operational tax of maintenance scheduling and the risk of forgotten compaction. Cost is harder to plan. The work runs as managed serverless jobs, so you are billed continuously, not in planned spikes. On a high-write table it can run often; on a low-write or mostly-read table, rarely. You will not know the exact cost without a test period. Skip it for tables with very specialised maintenance needs, such as custom Z-Order column rotation or custom VACUUM windows for forensic retention. For Bronze ingest tables with rapid micro-batch writes, weigh whether Optimised Writes and Auto Compaction at table level remain preferable.

WHAT CATCHES PEOPLE

Keeping a scheduled `OPTIMIZE` job on a table after enabling Predictive Optimization doubles the cost for no benefit; disable the manual job. It also does not manage partitioning or clustering choices. It works within the layout you declared: if the cluster columns are wrong, it cannot fix that unless the table uses `CLUSTER BY AUTO`.

ACID Transactions.

LAYER FILES **TABLE** STREAM COMPUTE GOVERN PIPELINE SERVE OPERATE

Delta gives you full ACID guarantees on table operations. A write commits fully or not at all (atomicity), the table is always in a valid state (consistency), and committed writes survive failures (durability). For isolation, readers get snapshot isolation and writes default to `WriteSerializable`; set `delta.isolationLevel` to opt in to the stricter `Serializable`.

HOW A COMMIT LANDS

Delta does this through the transaction log, with no traditional locking. A writer stages its Parquet files first, then commits a JSON entry to the log atomically. If that commit fails, the staged files are orphaned and no reader ever sees them.

WHAT READERS SEE

A reader opens a snapshot of the log when its query starts and sees only files committed before that point. No reader blocks a writer, and no writer blocks a reader.

COMMIT PROTOCOL · STAGED FILES BECOME VISIBLE ONLY AT COMMIT

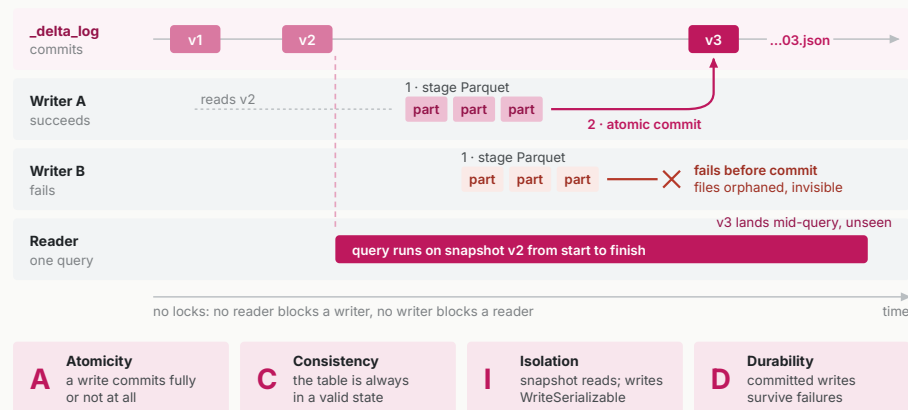


FIG. 8.1 Writers stage Parquet files first and become visible only when their JSON commit lands in the `_delta_log`; a writer that fails before committing leaves orphaned files no reader sees. A reader pins the snapshot it opened, so readers and writers never block each other.

WHEN A WRITE DIES HALFWAY

Without ACID, a failed write leaves the table partially updated and silently corrupt, and concurrent readers can see inconsistent data mid-write. With it, several production pipelines can safely read and write the same table at once.

Transaction Log.

LAYER **FILES** TABLE STREAM COMPUTE GOVERN PIPELINE SERVE OPERATE

Every Delta table has a `_delta_log` directory at its root, and that directory is the authoritative source of truth for the table's state. Each operation (appended or removed files, schema changes, metadata updates) lands there as a JSON commit file. The table's current state is the cumulative effect of all committed log entries.

WHAT A COMMIT FILE HOLDS

Each commit is a numbered JSON file, such as `00000000000000000001.json`. It records which files were added, which were removed, the schema at that version, and per-file statistics: min, max and null count per column.

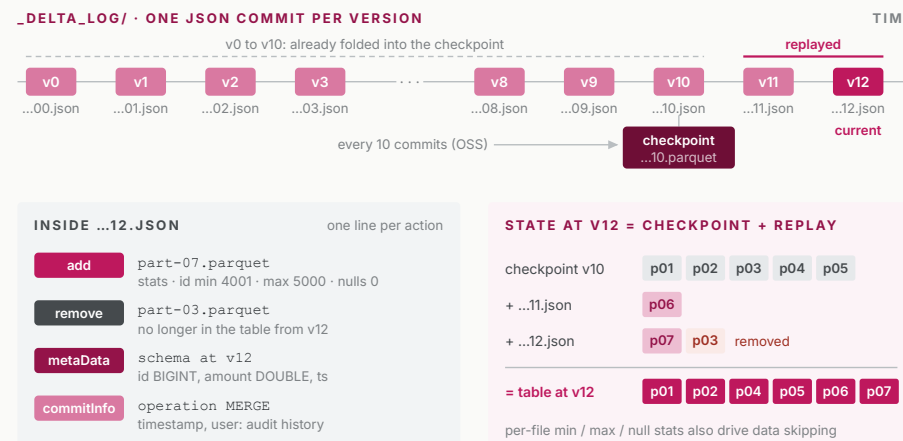
`_DELTA_LOG/` · ONE JSON COMMIT PER VERSION

FIG. 8.2 The `_delta_log` holds one numbered JSON commit per version, each listing files added and removed, the schema and per-file stats. Every 10 commits by default in open-source Delta (Databricks tunes the interval) a Parquet checkpoint captures the full state, so a reader loads the latest checkpoint and replays only the commits after it.

CHECKPOINTS

Every 10 commits by default in open-source Delta, Delta checkpoints the full table state into a Parquet file, so readers do not have to replay the whole log history from the beginning. Databricks tunes that interval to data size and workload.

WHAT READS FROM IT

Time travel (query any prior version), data skipping (the per-file stats live here), schema evolution tracking and audit history all come out of the log, and every other Delta Lake feature sits on it too. When you want to know why Delta operations are fast, how conflicts are detected or what VACUUM actually removes, look at the log.

Time Travel.

LAYER **FILES** **TABLE** STREAM COMPUTE GOVERN PIPELINE SERVE OPERATE

Every committed operation creates a new version of a Delta table, and you can query the table as it was at any earlier version or timestamp. Delta reads the transaction log to work out exactly which files made up the table at that point, then scans only those files.

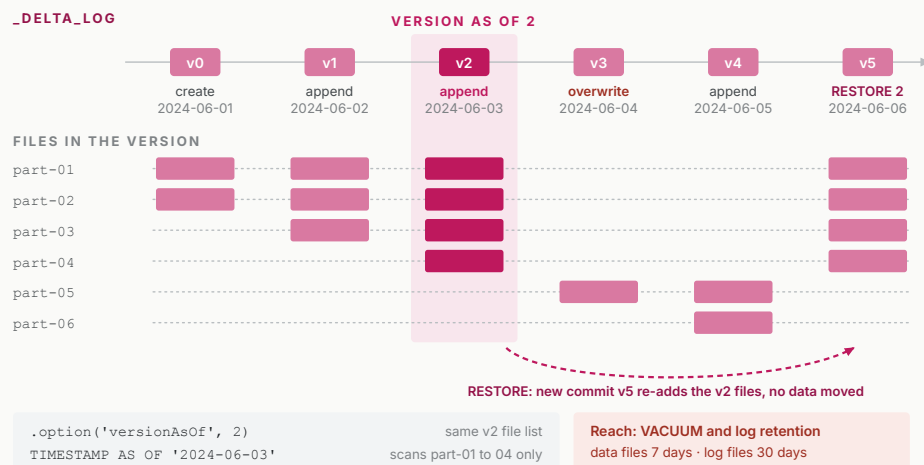


FIG. 8.3 Each commit defines which data files make up that version. Reading VERSION AS OF 2 rebuilds the v2 file list from the log and scans only those files; RESTORE writes a new commit that points back at them without moving data. In open-source Delta, reach ends where VACUUM has removed data files (7 days) or log cleanup has removed commits (30 days); on Databricks Runtime 18 and above, and on Unity Catalog managed tables from 12.2, time travel is also capped at `deletedFileRetentionDuration`, 7 days by default.

QUERYING AN OLD VERSION

By version: `df = spark.read.format('delta').option('versionAsOf', 5).load(path)`. By timestamp: `.option('timestampAsOf', '2024-06-01')`. In SQL: `SELECT * FROM table VERSION AS OF 5` or `SELECT * FROM table TIMESTAMP AS OF '2024-06-01'`. Delta rebuilds the file list for that version from the log and scans those files.

WHAT PEOPLE USE IT FOR

Typical production uses: reproducing a model training dataset from a specific date; auditing what data existed before a bad write, or reproducing past query results for a regulatory audit; debugging, by comparing the current table with a prior version to find when a data quality issue came in; and slowly changing dimension pipelines that compare current and historical state without maintaining separate snapshot tables. For incremental processing, read only the rows changed since a known version with Change Data Feed.

ROLLING BACK

To recover from an accidental deletion, overwrite or bad transformation, run `RESTORE TABLE table_name VERSION AS OF n`. It atomically reverts the table to that version in the transaction log without moving any data files. Before you issue `RESTORE TABLE`, run a time-travel query against the prior version so you know what changed.

HOW FAR BACK YOU CAN GO

The data files must still exist on storage. In open-source Delta, VACUUM removes old files and so sets the earliest version you can query. On Databricks Runtime 18 and above, and on Unity Catalog managed tables from 12.2, time travel is also blocked past `delta.deletedFileRetentionDuration` (7 days by default), whether or not VACUUM has run. The default VACUUM retention is 7 days: files older than that are deleted, and those versions become unqueryable even if the log entry still exists. If you want time travel for long-term archival, set `delta.logRetentionDuration` and `delta.deletedFileRetentionDuration` explicitly on the tables that need longer windows.

WHAT YOU PAY FOR IT

Time travel reads cost nothing extra; Delta keeps the old files until VACUUM runs. You pay in storage. A table that overwrites half its rows daily holds two copies of those rows until retention expires. Default retention is 7 days for data files and 30 days for log files.

RELATED CONCEPTS

Transaction Log stores the version history that makes time travel possible. VACUUM and Retention controls how far back time travel can reach.

Schema Enforcement and Evolution.

LAYER FILES **TABLE** STREAM COMPUTE GOVERN PIPELINE SERVE OPERATE

Schema enforcement (also called schema validation) rejects a write whose schema does not match the target table. **Schema evolution** updates the table schema automatically when a write brings in new columns. You control both per write operation: enforcement is the default and the recommended posture for production tables, and evolution is opt-in for specific writes or sessions.

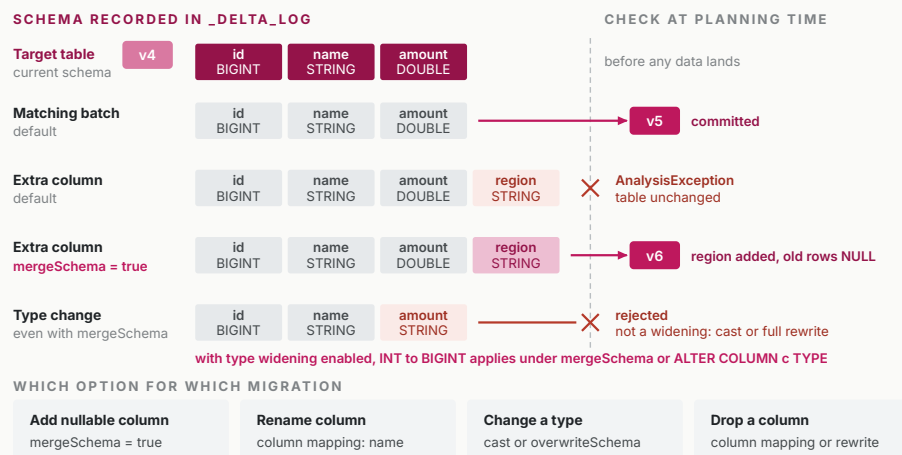


FIG. 8.4 Every write is checked against the schema recorded in the `_delta_log` before any data lands. A mismatch fails with `AnalysisException` and leaves the table unchanged; `mergeSchema` lets a new column through in the same commit, but a lossy type change is rejected either way; only safe widening such as INT to BIGINT passes, when type widening is enabled on the table.

WHEN A WRITE IS REJECTED

On every write Delta compares the source DataFrame schema with the table schema in the transaction log. Extra columns, missing required columns or mismatched types throw `AnalysisException` before any data lands. The check runs at planning time, not after partial writes, so a failed write leaves the table unchanged. You hear about an upstream contract violation the moment the malformed batch arrives.

LETTING NEW COLUMNS IN

Turn evolution on per write with `.option("mergeSchema", "true")` on the DataFrame writer, or for the whole session with `spark.databricks.delta.schema.autoMerge.enabled = true`. With `mergeSchema` on, **new columns are added** to the table schema and existing rows get NULL in them. The schema change is recorded in the transaction log as part of the same commit. Existing column types are **not** widened or changed unless the table has type widening enabled.

Changing types and dropping columns

Type changes on existing columns are rejected even with `mergeSchema`, unless type widening is on. **Type Widening** (GA in DBR 15.4 LTS and open-source Delta Lake 4.0) allows safe widening, int to long or float to double, once you enable it with a table property. After that it applies automatically under `mergeSchema`, or explicitly with `ALTER TABLE t ALTER COLUMN c TYPE BIGINT`. Making a column nullable is a separate change: `ALTER COLUMN c DROP NOT NULL`. Narrowing or lossy casts always need a full table rewrite. For breaking changes that evolution cannot handle (column renames, type changes, drops), `.option("overwriteSchema", "true")` with `.mode("overwrite")` replaces both data and schema in one commit. If column meanings change, older versions are no longer readable through time travel.

PICKING THE OPTION

Adding a nullable column: `mergeSchema=true` on write. Renaming a column without rewriting data: column mapping (`delta.columnMapping.mode=name`). Changing a type that needs re-encoding: an explicit cast on write, or `overwriteSchema=true` with a full rewrite. Dropping a column: column mapping or a full table rewrite.

WHERE AUTOMERGE BITES

The usual accident is turning `autoMerge` on for the session and forgetting about it. A malformed source with a typo in a column name then silently adds `amount_v2` as a new column instead of failing loudly. Over weeks the table drifts and downstream queries break. Keep `autoMerge` opt-in per write, and let enforcement be the default contract between pipelines.

Writing data correctly.

LAYER FILES TABLE STREAM COMPUTE GOVERN PIPELINE SERVE OPERATE

Every `df.write` has to say what happens to data already at the target. The four save modes are `errorIfExists` (the default, alias `error`), `append`, `overwrite`, and `ignore`, which writes nothing if data exists. A correct write also picks the overwrite scope, the target (table name or path) and a way to survive reruns without duplicates.

FILES VERSUS DELTA

For plain file formats the save modes use no locking and are not atomic. `overwrite` deletes the old files before writing the new ones, so a failed job can leave a half-empty directory. Delta commits each write as one transaction. Readers see the old version until the commit, and the old files stay until `VACUUM`, so `RESTORE` can undo a bad overwrite.

SAVE MODE · TARGET ALREADY HOLDS DATA

errorIfExists the default: throws an error	append adds the rows	overwrite replaces the data	ignore writes nothing
--	--------------------------------	---------------------------------------	---------------------------------

OVERWRITE SCOPE · DELTA TABLE PARTITIONED BY DATE



RERUN OF AN APPEND · DELTA LOG VERSIONS

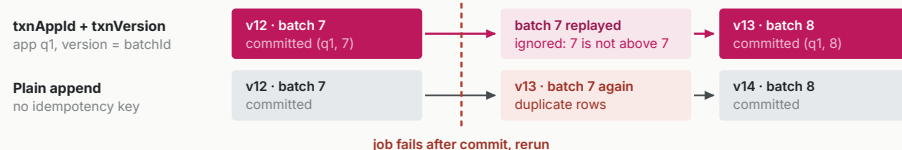


FIG. 8.5 Overwriting a date-partitioned Delta table with one day of new rows: a plain overwrite leaves only that day, while `replaceWhere` and dynamic partition overwrite replace just the matching slice. Below, a replayed batch is skipped when the write carries `txnAppId` and `txnVersion`, and appended twice when it does not.

SCOPING AN OVERWRITE

On Delta, plain `overwrite` replaces the whole table. `replaceWhere` atomically replaces only rows matching a predicate, which need not be on partition columns, and by default fails if any new row falls outside it. Dynamic partition overwrite (`partitionOverwriteMode=dynamic`, Delta 2.0+) replaces only the partitions that receive rows. Delta 4.3 adds the `replaceUsing` and `replaceOn` options.

SURVIVING RERUNS

Jobs and micro-batches get retried, and rerunning an append that already committed writes the same rows twice. Delta's `txnAppId` (a stable string) and `txnVersion` (a monotonically increasing number) make a write idempotent: Delta records the pair and ignores any write whose version is not above the last one recorded for that app ID. `foreachBatch` is only at-least-once, so pass a fixed ID as `txnAppId` and `batchId` as `txnVersion`, and a replayed batch is skipped.

WHERE WRITES GO WRONG

Refreshing one day of a partitioned table with plain `overwrite`.

`spark.sql.sources.partitionOverwriteMode` defaults to `static`, so the whole table becomes that one day. Dynamic overwrite with one stray row, which replaces that entire partition. Appending in a retried job with no idempotency key. Deleting a streaming checkpoint but keeping the same `txnAppId`: every write from the restarted query is ignored, and the data is silently lost.

ON DATABRICKS AND FABRIC

On Databricks, `saveAsTable("catalog.schema.table")` creates a Unity Catalog managed Delta table. Managed tables cannot be accessed by path. Dropping one deletes its files after a 7-day recovery window (`UNDROP TABLE`); dropping an external table leaves its files. For new work Databricks recommends `REPLACE USING` (SQL from Runtime 16.3, the `replaceUsing` option from 18.2) or `replaceWhere`. Dynamic partition overwrite is Public Preview, classic compute only, partitioned tables only, and cannot be combined with `replaceWhere` or `overwriteSchema`. On Fabric, `saveAsTable` writes a managed Delta table under `Tables/`. Only Delta tables there are auto-registered and visible to the SQL analytics endpoint, so put Parquet and CSV under `Files/`. Runtime 2.0 ships Delta 4.2, so use `replaceWhere` or dynamic partition overwrite.

RELATED CONCEPTS

ACID Transactions, Transaction Log, Time Travel, Schema Enforcement and Evolution, MERGE / Upsert, Output Modes and Exactly-once, Streaming checkpoints, Unity Catalog, OneLake.

MERGE / Upsert.

LAYER FILES **TABLE** STREAM COMPUTE GOVERN PIPELINE SERVE OPERATE

MERGE does insert, update and delete in one atomic pass over a Delta table. It matches source rows against the target on a join condition and applies a different action for each match outcome. It is the standard tool for upserts, CDC application and SCD Type 2 patterns.

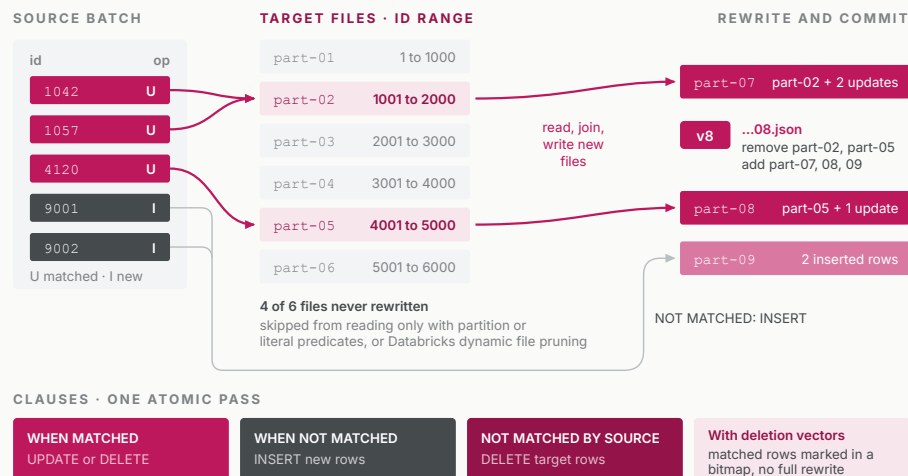


FIG. 8.6 MERGE uses the join condition and per-file id ranges to find the target files that hold matched rows, rewrites only those files with updates applied, writes unmatched source rows to a new file, and swaps them in with one atomic commit. Files with no matches are never rewritten; open-source Delta still reads them unless the ON clause has partition or literal predicates, while Databricks can skip them with dynamic file pruning.

WHAT DELTA DOES WITH IT

You write `MERGE INTO target USING source ON condition WHEN MATCHED THEN UPDATE ... WHEN NOT MATCHED THEN INSERT ...`. Delta uses the join condition to find the target files that contain matched rows, reads those files, joins them with the source, writes new files with the merged result and atomically commits the file replacements in the transaction log. Files with no matched rows are never rewritten. Whether they are read depends on the `ON` clause. Open-source Delta scans the whole target to find matches unless the condition holds literal or partition predicates; Databricks can also skip files using the source keys (dynamic file pruning). With deletion vectors enabled, MERGE marks changed rows in a small bitmap instead of rewriting whole files; see Deletion Vectors.

SCOPING THE ON CLAUSE

MERGE cost is dominated by how much of the target has to be scanned, not by the size of the source. Match on the partition or clustering column and Delta reads only the relevant files. On a date-partitioned table, `ON target.date = source.date AND target.id = source.id` reads only the partitions present in the source. On a non-partitioned table, a broad condition like `ON target.id = source.id` reads every file.

DELETES AND CDC INPUT

Standard MERGE only acts on rows in the source, so deletions in the source are invisible to the target. `WHEN NOT MATCHED BY SOURCE` (Delta 2.3+) acts on target rows with no source match. In full-refresh CDC patterns it is typically a delete for rows no longer in the source system. For CDC input with an operation column, the canonical pattern is `WHEN MATCHED AND source.op = "D" THEN DELETE WHEN MATCHED AND source.op = "U" THEN UPDATE SET * WHEN NOT MATCHED AND source.op = "I" THEN INSERT *`. Combine it with Change Data Feed on the target to pass the changes downstream incrementally; see Change Data Feed.

WHERE IT GOES SLOW OR FAILS

A broad join condition that matches most of a large unpartitioned target rewrites most files on every run, so the MERGE costs as much as a full overwrite. Partition or cluster the target by a column the source naturally filters on. Calling MERGE in a tight loop, one row per call, pays planning and commit overhead on every row; batch source rows into larger chunks first. Duplicate keys in the source break it: if two source rows match one target row, an update or delete fails with

`DELTA_MULTIPLE_SOURCE_ROW_MATCHING_TARGET_ROW_IN_MERGE`. Deduplicate first and keep the latest row per key.

Deletion Vectors.

LAYER FILES TABLE STREAM COMPUTE GOVERN PIPELINE SERVE OPERATE

With deletion vectors on, a DELETE or UPDATE leaves the Parquet file intact and records the affected rows as a bitmap of row positions in a small deletion vector file. Readers skip the marked rows. The file is physically rewritten later, by `OPTIMIZE`, `REORG TABLE ... APPLY (PURGE)` or Auto Compaction.

WHAT A DELETE WRITES

When a DELETE or UPDATE touches rows in a file, Delta writes a deletion vector alongside that file in the transaction log entry. The Parquet file itself is not touched. At scan time readers apply the vector as a filter and skip every row whose position is marked. Without deletion vectors, every such operation rewrites the whole file: read it, filter the rows, write a new one. Deleting 10 rows from a 1 GB file goes from a 1 GB rewrite to writing a tiny bitmap, which is why frequent small deletes and CDC processing get so much cheaper.

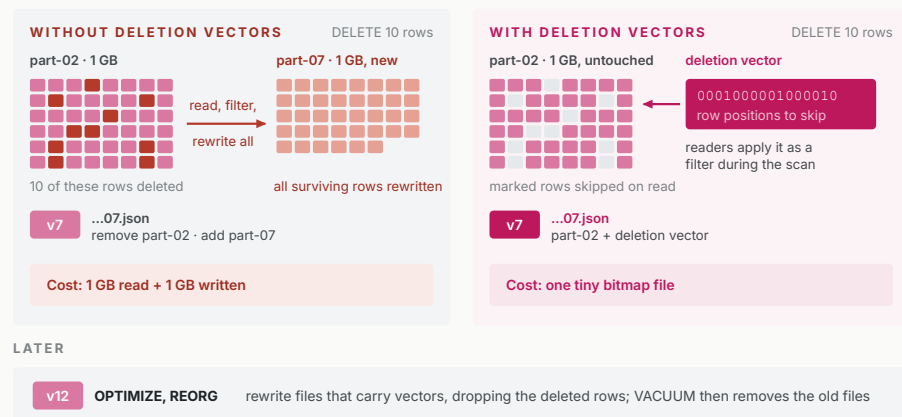


FIG. 8.7 Deleting 10 rows from a 1 GB file without deletion vectors means reading and rewriting the whole file. With deletion vectors the file stays untouched: the commit adds a small bitmap of row positions that readers skip, and a later rewrite drops the vector. OPTIMIZE rewrites only files with enough deleted rows (over 5% in open-source Delta); for erasure, run REORG TABLE ... APPLY (PURGE) and then VACUUM.

WHEN ROWS REALLY GO

Do not count on OPTIMIZE to apply deletions. It rewrites a file that carries a deletion vector only once enough of its rows are deleted (more than 5% by default in open-source Delta). REORG TABLE t APPLY (PURGE) rewrites every such file. This matters for GDPR right-to-erasure: the deleted rows stay in the old Parquet files until REORG TABLE t APPLY (PURGE) rewrites them and VACUUM removes the old files.

ON DATABRICKS

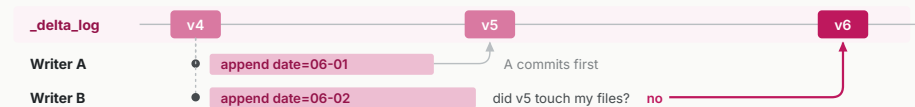
Databricks has announced automatic upgrades that turn deletion vectors on for existing Unity Catalog managed tables from August 2026, once every client reading a table supports them.

Optimistic Concurrency.

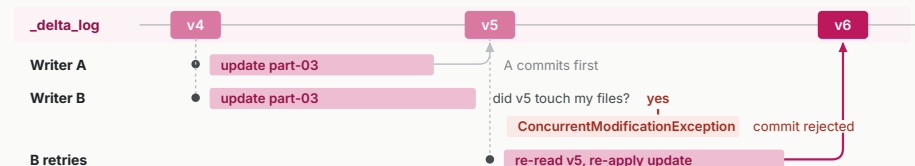
LAYER FILES TABLE STREAM COMPUTE GOVERN PIPELINE SERVE OPERATE

Delta handles concurrent writes to the same table without locking it. Each writer reads the current version, stages its changes and tries to commit. If another writer committed a conflicting change in the meantime, the commit fails and the writer has to retry from the new version.

CASE 1 · APPENDS TO DIFFERENT PARTITIONS



CASE 2 · UPDATES TO THE SAME FILE



Cheap when conflicts are rare
writers on different partitions never wait

Degrades under contention
same-file retries add latency, can starve writers

FIG. 8.8 Two writers both start from v4 and stage their changes without locks. At commit time the second writer checks whether v5 touched files it read or modified: appends to different partitions go through as v6, while an update to the same file fails with ConcurrentModificationException and is retried from v5.

WHAT GETS CHECKED AT COMMIT

At commit time Delta checks whether a concurrent commit has changed any file this writer read or modified since it started. Blind appends never conflict with each other, even into the same partition. A concurrent change to files the writer read raises a specific subclass of ConcurrentModificationException, such as ConcurrentAppendException or ConcurrentDeleteReadException. On Databricks, with deletion vectors and row-level concurrency, updates to different rows of the same file can both succeed.

AFTER A CONFLICT

The losing writer must re-read the updated table, re-apply its changes and attempt the commit again. That is cheap while conflicts are rare, which is the usual case on partitioned tables where different writers target different partitions.

WHEN CONTENTION BUILDS

Put several writers on the same files and it degrades: frequent retries add latency and can starve a writer. For high write rates, prefer append-only writes, partition or cluster so writers touch different files, and keep MERGE conditions narrow.

VACUUM and Retention.

LAYER FILES TABLE STREAM COMPUTE GOVERN PIPELINE SERVE OPERATE

`VACUUM table_name` permanently deletes data files that the current or recent versions of a Delta table no longer reference. It is how you reclaim storage after deletes, updates, MERGE operations and OPTIMIZE rewrites.

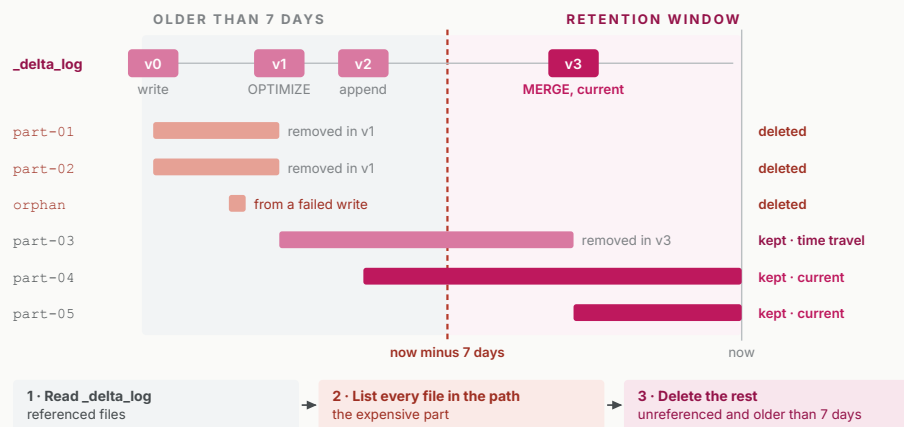


FIG. 8.9 Each bar is a data file from the commit that added it to the commit that removed it. VACUUM keeps every file the current version or a version inside the 7 day retention window still references, and deletes files that are unreferenced and older than the threshold, including orphans from failed writes.

WHAT IT DELETES

VACUUM reads the transaction log to find the files still referenced by the current version, or by time travel within the retention window. Then it lists every file in the storage path and deletes those that are unreferenced and older than the retention threshold: 7 days by default, controlled by `delta.deletedFileRetentionDuration`.

SCHEDULING IT

On large tables the full storage-path listing is the expensive part, so schedule VACUUM during off-peak hours. Skipping it altogether is the other mistake: tables with frequent writes or deletes then grow their storage without bound.

GOING BELOW 7 DAYS

Ask for a retention period shorter than 7 days and Delta blocks it by default, because active Spark jobs may still hold references to recently committed files. You have to disable the safety check to go lower.

Change Data Feed.

LAYER FILES TABLE STREAM COMPUTE GOVERN PIPELINE SERVE OPERATE

Change Data Feed lets downstream consumers read only the rows inserted, updated or deleted since a given table version, without scanning the full table. In effect the Delta table becomes a change stream.

TURNING IT ON

Run `ALTER TABLE t SET TBLPROPERTIES ('delta.enableChangeDataFeed' = 'true')`. Delta records changes from that version onwards. UPDATE, DELETE and MERGE write `_change_data` files holding the before and after image of each modified row; inserts and appends are read straight from the added data files.

READING THE FEED

Read it with `spark.read.format('delta').option('readChangeFeed', 'true').option('startingVersion', n).load(path)`. Each row comes back with its operation type (insert, update, delete) in a `_change_type` column.

WHAT IT SAVES YOU

Without CDF you diff two full table snapshots to find what changed. With it, a pipeline that maintains derived tables, aggregates or search indexes processes only the changed rows, which can cut compute cost by orders of magnitude compared with reprocessing the full table. It is the main building block for incremental processing on Delta tables.

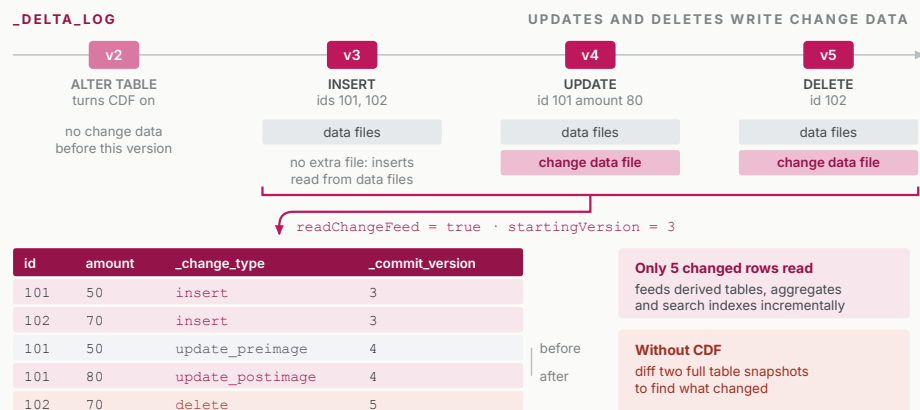


FIG. 8.10 Once `delta.enableChangeDataFeed` is set, UPDATE, DELETE and MERGE commits also write change data files with the row images and operation type; insert-only commits write none, because their rows are read straight from the added data files. Reading with `readChangeFeed` from `startingVersion` 3 returns only the changed rows, tagged with `_change_type` and `_commit_version`.

Type widening and row tracking.

LAYER FILES TABLE STREAM COMPUTE GOVERN PIPELINE SERVE OPERATE

These two Delta table features change what the log records about columns and rows. **Type widening** moves a column to a wider type, such as `INT` to `BIGINT`, without rewriting data files. **Row tracking** gives every row a stable `row_id` and a `row_commit_version` that survive `UPDATE` and `MERGE`.

WIDENING A COLUMN

Set `delta.enableTypeWidening = true`, then run `ALTER TABLE t ALTER COLUMN qty TYPE BIGINT`. Only the schema in the log changes. Old Parquet files keep their narrow type and are upcast on read, so an undersized column is fixed in seconds instead of by a full table rewrite. Delta 4.0 and later supports byte, short and int to wider integers, decimal or double; long to decimal; float to double; decimal to higher precision or scale; and date to `timestamp_ntz`. Under `mergeSchema` or `MERGE ... WITH SCHEMA EVOLUTION` widening happens automatically, except integer to decimal or double, which needs an explicit `ALTER`.

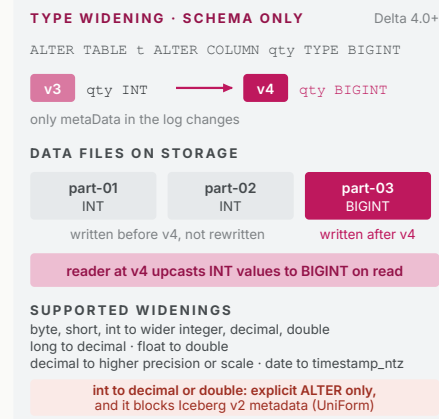
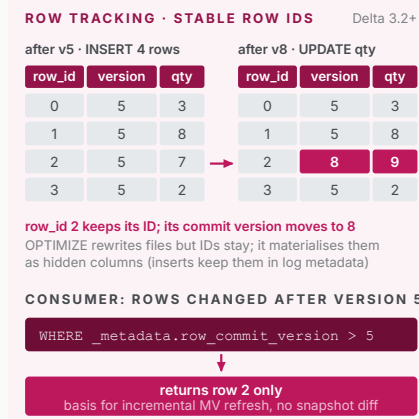


FIG. 8.11 Type widening changes only the schema in the log, so older INT files stay on disk and are upcast on read. Row tracking gives each row an ID that survives UPDATE, plus the version that last changed it, so a consumer can select just the rows changed since version 5.



TRACKING ROWS

Turn it on with `delta.enableRowTracking = true`. For inserts, IDs are derived from per-file metadata in the log; `OPTIMIZE` and rewrites materialise them as hidden columns. Read them with `SELECT _metadata.row_id, _metadata.row_commit_version`. An engine can then tell which rows changed since version N without diffing snapshots. Databricks requires row tracking on source tables for some incremental refresh operations of materialised views, and recommends it together with deletion vectors and Change Data Feed.

VERSION NOTES

Type widening was a preview in Delta 3.2 (feature `typeWidening-preview`) and went GA in Delta 4.0 as `typeWidening`. It is GA on DBR 15.4 LTS and later, with streaming reads from DBR 16.4 LTS. Row tracking needs Delta 3.2 and later, or DBR 14.0 and later. From July 2026, Databricks automatic upgrades turn it on for existing Unity Catalog managed tables once every client reading them supports it. Both features upgrade the table protocol (writer version 7), so older external readers may fail. Setting either property to `false` stops new use but keeps the feature; remove widening with `ALTER TABLE t DROP FEATURE 'typeWidening'`.

BEFORE YOU TURN THEM ON

Widening a UniForm table can block its Iceberg metadata: Iceberg v2 cannot express integer to decimal or double, decimal scale increases, or date to timestamp. Do not rely on `hash()` or `typeof()` results across a type change. Enabling row tracking on a busy table while a stream writes to it triggers `MetadataChangedException`. And do not expect row IDs in a Change Data Feed read; they are not exposed there.

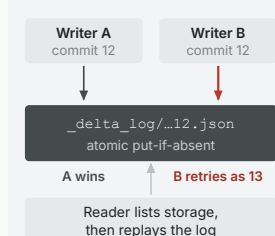
RELATED CONCEPTS

Schema Enforcement and Evolution, Transaction Log, Deletion Vectors, Change Data Feed, Materialised Views.

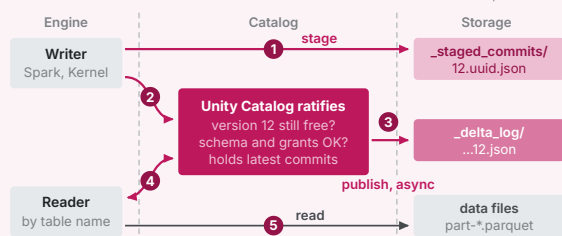
Catalog-managed tables.

LAYER FILES TABLE STREAM COMPUTE GOVERN PIPELINE SERVE OPERATE

The `catalogManaged` Delta protocol feature moves commit coordination from the filesystem to the catalog. The catalog, not an atomic file write in `_delta_log`, decides whether a commit succeeded, and it holds the latest commits and metadata. Clients address the table by name through the catalog instead of by storage path. Databricks calls the feature **catalog commits**.

FILESYSTEM LOG ONLY**LIMITS**

one table per commit
path access bypasses governance
schema changes need only
storage write rights

CATALOG-MANAGED COMMITS

1 stage commit · 2 ask catalog to commit · 3 publish to log
4 reader gets latest version from catalog · 5 read files

one ratified commit: orders v12 payments v40 all or nothing

path-based reads and writes are not supported: every client goes through the catalog

FIG. 8.12 With the filesystem log alone, writers race to create the next JSON file and readers list storage. On a catalog-managed table the writer stages its commit, the catalog ratifies it, and the commit is published to `_delta_log` later; readers ask the catalog for the latest version, and one commit can span several tables.

HOW A COMMIT LANDS

A writer stages its commit, either as a file under `_delta_log/_staged_commits` or inline in the request. The catalog then ratifies it: it checks the version, enforces constraints and schema rules, and makes the commit visible. Later, ratified commits are published to `_delta_log` as normal JSON commits, so the log still exists for long-term storage. Publishing is asynchronous, which is why the feature requires In-Commit Timestamps, reader version 3 and writer version 7. Readers ask the catalog for the latest version first, skipping the cloud storage listing that normally starts log replay. Create one with `TBLPROPERTIES ('delta.feature.catalogManaged' = 'supported')`.

WHAT THE CATALOG ADDS

A filesystem can only give you per-table atomic writes and coarse path permissions. Once the catalog is the single source of truth, it can enforce governance on every write, reject incompatible schema changes, speed up metadata resolution and group writes to several tables into one atomic transaction. This replaces the earlier **coordinated commits** proposal. The Delta RFC explicitly rejected that one, because a non-catalog commit coordinator competed with the catalog to own the table.

VERSION NOTES

In open-source Delta the feature was renamed from `catalogOwned-preview` to `catalogManaged` in Delta 4.0.1. Early 2026 delta.io posts still called it experimental, with Unity Catalog 0.4.0 as the first catalog. Delta 4.2 makes commits catalog-coordinated, and Delta 4.3 routes all operations through the Unity Catalog Delta APIs. On Databricks it is supported on Unity Catalog managed tables from DBR 16.4. You can enable it on existing tables from DBR 18.0, and on streaming tables and materialised views from DBR 17.3. Databricks has announced automatic upgrades of existing Unity Catalog managed tables to catalog commits from August 2026. A table is upgraded only after an observation window shows every client supports it, and path-based access blocks the upgrade. Writes from external engines are Beta; transactions on managed Iceberg tables are Private Preview.

WHERE IT BITES

Reading the table by path fails, because filesystem-based access is not supported. That rules out `DeltaTable.forPath` readers and any tool that reaches the folder directly, such as a OneLake shortcut to the table path. Do not use `CREATE OR REPLACE TABLE` to switch the feature on or off; use `ALTER TABLE`. In open-source Delta it is one-way: you set it at creation and cannot remove it. To confirm it is on, run `DESCRIBE DETAIL` and look for `catalogManaged` in `tableFeatures`.

RELATED CONCEPTS

Transaction Log, Optimistic Concurrency, ACID Transactions, Unity Catalog.

Table and catalog object model.



Each platform has its own objects for naming a table and deciding who owns its files. Unity Catalog nests **metastore** (one per region, shared by every workspace in it), **catalog**, **schema**, then tables, views, volumes and functions, addressed as `catalog.schema.object`. Fabric nests workspace, lakehouse, schema and table, addressed as `workspace.lakehouse.schema.table`.

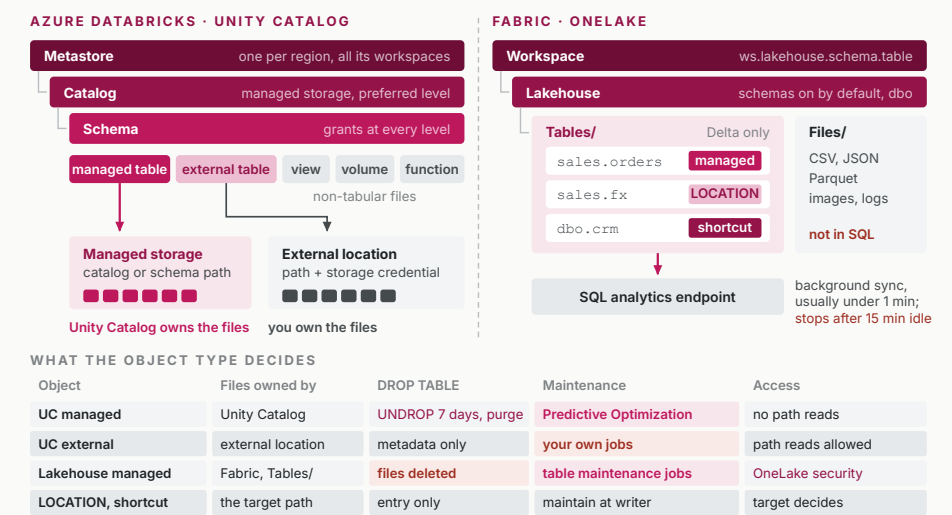


FIG. 8.13 Unity Catalog nests metastore, catalog, schema and objects, and a managed table keeps its files in catalog storage while an external table points into an external location. A Fabric lakehouse splits into Tables and Files, and the SQL analytics endpoint follows Tables through a background sync. The object type then decides what DROP does and which maintenance and security apply.

Managed or external in Unity Catalog

A **managed** table or volume lives in managed storage set at metastore, catalog or schema level, and Unity Catalog owns the files. After `DROP TABLE` they stay recoverable with `UNDROP` for 7 days by default, then they are purged. An **external** table sits under an **external location**, which pairs a storage path with a **storage credential** (an Azure managed identity or service principal); DROP removes only the metadata. Volumes govern non-tabular files under `/Volumes/catalog/schema/volume/`.

INSIDE A FABRIC LAKEHOUSE

A lakehouse has a **Tables** area, where Delta folders are registered automatically, and a **Files** area for any format. Schemas are on by default for new lakehouses, with `dbo` as the default schema. Save a table without `LOCATION` and it is managed, so DROP deletes its files; a table with `LOCATION`, or a shortcut, loses only its entry. The read-only **SQL analytics endpoint** follows changes through a background sync, usually under one minute, which stops after 15 minutes without queries.

WHAT THE OBJECT TYPE DECIDES

The object type decides which automation you get. Predictive Optimization runs `OPTIMIZE`, `VACUUM` and `ANALYZE` only on Unity Catalog managed tables; external tables need your own maintenance jobs. Catalog-managed tables (catalog commits) also require managed tables. In Fabric, lakehouse table maintenance (compaction, V-Order, `VACUUM` with a 7-day default retention) covers Delta tables only, and OneLake security filters rows and columns only on Delta tables read through the catalog. That is why Databricks recommends managed tables and keeps external ones for existing data in formats such as JSON or Avro, or for clients that need direct path access.

WHERE PEOPLE TRIP

Reading a Unity Catalog managed table by path: Databricks does not support it, because it bypasses access control. Expecting to undo DROP on a Fabric managed table. There is no `UNDROP` and the files are deleted from OneLake; OneLake soft delete keeps deleted files for 7 days, and you restore them with Storage Explorer, PowerShell or the REST API. Landing Parquet or CSV in Files and then looking for it in the SQL analytics endpoint, which lists Delta tables only. Querying the endpoint straight after a Spark write while its sync is idle; start an on-demand metadata refresh from the portal or the REST API.

RELATED CONCEPTS

Unity Catalog, OneLake, OneLake Shortcuts, Predictive Optimization, `VACUUM` and Retention, Catalog-managed tables.

Micro-batch Model.

LAYER FILES TABLE **STREAM** COMPUTE GOVERN PIPELINE SERVE OPERATE

Structured Streaming runs a stream as a continuous sequence of small batch jobs, not as a true record-at-a-time stream. Each micro-batch reads the data that arrived since the last one, processes it as a static DataFrame and writes the result.

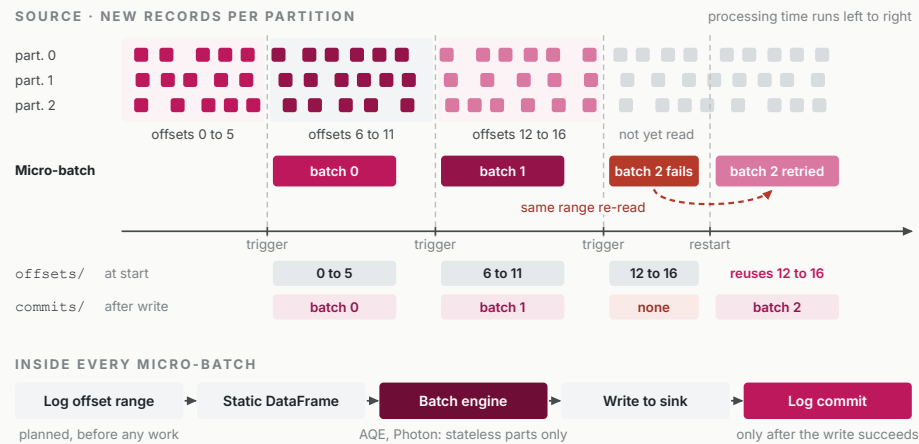


FIG. 9.1 Records keep arriving on each source partition; at every trigger Spark cuts the new offset range and records it in `offsets/`, runs it as one static batch job (Catalyst plans every batch; AQE and Photon apply only to stateless parts), writes the sink and only then adds an entry to `commits/`. A failed batch has an `offsets` entry but no commit, so on restart the same range is run again.

ONE BATCH, STEP BY STEP

The query keeps an offset log of how far it has read in each source partition. On each trigger Spark writes the planned offset range to `offsets/` before the batch starts, runs the query plan, writes the output, and only then records the batch in `commits/`.

AFTER A FAILURE

If the batch fails, no commit is recorded, and Spark re-runs the same offset range on restart.

WHICH OPTIMISERS APPLY

Every micro-batch goes through the full Spark SQL engine, so Catalyst optimises each one. AQE and Photon apply only to stateless work. Photon supports stateless streaming only; AQE reaches streaming only for stateless operators, inside `foreachBatch` on Databricks and in stateless queries from Apache Spark 4.1. Fabric NEE does not support Structured Streaming and falls back to the JVM engine.

Triggers.

LAYER FILES TABLE **STREAM** COMPUTE GOVERN PIPELINE SERVE OPERATE

The trigger sets when Structured Streaming starts a new micro-batch. That cadence decides how often data is read from the source, processed and written to the sink.

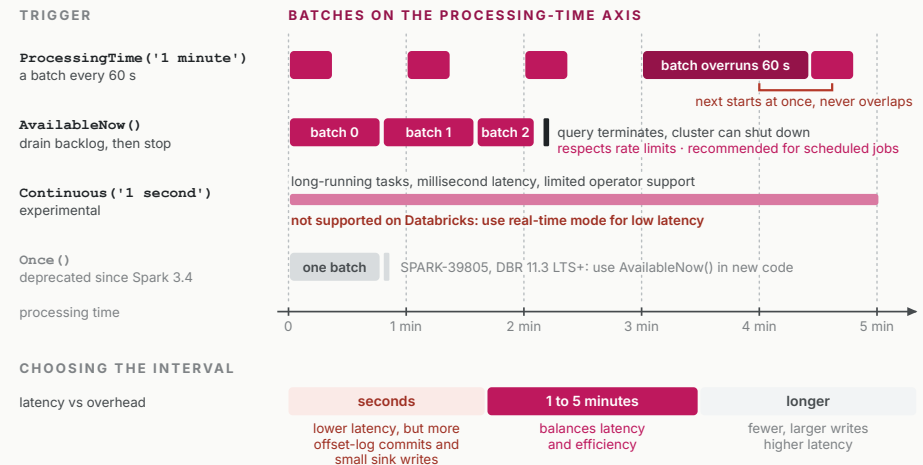


FIG. 9.2 The same five minutes of processing time under each trigger: fixed one minute cadence, a backlog drained by `AvailableNow` before the query stops, experimental continuous processing (not supported on Databricks, where real-time mode is the low-latency path), and the deprecated `Once`.

THE OPTIONS

`Trigger.ProcessingTime('1 minute')` starts a new batch every 60 seconds. If a batch overruns the interval, the next one starts as soon as it finishes; batches never overlap.

`Trigger.AvailableNow()` processes all available data in one or more optimised batches, respecting rate limits, then terminates. It is the one to pick when streaming replaces a scheduled batch job. `Trigger.Continuous('1 second')` is experimental continuous processing with millisecond latency but limited operator support. `Trigger.Once()` is **deprecated since Spark 3.4 / DBR 11.3 LTS+** (SPARK-39805). Use `AvailableNow()` instead for rate-limiting and incremental backlog handling, and keep `Once` out of new code.

CHOOSING AN INTERVAL

A shorter interval buys lower latency and costs overhead: more frequent commits to the offset log, more small writes to the sink, and more cluster scheduling per unit of data. For most production pipelines, a trigger of one to five minutes balances latency and efficiency.

ON DATABRICKS

Serverless compute supports only `AvailableNow` and `Once`, not `ProcessingTime`. `Trigger.Continuous` is not supported on Databricks, and it is still experimental in Apache Spark.

FOR LOW LATENCY

Spark 4.1 adds **real-time mode**, the successor path for low-latency work: stateless Scala queries in open source, and `.trigger(realTime='5 minutes')` in Python and Scala on Databricks. See Real-time mode for requirements and limits.

Watermarks and State.

LAYER

FILES

TABLE

STREAM

COMPUTE

GOVERN

PIPELINE

SERVE

OPERATE

A **watermark** tells Spark the maximum lateness you expect from events, and trails the maximum event time seen so far by that threshold. Data within the threshold is always processed; data older than the watermark may be dropped, though Spark does not guarantee it. **State** is what Spark keeps across micro-batches for stateful operations on streams, such as aggregations, joins and deduplication.

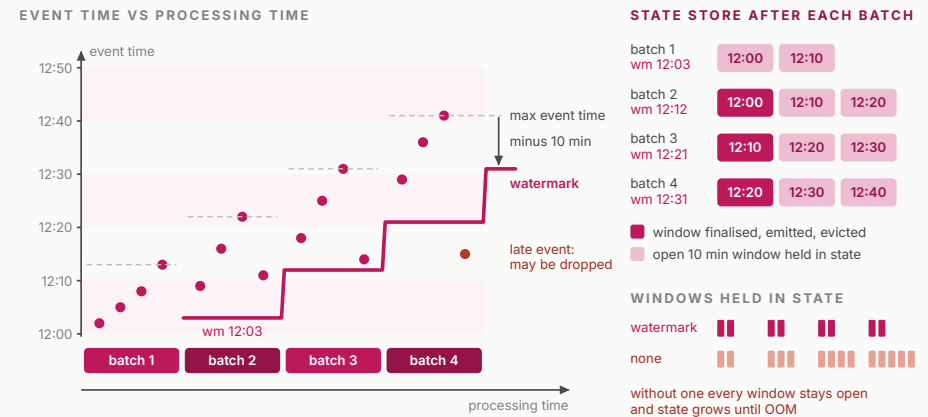


FIG. 9.3 Each dot is an event placed by its event time (up) and the batch that read it (across). The watermark trails the maximum event time seen by 10 minutes; once it passes a window's end the window is finalised and evicted from state, and events older than the watermark may be dropped; Spark guarantees only that events within the threshold are kept.

SETTING ONE

Define it with `.withWatermark('event_time', '10 minutes')`. Spark tracks the maximum observed event time across all batches and sets the watermark to that maximum minus the delay threshold.

WHEN STATE IS EVICTED

State for a time window stays in the state store until the watermark advances past the window's end time. At that point Spark finalises the state and evicts it.

LEAVING IT OUT

Without a watermark, a stateful aggregation accumulates state indefinitely. Every incomplete window stays open, the state store grows without bound across batches, and the query eventually hits OOM. Whenever you use `groupBy` on event time in a streaming query, define a watermark.

Output Modes and Exactly-once.

LAYER FILES TABLE **STREAM** COMPUTE GOVERN PIPELINE SERVE OPERATE

The output mode decides what reaches the sink on each micro-batch. **Append** writes only the rows added since the last trigger. **Complete** rewrites the entire result table on every trigger, and **Update** writes only the rows that changed since the last trigger.

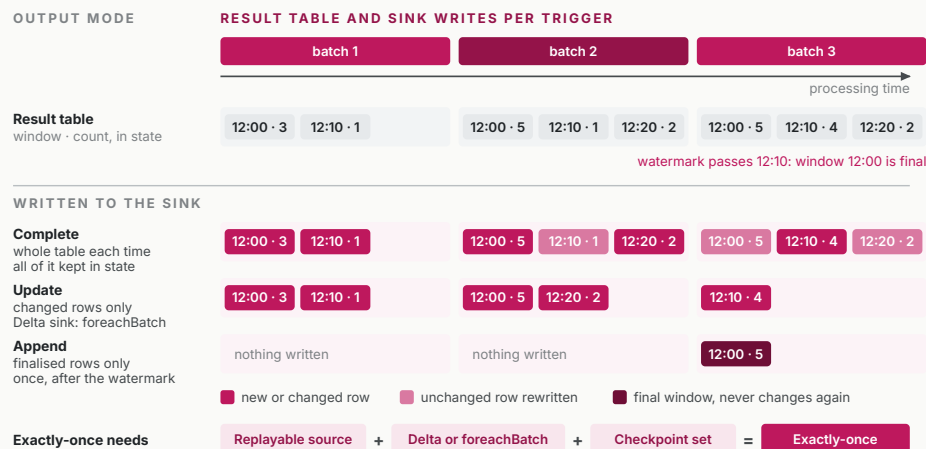


FIG. 9.4 One windowed count evolving over three triggers, and what each output mode writes to the sink on each trigger: Complete rewrites everything, Update writes only changed rows, Append writes a window once the watermark finalises it. The Delta sink takes Append and Complete only, so Update into Delta goes through foreachBatch with MERGE.

PICKING A MODE

Append is safe for stateless queries, and for windowed aggregations once the watermark has finalised the window, but not for aggregations that update existing rows. Complete keeps the entire result in the state store and rewrites it every trigger, so keep it for small aggregations such as running totals. Update is the most efficient mode for aggregations, because it writes only changed rows.

WHICH SINKS TAKE UPDATE

The Delta sink accepts only Append and Complete. To get Update into Delta, write through `foreachBatch` and run a `MERGE` there. The Kafka and `foreach` sinks accept Update directly.

GETTING EXACTLY-ONCE

You need three things: a replayable source, an idempotent or transactional sink (Delta, or `foreachBatch` with idempotent writes), and a correctly configured checkpoint location. The Kafka sink is at-least-once, so give downstream consumers dedup keys.

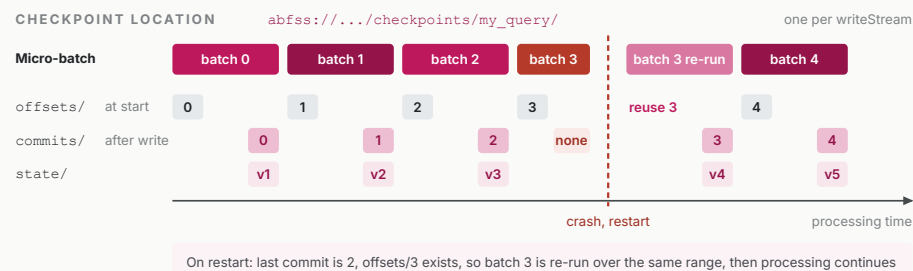
RELATED CONCEPTS

Micro-batch Model explains the processing model output modes apply to. Watermarks determine when Append mode can emit finalised window results.

Streaming checkpoints.

LAYER FILES TABLE **STREAM** COMPUTE GOVERN PIPELINE SERVE OPERATE

Structured Streaming writes its progress to a checkpoint directory: the position it has read from each source, the state of stateful operators, and metadata for the active query. It makes a stream restartable and exactly-once-capable. Without one, a stream cannot resume; with a corrupted or wrong one, it may reprocess or skip data silently.



EXACTLY-ONCE NEEDS ALL THREE

Replayable source
Kafka, Delta, files

Idempotent sink
Delta, or foreachBatch with idempotent writes

Durable checkpoint
ADLS or S3, not /tmp

missing any one: exactly-once is lost

COMMON MISTAKES

one path shared by two queries
deleting it to reset: sink double-writes
Kafka sink: at-least-once only

FIG. 9.5 The checkpoint directory grows batch by batch: an offsets file is written when a batch starts, a commits file only when its sink write succeeds, and state versions alongside. After a crash Spark finds offsets without a matching commit and re-runs that batch with the same range.

WHAT SPARK KEEPS THERE

Set it on the writeStream with `.option("checkpointLocation", "abfss://.../checkpoints/my_query/")`. Spark writes a **metadata** file holding the query ID, and several subdirectories. **offsets** records source positions per batch and **commits** records successful sink writes per batch. **state** holds the state-store contents for stateful operators (aggregations, joins, deduplication), and **sources** tracks the files a file source has already read. Offsets are written when a batch starts, commits after its sink write succeeds. On restart, Spark resumes from the last committed batch and re-runs any in-flight batch.

WHAT EXACTLY-ONCE NEEDS

First, a source with replayable offsets (Kafka, Delta, files, Event Hubs). Second, a sink that is idempotent or transactional: Delta, or `foreachBatch` with manual idempotency. The Kafka sink is at-least-once only. Third, a checkpoint that survives restarts. Miss any of the three and the stream is at-least-once or worse.

HOW CHECKPOINTS GET BROKEN

The classic break is two queries sharing a checkpoint path: the second corrupts the first, so give every writeStream its own. A checkpoint on `/tmp` or local disk, on a cluster that gets terminated, loses its state on the next start. Put production checkpoints on cloud storage (ADLS, S3). Deleting the checkpoint to "reset" a stream without dropping the sink makes the stream re-read all data, and a sink that is not idempotent double-writes.

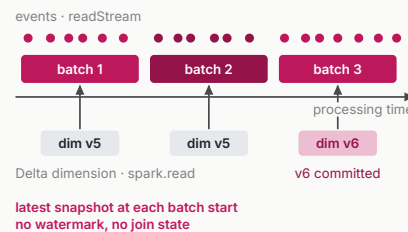
RESETTING A STREAM

Delete a checkpoint only when the query definition changes incompatibly: new stateful operators, changed group-by keys, changed schema. When you do, also truncate the sink or rely on idempotent writes. If all you want is a new source position, leave the old checkpoint in place. Start the query on a new checkpoint path with the source's starting-position option: `startingOffsets` for Kafka, `startingVersion` or `startingTimestamp` for Delta, `cloudFiles.includeExistingFiles` for Auto Loader.

Streaming joins.

LAYER FILES TABLE **STREAM** COMPUTE GOVERN PIPELINE SERVE OPERATE

A streaming join pairs a streaming DataFrame with another stream or with a static DataFrame. The "complete" set of rows on each side is never known, so the semantics differ from batch joins. Stream-stream joins need watermarks to bound state; stream-static joins are simpler, but the static side must be safely re-readable.

STREAM-STATIC**CHOOSING A JOIN**

Stream-static
dimension lookups per event
small, stable static side

Stream-stream inner
both sides are events
watermarks + time condition

Stream-stream outer
delayed arrivals must join
results late, most state

Unbounded state
no watermark, or no
time condition: OOM

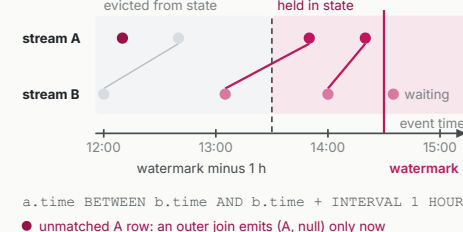
STREAM-STREAM

FIG. 9.6 Left: a stream-static join looks each micro-batch up against the latest Delta snapshot and keeps no state. Right: a stream-stream join buffers both sides in state on the event-time axis; watermarks plus a one hour join condition let Spark evict rows older than the watermark minus one hour.

JOINING TO A STATIC TABLE

The other side is a regular DataFrame, typically a Delta table loaded with `spark.read`, not `spark.readStream`. Each micro-batch sees the latest snapshot of the static side at batch start, and for Delta sources that snapshot updates whenever a new batch reads it. You need no watermark. The common pattern is streaming events joined to a slowly-changing dimension table.

TWO STREAMS, INNER JOIN

With `stream_a.join(stream_b, "key")` and both sides streaming, Spark holds rows from each side in state until matching rows arrive from the other. Leave out the watermark and state grows without bound until the query eventually OOMs. Put watermarks on both sides and add a time constraint to the join condition, such as `WHERE a.event_time BETWEEN b.event_time AND b.event_time + INTERVAL 1 HOUR`. Spark can then drop state for rows older than the watermark minus the time constraint. Watermarks without a time-based join condition are not enough: Spark cannot bound when matches might arrive, and state still grows.

TWO STREAMS, OUTER JOIN

An outer join also has to decide when it is safe to emit a null match. Spark waits until the watermark passes the latest time the join could still receive a matching row. Outer-joined results therefore arrive late, and only after the watermark advances past the relevant time window. Plan for that latency in downstream consumers.

PICKING THE JOIN

Use stream-static when the static side is small, stable and looked up per event, as in dimension lookups. Modelling a slowly-changing dimension as a stream-stream join instead adds stateful overhead for no benefit. Use stream-stream inner when both sides are events and you need them within a bounded time window (sessions, attribution). Use stream-stream outer only when delayed arrivals must still join, and expect higher state cost and stricter watermark requirements.

Auto Loader.

LAYER FILES TABLE STREAM COMPUTE GOVERN PIPELINE SERVE OPERATE

Auto Loader picks up new files as they land in cloud object storage (ADLS, S3, GCS) and streams them into Delta Lake as a Structured Streaming source. It finds new files on its own, processes each file exactly once and scales to billions of files, with no file tracking code on your side. It is a Databricks feature: `cloudFiles` is not part of Apache Spark or Fabric Spark.

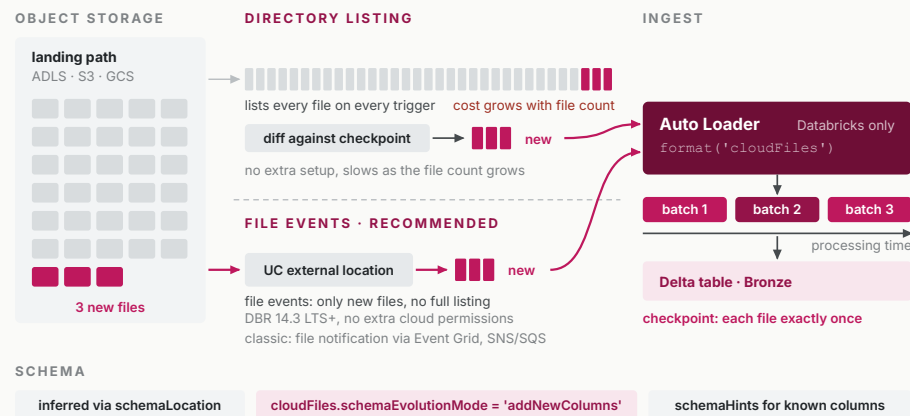


FIG. 9.7 Auto Loader, a Databricks feature, finds new files in two ways: directory listing scans every file on each trigger and diffs against the checkpoint, while file events mode (recommended, DBR 14.3 LTS and above) reads only new files from the file events of a Unity Catalog external location. The older Event Grid or SNS/SQS file notification mode is now called classic. Either way each file is ingested once into Delta in micro-batches.

HOW IT FINDS FILES

You pick one of two discovery modes. **Directory listing** lists the source on every trigger and compares the result against the checkpoint; it is simple but scales only to hundreds of thousands of files. **File events**, recommended on DBR 14.3 LTS and above, uses the file events of a Unity Catalog external location, needs no extra cloud permissions and finds new files without listing the directory. So does the older file notification mode, which Databricks now calls classic, but it subscribes the stream itself to Event Grid or SNS/SQS and needs extra cloud setup.

WHEN TO REACH FOR IT

Without Auto Loader you either schedule a batch job that rescans the whole directory or write your own file tracking. Both are error-prone, and neither scales to large file counts or gives you exactly-once. Auto Loader processes each file once, and its checkpoint means a restart neither loses nor duplicates files. Make it the standard pattern for any files landing in object storage: raw data ingestion, log processing, event export files from external systems, any Bronze layer in a medallion architecture. It is the recommended replacement for manual directory scanning and the plain Spark file source.

WRITING THE STREAM

The API is `spark.readStream.format('cloudFiles').option('cloudFiles.format', 'json').load(path)`. Schema inference and evolution need `cloudFiles.schemaLocation`. Lakeflow pipelines set it for you; without it you must pass a schema. Inference is automatic, and evolution is controlled with `cloudFiles.schemaEvolutionMode = 'addNewColumns'`.

SCHEMA SURPRISES

Inference samples a subset of files and, by default, types JSON and CSV columns as strings. When a new column shows up later, the stream stops with `UnknownFieldException`, adds the column to the schema and carries on after a restart. So run it under a job that retries, and put the columns you already know in `cloudFiles.schemaHints`. And avoid directory listing for a source that will grow past hundreds of thousands of files: listing slows with every trigger, where file events do not.

ON FABRIC

Auto Loader is Databricks only, and Fabric has no `cloudFiles` source. Use a plain Structured Streaming file source, a pipeline with a storage event trigger, or Eventstream for Event Hubs and Kafka.

RELATED CONCEPTS

Micro-batch Model is the execution model Auto Loader uses. Transaction Log explains the Delta sink it writes to. Schema Enforcement and Evolution controls how schema changes in incoming files are handled.

Real-time mode.

LAYER FILES TABLE **STREAM** **COMPUTE** GOVERN PIPELINE SERVE OPERATE

This Structured Streaming trigger processes each record as it arrives, with end-to-end latency as low as 5 ms, instead of waiting for the next micro-batch. Apache Spark 4.1 ships the first open-source version (SPARK-53736), for Scala and stateless queries only; Spark 4.2 adds the trigger to PySpark. On Databricks it is **GA** on Runtime 16.4 LTS and above, in Python and Scala, and adds stateful queries through `transformWithState`.

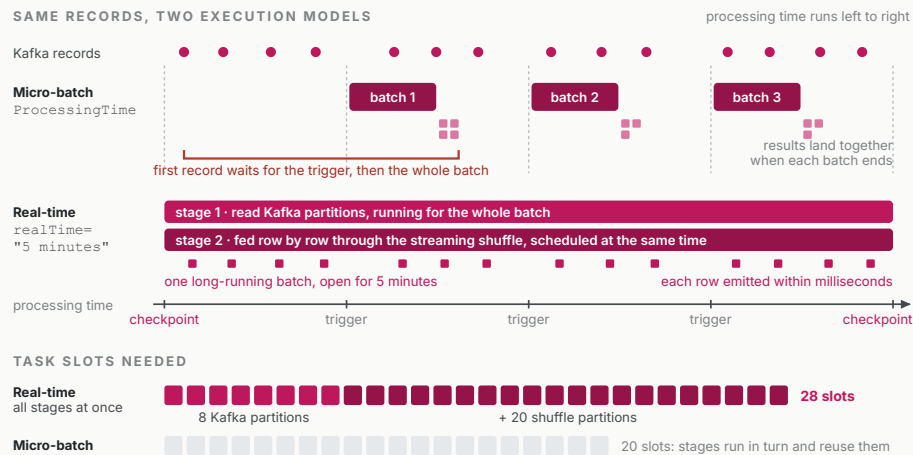


FIG. 9.8 The same Kafka records under both models on a shared processing-time axis. Micro-batch holds each record until the next trigger and emits results when the batch ends; real-time mode keeps one long-running batch open, runs every stage at once and emits each record within milliseconds, checkpointing every 5 minutes.

WHAT CHANGES FROM MICRO-BATCH

Batches are long-running, 5 minutes by default, and Spark processes rows inside the batch as soon as the source delivers them. The interval therefore sets how often Spark checkpoints, not how often it reads. All stages of the query are scheduled at the same time, and a streaming shuffle passes rows to the next stage as soon as they are produced. Python: `.trigger(realTime="5 minutes")`; Scala:

`.trigger(RealTimeTrigger.apply("5 minutes"))`. Output mode must be `update`. The older experimental `Continuous` trigger (Spark 2.3, at-least-once) is a separate engine limited to map-like operations; real-time mode supports aggregations, tumbling and sliding windows, deduplication and stream-table joins.

ON DATABRICKS

Use classic compute in dedicated or standard access mode (standard for Python only); serverless is not supported. Set `spark.databricks.streaming.realTimeMode.enabled` to `true`, and turn off autoscaling, Photon and spot instances. Sources: Kafka, Event Hubs through the Kafka connector, Kinesis. Sinks: Kafka, Event Hubs, `foreach` writers. Delta is neither source nor sink. Not supported: `foreachBatch`, session windows, `mapGroupsWithState`, `applyInPandasWithState` or `transformWithStateInPandas`.

WHEN IT PAYS OFF

Use it where each event needs an answer in under a second: fraud detection, real-time personalisation, operational decisions taken per event. Keep micro-batch for analytical ETL and transformations, and wherever cost matters more than sub-second latency or fast recovery from frequent checkpoints matters. Real-time mode for ETL that tolerates minutes of delay is a common mistake: micro-batch is cheaper and recovers faster because it checkpoints more often.

SIZING AND LATENCY TRAPS

Size the cluster for every stage at once: task slots must be at least the total tasks across all stages, so 8 Kafka partitions plus a 20-partition shuffle need **28 slots**. Low-throughput Python queries (under 5 records per second) on DBR 18.1 and below can see several hundred ms of added latency; DBR 18.2 fixes this.

RELATED CONCEPTS

Micro-batch Model, Triggers, Streaming checkpoints, Output Modes and Exactly-once, `transformWithState` and State Data Source.

transformWithState and State Data Source.

LAYER FILES TABLE **STREAM** COMPUTE GOVERN PIPELINE SERVE OPERATE

`transformWithState` is the arbitrary stateful operator added in Apache Spark 4.0, replacing `mapGroupsWithState` and `flatMapGroupsWithState` in Scala and `applyInPandasWithState` in Python. You write a `StatefulProcessor` class; Spark calls it per grouping key and keeps its typed state variables and timers in the state store. Over the old APIs it adds several named state variables and timers per key, TTL eviction and state schema evolution, so a query can change its state layout without reprocessing rows. The **State Data Source** reads that state back as a `DataFrame`.

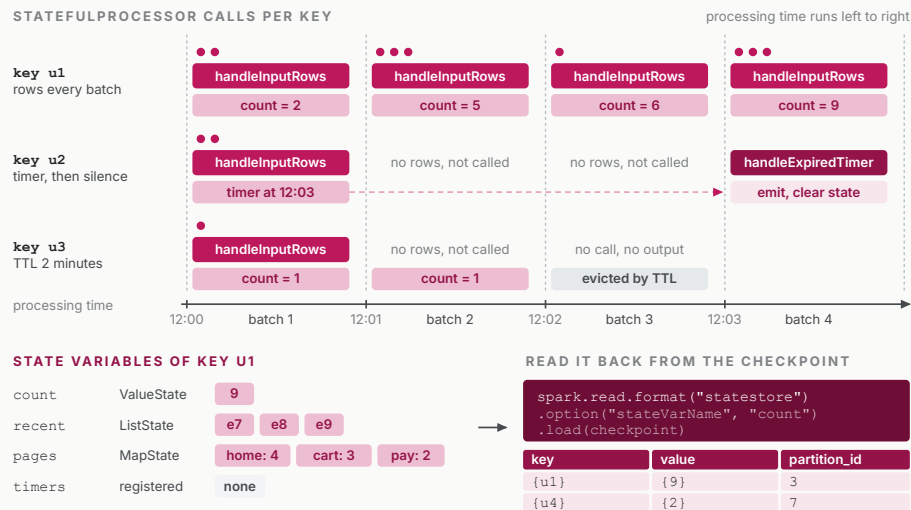


FIG. 9.9 A `StatefulProcessor` is called per key on each micro-batch. Key u1 updates its `ValueState` every batch; u2 registers a timer that fires `handleExpiredTimer` with no new rows; u3 is evicted silently by TTL. The state lands in the checkpoint, where the `statestore` reader returns it as rows.

WRITING THE PROCESSOR

In `init`, create state variables through the handle: **ValueState** holds one value per key, **ListState** is ordered and append-optimised, and **MapState** holds key-value pairs inside one grouping key. `handleInputRows` reads and updates state, registers or deletes timers and emits rows. `handleExpiredTimer` runs when a timer fires, even if no rows arrive for that key. Set `timeMode` to `None`, `ProcessingTime` or `EventTime`; an optional TTL per variable evicts state values in processing-time mode. Python offers `transformWithStateInPandas` and, since Spark 4.1, a row-based `transformWithState`.

CHANGING THE STATE LAYOUT

You can add or drop variables between runs. Fields can be added, removed, widened or reordered when `spark.sql.streaming.stateStore.encodingFormat` is `avro`. Renames and type narrowing fail: rename a field in a state schema and the checkpoint will not restore.

READING STATE BACK

`spark.read.format("statestore").load(checkpoint)` returns `key`, `value` and `partition_id` for the latest committed batch. Options: `batchId`, `operatorId`, `stateVarName` (required for this operator), `readRegisteredTimers` for timers, and `readChangeFeed` with `changeStartBatchId` for per-batch update and delete rows. `format("state-metadata")` lists operators and batch ranges. The source is read-only and marked experimental. When you debug, read the checkpoint with `statestore` instead of printing inside the processor.

ON DATABRICKS

Supported on Runtime 16.2 and above. RocksDB is the default state store provider from DBR 17.3; on 17.2 and below, set `spark.sql.streaming.stateStore.providerClass` to `RocksDBStateStoreProvider`. In real-time mode (DBR 17.3 LTS and above) only the row-based API is supported, `handleInputRows` is called once per row, and there are no event-time timers.

WHERE STATE LEAKS

Set no TTL and write no eviction logic, and state grows with every new key until executors run out of memory. Calling `put` on a `ListState` to add one element replaces the whole list and resets every TTL.

RELATED CONCEPTS

Watermarks and State, Streaming checkpoints, Real-time mode, Output Modes and Exactly-once.

Spark Declarative Pipelines.

LAYER FILES TABLE **STREAM** COMPUTE GOVERN **PIPELINE** SERVE OPERATE

Spark Declarative Pipelines (SDP), added to Apache Spark in 4.1 (SPARK-51727), let you define datasets and the queries that fill them. Spark works out the dependency graph, execution order, parallelism, checkpoints and retries. Databricks **Lakeflow pipelines**, formerly Delta Live Tables, extend it and stay compatible with it.

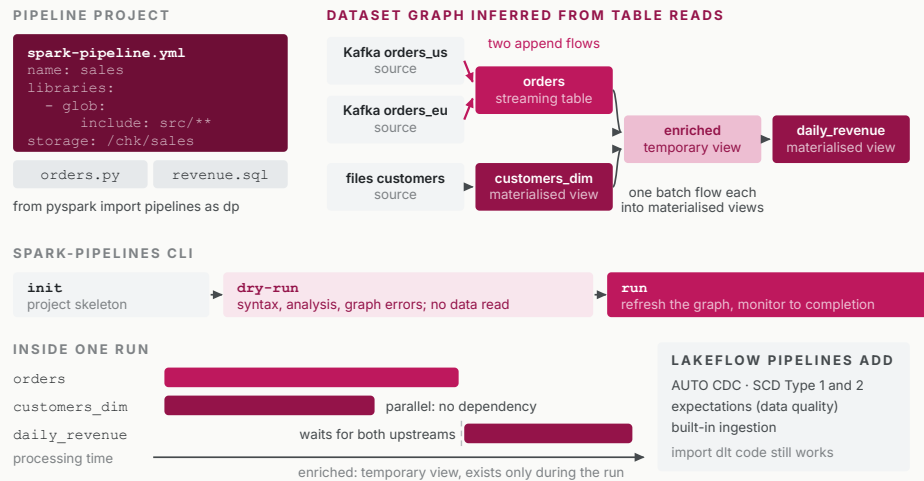


FIG. 9.10 A pipeline project is a spec file plus Python and SQL sources. SDP infers the dataset graph from table reads, dry-run validates it without touching data, and run refreshes datasets in dependency order, running independent ones in parallel. Lakeflow pipelines on Databricks add features Apache SDP does not have.

DATASETS AND FLOWS

There are three dataset types. A **streaming table** is filled by one or more streaming flows and processes only new data. A **materialised view** is a query precomputed into a table, written by exactly one batch flow. A **temporary view** exists only during a pipeline run. A **flow** reads a source, applies logic and writes a target. In Python, import `from pyspark import pipelines as dp` and use `@dp.table`, `@dp.materialized_view`, `@dp.temporary_view` and `@dp.append_flow`. In SQL, use `CREATE STREAMING TABLE`, `CREATE MATERIALIZED VIEW` and `CREATE FLOW`. Spark infers dependencies from `spark.read.table` and `spark.readStream.table` references.

RUNNING A PIPELINE

Without SDP, each pipeline hand-codes the order of its jobs, a checkpoint path per stream and the logic to rerun after a failure. With it, you declare the graph once. A `spark-pipeline.yml` spec lists `libraries` (source globs) and `storage` (checkpoint directory), plus optional `catalog`, `database` and `configuration`. The CLI is built on `spark-submit` and offers `spark-pipelines init`, `dry-run` and `run`. Run `dry-run` first: it validates the whole pipeline and catches syntax, analysis and graph errors without touching data. `run` launches the pipeline, refreshes every dataset in dependency order with independent ones in parallel, and monitors it until it completes.

LAKEFLOW AND VERSION NOTES

On Databricks, Lakeflow pipelines run SDP code on the Databricks Runtime and add built-in ingestion, data quality expectations and the `AUTO CDC` APIs (SCD Type 1 and 2). Apache SDP has no `AUTO CDC` in 4.1; Spark 4.2 adds it for SCD Type 1. Existing `import dlt` code keeps working. New code uses `dp`, where `@dlt.view` becomes `@dp.temporary_view`. From Spark 4.1 the `run` command takes per-dataset `--refresh` and `--full-refresh` options, and Spark 4.2 injects `spark` into each pipeline file.

WHERE IT BITES

SDP evaluates your code several times during planning, and a dataset function must only return a `DataFrame`. Keep `collect()`, `count()`, `toPandas()`, `saveAsTable()` and `start()` out of it. `PIVOT` is unsupported in SDP SQL. And do not expect `AUTO CDC` in open-source SDP before Spark 4.2, or SCD Type 2 anywhere outside Databricks.

RELATED CONCEPTS

Lakeflow pipelines (formerly DLT), Streaming Tables, Materialised Views, Streaming checkpoints, Auto Loader.

04

Azure Databricks.

The Spark and Delta pages from earlier parts, as you meet them on Databricks: picking compute (autoscaling, all-purpose and job clusters, pools, policies), the access modes and Unity Catalog rules that decide what your code may run and read, and the Lakeflow pipelines and Jobs that take a notebook into production.

X Running PySpark on Databricks

72	Autoscaling	68
73	All-purpose vs Job Clusters	69
74	Instance Pools and Spot VMs	69
75	Cluster Policies and DBR	70
76	Access modes	70
77	Unity Catalog	71
78	Lakeflow pipelines (formerly DLT)	72
79	Materialised Views	73
80	Streaming Tables	74
81	Lakeflow Jobs	74

READING PATH

Layer heat: A B C D E F G H = Files to Operate

X

Which compute, which access mode, which orchestration?



Autoscaling.

LAYER FILES TABLE STREAM **COMPUTE** GOVERN PIPELINE SERVE OPERATE

Autoscaling adds executors when the job has more tasks queued than free slots, and removes executors that have had no tasks for a configurable period. The cluster sizes itself to what the workload actually demands, so you do not have to fix an executor count upfront.

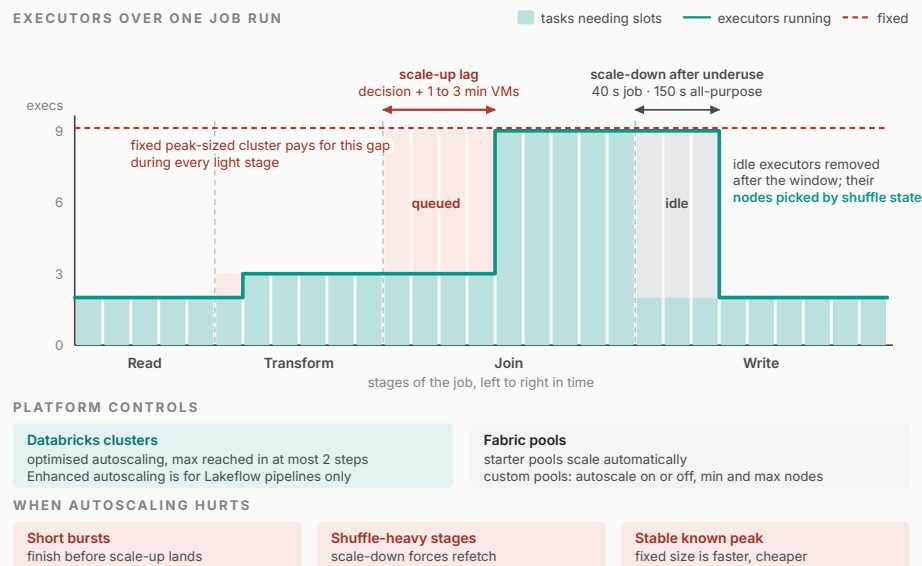


FIG. 10.1 Executors track demand across a job's stages: when tasks queue with no free slots, new executors arrive after a decision plus VM provisioning lag; underused nodes are removed after 40 s on job compute or 150 s on all-purpose compute, with Databricks choosing which nodes to remove from their shuffle file state (shuffle data is migrated only if decommissioning is turned on). Databricks and Fabric expose different controls.

WHAT THE CLUSTER MANAGER DOES

The cluster manager watches the task queue. When tasks are pending and no slots are free, it requests more executors; VM provisioning typically takes 1 to 3 minutes. Idle executors are removed after a timeout, and their shuffle files go with them unless you turn on decommissioning to migrate them (`spark.decommission.enabled` and `spark.storage.decommission.enabled`).

ON DATABRICKS AND FABRIC

Databricks clusters use optimised autoscaling, driven by the Databricks cluster manager instead of standard Spark dynamic allocation. On classic compute it reaches the maximum in at most two steps. It scales down after 40 seconds of underuse on job compute and 150 seconds on all-purpose compute, checking shuffle file state to pick which nodes to remove. `spark.databricks.aggressiveWindowDownS` sets how often scale-down decisions are made (600 seconds at most). Enhanced autoscaling is a separate feature, for Lakeflow pipelines. On Fabric, starter pools scale automatically; custom pools let you turn autoscale on or off and set min and max node bounds. Open-source dynamic allocation needs explicit configuration to avoid losing data when executors are removed mid-job: shuffle tracking, an external shuffle service, or decommissioning with shuffle migration.

WHERE IT PAYS OFF

Most Spark jobs have uneven demand. A pipeline that reads, transforms and then runs a complex join needs far more executors in the join stage than in the read stage. Autoscaling stops you paying for peak capacity during the light phases and running short during the heavy ones. For interactive and ad-hoc work, where query complexity is unpredictable, autoscaling is essential.

WHEN IT COSTS YOU

Provisioning executors mid-job adds latency, more so when spot instances are slow to allocate. Short-burst jobs finish before scale-up completes, so they pay for the spin-up and never use the new capacity. Scale-down takes shuffle files with the node, so later stages may have to recompute them, and a shuffle-heavy stage can stall when the cluster shrinks mid-shuffle and blocks have to be refetched. Databricks limits this by choosing nodes from shuffle file state. If demand is predictable and stable, with a known peak, a fixed-size cluster avoids provisioning latency entirely and is often faster and cheaper.

RELATED CONCEPTS

Driver, Executor, Cores, Slots defines the unit being scaled. Instance Pools and Spot VMs reduce scale-up latency significantly by pre-warming VM instances before they are needed.

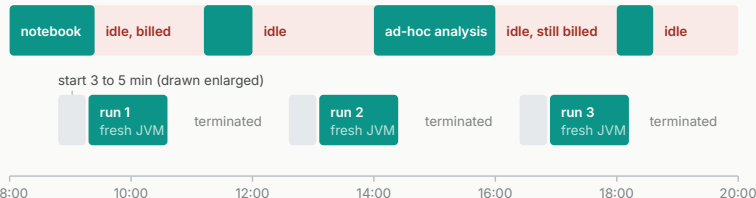
All-purpose vs Job Clusters.

LAYER FILES TABLE STREAM **COMPUTE** GOVERN PIPELINE SERVE OPERATE

An **all-purpose cluster** (Databricks) is the persistent, interactive cluster you use for notebooks, development and ad-hoc analysis. A **job cluster** is ephemeral: created for a single job run, terminated automatically when the job completes.

ONE WORKING DAY

All-purpose
persistent, shared



All-purpose

Use notebooks, development, ad-hoc
Start already running, instant
Cost DBUs accrue while idle

Job cluster

Use scheduled production runs on classic compute
Start 3 to 5 min, cut by Instance Pools
Cost runtime only, no state leak between runs

FIG. 10.2 Over one working day, an all-purpose cluster stays up between notebook sessions and accrues DBUs while idle, while each scheduled job run gets its own fresh job cluster that starts, runs and terminates.

WHERE THE DBUS GO

An all-purpose cluster stays running between uses: available immediately, and accruing DBU cost while idle. A job cluster is provisioned at job start and terminated at job end, so you pay only for actual runtime. Each run gets a fresh JVM, so no state leaks between runs.

CHOOSING FOR PRODUCTION

All-purpose suits iterative development but is expensive for scheduled production jobs, where cluster startup time (typically 3 to 5 minutes) is acceptable. Job clusters are cheaper per run but add that startup latency. The Lakeflow Jobs UI now selects serverless compute by default; choose classic jobs compute when a task needs it.

SHORTENING THE WAIT

Instance Pools cut job cluster startup time by pre-warming VMs.

Instance Pools and Spot VMs.

LAYER FILES TABLE STREAM **COMPUTE** GOVERN PIPELINE SERVE OPERATE

An instance pool is a set of pre-allocated, idle VM instances that Databricks keeps ready. When a cluster or autoscaling event requests a new executor, it draws an instance from the pool instead of provisioning one from scratch, cutting startup from several minutes to under 30 seconds.

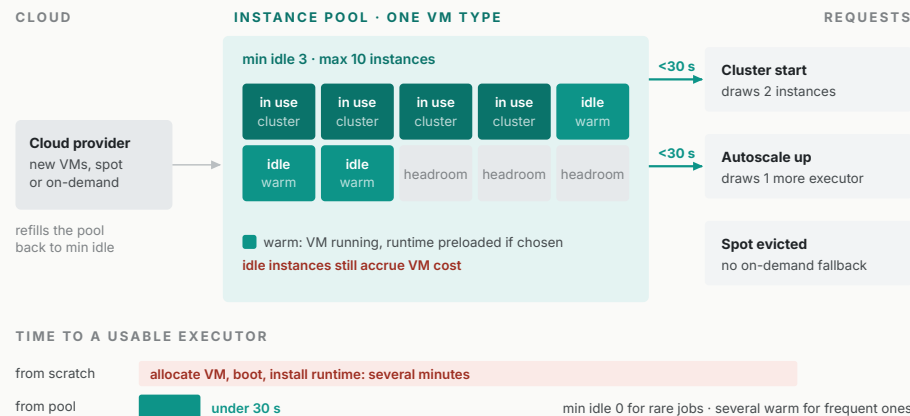


FIG. 10.3 A pool keeps a minimum number of idle VMs already running, with a runtime preloaded if you choose one; cluster starts and autoscaling events draw from it in under 30 seconds instead of provisioning from scratch, at the price of paying for idle instances.

SETTING ONE UP

Create the pool with a minimum and maximum idle instance count and an instance type. Databricks keeps the minimum idle instances running, with a runtime version preloaded if you choose one. Clusters that reference the pool draw from it on startup.

SPOT IN A POOL

A pool can hold spot instances, but evicted ones are not replaced with on-demand instances.

SIZING THE IDLE COUNT

Idle instances accrue VM compute cost even when no jobs run, so set the minimum idle count from actual job frequency. For infrequent jobs, a minimum of zero with a small maximum is enough. For high-frequency scheduled jobs, keeping several instances warm eliminates startup latency entirely.

Cluster Policies and DBR.

LAYER FILES TABLE STREAM **COMPUTE** GOVERN PIPELINE SERVE OPERATE

Admins write **cluster policies** to restrict which cluster configurations users can create, for cost control, security compliance and consistent runtime versions. **DBR** (Databricks Runtime) is the versioned software stack each cluster runs, including Spark, Delta and platform integrations.

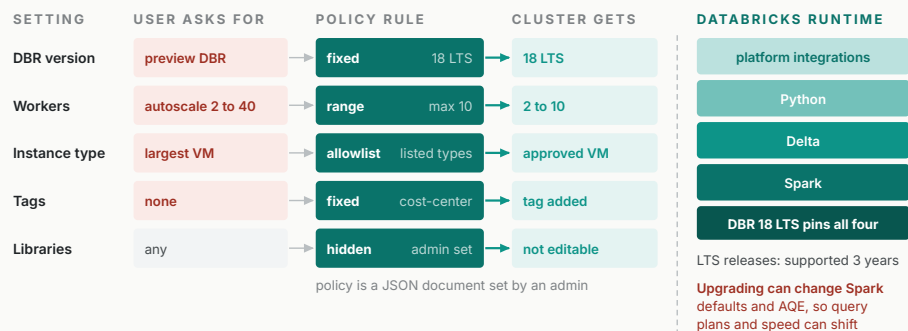


FIG. 10.4 A cluster policy fixes, bounds or hides each setting, so an oversized request on a preview runtime becomes a cluster on DBR 18 LTS with at most 10 workers and the required tags. The DBR version pins the whole software stack.

WRITING A POLICY

A policy is a JSON document. For each setting it can fix the value, allow a range or hide it: instance type, autoscaling bounds, DBR version, libraries and tags. You might force every production job onto DBR 18 LTS with a maximum of 10 workers, so nobody picks a preview runtime or an oversized cluster by accident. That is how policies stop runaway spending.

RUNTIME VERSIONS

Each DBR version is tied to specific Spark, Delta and Python versions. LTS releases are supported for three years.

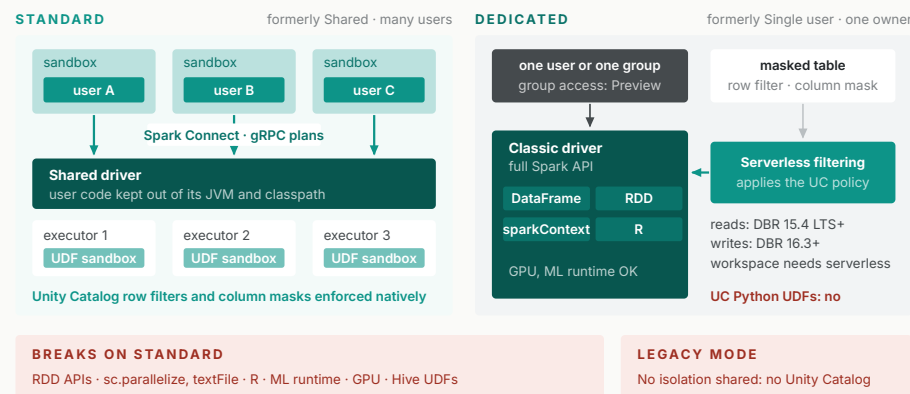
BEFORE YOU UPGRADE

Pin the DBR version so production jobs run on stable, tested runtimes. It matters most at upgrade time: a new DBR version can change default Spark configurations and AQE behaviour, which can change your query plans and performance.

Access modes.

LAYER FILES TABLE STREAM **COMPUTE** GOVERN PIPELINE SERVE OPERATE

On classic compute, the access mode decides who can attach to a cluster and how their code is isolated. Two modes are current: **Standard** (formerly Shared) and **Dedicated** (formerly Single user). **No isolation shared** is marked Legacy, not recommended, and does not support Unity Catalog. The default, **Auto**, gives you Standard unless you pick an ML runtime, a GPU instance type or a runtime below 14.3, in which case you get Dedicated.



AUTO DEFAULT Standard, unless an ML runtime, a GPU instance or DBR below 14.3 is chosen: then Dedicated

FIG. 10.5 Standard compute runs each user in a sandbox and talks to a shared driver over Spark Connect, so Unity Catalog policies apply natively but low-level APIs are gone; Dedicated compute gives one user or group the classic driver and full API, delegating row filters and column masks to serverless compute.

HOW USERS ARE KEPT APART

On **Standard**, many users attach and run at once, and Lakeguard isolates them. Spark Connect decouples each client from the driver, so clients no longer share a JVM or classpath. Each client runs in its own container sandbox, and UDFs run sandboxed on executors with egress traffic isolated. Row filters and column masks are enforced natively. **Dedicated** serves one user or one group (group assignment is Public Preview) on the classic Spark architecture with the full API. Fine-grained access control there is delegated to serverless compute: reads on DBR 15.4 LTS and above, writes on 16.3 and above, and the workspace must be enabled for serverless.

WHAT STANDARD TAKES AWAY

You pay for that isolation in API surface. Standard does not support RDD APIs or `sc` functions such as `parallelize`, `textFile` and `emptyRDD`. R, Databricks Runtime for ML, GPU instances, Hive UDFs, socket sources and spark-submit tasks are out too. UDF support follows the runtime: Python scalar and pandas UDFs need 13.3 LTS, while UDAFs, UDTFs, `applyInPandas` and `mapInPandas` need 14.3 LTS. On DBR 19 and above you cannot set properties such as `spark.jars` and `spark.driver.extraJavaOptions`.

PICKING A MODE

Databricks recommends Standard unless it lacks a feature you need. Go to Dedicated for RDD code, R, GPU or ML runtimes, and privileged machine access. The limit also runs the other way: Unity Catalog Python UDFs are not supported on Dedicated, so run them on Standard, serverless or a SQL warehouse.

WHERE IT BITES

Create job clusters through the API without `data_security_mode` and, unless an admin has set a workspace default, they fall back to No isolation shared and lose Unity Catalog. A notebook built on `sc.parallelize` will not run after you move it to Standard. On Dedicated with DBR 15.3 or below, tables with row filters or column masks and dynamic views cannot be read at all.

RELATED CONCEPTS

Unity Catalog, Spark Connect, Classic vs serverless compute, Cluster Policies and DBR, PySpark UDF variants, All-purpose vs Job Clusters.

Unity Catalog.

LAYER FILES TABLE STREAM COMPUTE **GOVERN** PIPELINE SERVE OPERATE

Unity Catalog, Databricks' unified governance layer for data and AI assets, replaces the per-workspace Hive metastore with an account-level catalog hierarchy. There is one metastore per region, shared by every workspace in that region, with access control centralised at catalog, schema, table, column and row level.

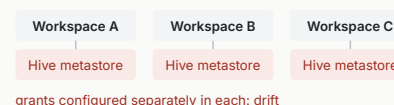
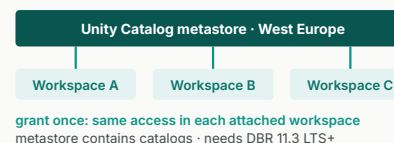
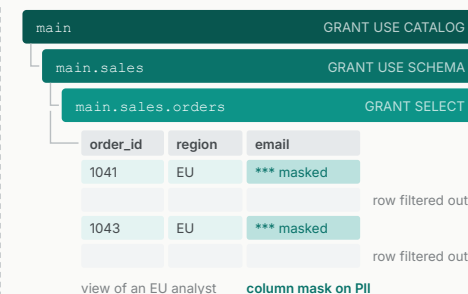
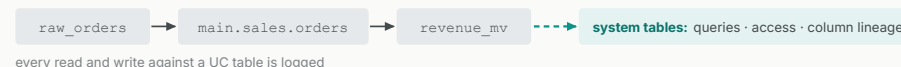
BEFORE · METASTORE PER WORKSPACE**AFTER · ONE METASTORE PER REGION****CATALOG.SCHEMA.TABLE · GRANTS AT ANY LEVEL****LINEAGE AND AUDIT · CAPTURED AUTOMATICALLY**

FIG. 10.6 Unity Catalog replaces per-workspace Hive metastores with one metastore per region, which contains the catalogs, so a grant applies in every workspace attached to it. Access is controlled from catalog down to column masks and row filters, and every read and write feeds lineage and audit system tables.

NAMES AND GRANTS

You address tables as `catalog.schema.table`. A metastore contains catalogs, and each workspace is attached to exactly one metastore in its region. Grant and revoke at any level (catalog, schema, table, column, row) with standard SQL semantics. The policy is the same everywhere: a user granted a table has that access in every workspace attached to the metastore, and a user denied is denied everywhere. Before Unity Catalog you configured permissions separately per workspace, which made consistent access control across a multi-workspace deployment manual and error-prone.

AUDIT AND LINEAGE

Lineage is captured automatically: queries that read or write Unity Catalog tables emit lineage events, and you see them in the Catalog Explorer UI. Every read and write against a Unity Catalog table is logged. Query the system tables with SQL for query history, table access and column-level lineage, and for billing and usage analytics. On Databricks this replaces separate audit tooling; on Fabric it maps to the OneLake audit log.

MOVING TO IT

Make Unity Catalog the default for all new Databricks deployments. Workspaces on the legacy Hive metastore can be migrated with the Unity Catalog migration tool. What you get for it: column-level masking for PII, row-level security for multi-tenant tables, centralised lineage tracking and the system tables.

RUNTIMES AND PLATFORMS

Unity Catalog is available on Azure Databricks, AWS Databricks and GCP Databricks. Minimum DBR versions differ by feature: Unity Catalog needs DBR 11.3 LTS or above, including standard access mode (formerly shared), and Python UDFs in UC need 13.3+. Check the Databricks documentation for the specific feature you plan to use. The closest Microsoft Fabric equivalent is OneLake security. Its roles scope access to folders, tables, schemas, rows and columns and are enforced across Fabric engines including Spark, alongside workspace roles and Microsoft Purview integration.

RELATED CONCEPTS

Access modes decide which compute can read Unity Catalog data. Cluster Policies and DBR can enforce that clusters use Unity Catalog-compatible access modes.

Lakeflow pipelines (formerly DLT).

LAYER FILES TABLE **STREAM** COMPUTE GOVERN **PIPELINE** SERVE OPERATE

Lakeflow pipelines (formerly Delta Live Tables, DLT) are the Databricks framework for declarative pipelines. You define tables with SQL or Python decorators and declare what each one reads, instead of writing imperative Spark code that orchestrates reads, transforms and writes. Databricks infers the execution order, manages pipeline state, and handles incremental processing, retries and data quality enforcement.

```
from pyspark import pipelines as dp
```

```
@dp.table
def bronze():
    return spark.readStream
        .format("cloudFiles")...
```

```
@dp.table
@dp.expect_or_drop("id_ok", ...)
def silver():
    return spark.readStream.table("bronze")
```

```
@dp.materialized_view
def gold():
    return spark.read.table("silver")
        .groupBy("day").agg(...)
```

The pipeline manages order, incremental state, retries, checkpoints, metrics; import dlt still works

EXPECTATIONS ON BAD ROWS

```
@dp.expect
keep row, log violation
```

```
@dp.expect_or_drop
remove the row
```

```
@dp.expect_or_fail
abort the run
```

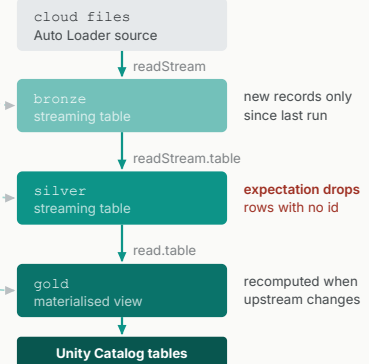
PIPELINE INFERS AND RUNS

FIG. 10.7 Each `@dp.table` or `@dp.materialized_view` definition returns a DataFrame; the Lakeflow pipeline reads the `spark.read.table` and `spark.readStream.table` calls to infer the dependency graph, runs it in order, processes only what changed, and applies expectations that keep, drop or fail on bad rows.

DEFINING TABLES

Declare a streaming or batch table with `@dp.table` or `@dp.materialized_view` in Python, or `CREATE OR REFRESH STREAMING TABLE` / `MATERIALIZED VIEW` in SQL, and return a DataFrame or query as its definition. The pipeline infers the dependency graph from the tables each definition reads with `spark.read.table()` or `spark.readStream.table()` (older code uses `dlt.read()` and `dlt.read_stream()`), builds a DAG and works out the execution order itself. Each run computes only what has changed since the last successful run: streaming tables process new records incrementally, and materialised views recompute when upstream data changes.

WHAT YOU STOP WRITING

The boilerplate every pipeline otherwise reimplements goes away: dependency ordering, incremental processing logic, retry on failure, data quality monitoring and lineage. You end up with something simpler to write and easier to watch than the hand-coded PySpark job. The pipeline UI shows data flow, row counts, expectation pass rates and pipeline history in one view.

ADDING EXPECTATIONS

Define expectations with `@dp.expect` in Python or `CONSTRAINT` clauses in SQL. Base `@dp.expect` logs the violation and keeps the row; `@dp.expect_or_drop` removes it; `@dp.expect_or_fail` aborts the run. The older `@dlt.expect` names still work. The pipeline manages state, checkpoints and metrics and shows them in the pipeline UI, so observability comes built in.

WHERE IT GETS AWKWARD

Pipelines run on pipeline-managed compute billed under their own DBU SKUs (Core, Pro, Advanced or serverless), and only serverless pipelines refresh materialised views incrementally. Custom logic that does not fit the declarative model, such as multi-pass algorithms or dynamic schema generation, can be awkward to express. Output is constrained too. Datasets go to Unity Catalog tables; anything else, such as a Delta table by path, Kafka, Event Hubs or a custom Python sink, needs a pipeline sink (`create_sink` or a `ForEachBatch` sink).

NAMES, APIS AND SDP

Classic SKUs and event log schemas still carry the DLT name. Lakeflow pipelines run on Databricks Runtime and extend and interoperate with **Apache Spark Declarative Pipelines** (SDP), the open-source framework added in Spark 4.1. The `AUTO CDC` APIs (formerly `APPLY CHANGES`, same syntax) and expectations are Databricks additions that SDP 4.1 does not have; Spark 4.2 adds Auto CDC for SCD Type 1 only. The documented Python API is `from pyspark import pipelines as dp`, with `@dp.table` for streaming tables, `@dp.materialized_view`, and `@dp.temporary_view` (formerly `@dlt.view`). `import dlt` still works and existing pipelines need no migration. In SQL, `CREATE OR REFRESH LIVE TABLE` is deprecated; use `CREATE OR REFRESH MATERIALIZED VIEW`.

RELATED CONCEPTS

Unity Catalog is the target metastore for pipeline tables in production; pipelines write to Unity Catalog tables by default when the workspace uses Unity Catalog. Auto Loader is the standard pipeline source for file-based ingestion. Change Data Feed and row tracking are used internally to compute incremental updates efficiently on materialised views.

Materialised Views.

LAYER FILES TABLE STREAM COMPUTE GOVERN PIPELINE SERVE OPERATE

A materialised view stores a precomputed query result as a managed Delta table, refreshed on a schedule or when source data changes. It exists in Lakeflow pipelines (formerly DLT; declarative, recommended) and as standalone Unity Catalog MVs, and replaces writing a query and materialising its output on a schedule.

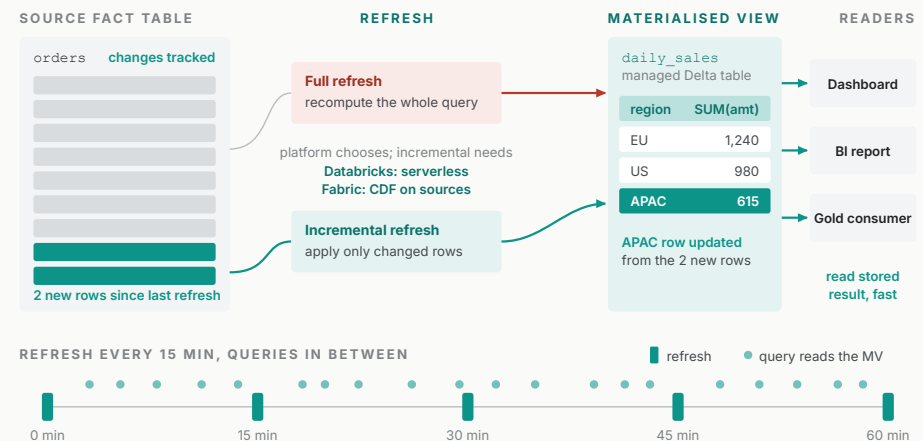


FIG. 10.8 A materialised view stores a query result as a managed Delta table. On each refresh the platform applies only the source changes when it can (serverless pipelines on Databricks, Change Data Feed on Fabric) or recomputes the whole query; dashboards between refreshes read the stored result instead of re-aggregating the facts.

DEFINING ONE

Write `CREATE MATERIALIZED VIEW` in SQL, or `@dp.materialized_view` in pipeline Python (formerly `@dlt.table`). The platform compiles the query and identifies dependencies.

HOW IT REFRESHES

The platform chooses between full refresh (recompute) and incremental refresh (apply changes since last run). On Databricks, incremental refresh requires serverless pipelines, and some operations need row tracking on the source tables.

WHEN IT PAYS BACK

Use it for Gold-layer aggregations that downstream consumers query frequently: a materialised SUM/AVG over fact data refreshed every 15 minutes serves dashboards far faster than recomputing on every query. Not for ad-hoc analysis; the materialisation overhead does not pay back.

Streaming Tables.

LAYER FILES TABLE STREAM COMPUTE GOVERN PIPELINE SERVE OPERATE

A streaming table is a Delta table filled by Structured Streaming, usually declared in a Lakeflow pipeline. It reads streaming sources (Auto Loader, Kafka, Delta CDF) and writes new rows continuously or on micro-batch triggers. Append flows add rows; `AUTO CDC` flows apply updates and deletes. It is the primary Bronze ingest mechanism in Lakeflow pipelines.

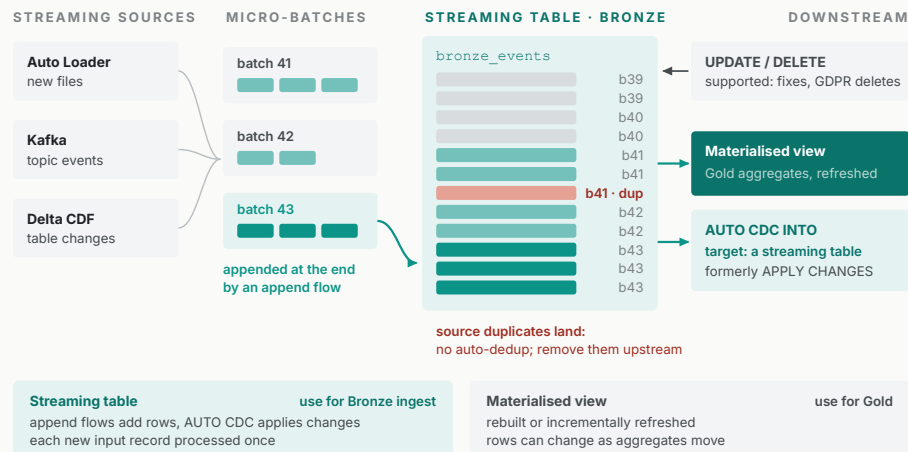


FIG. 10.9 Streaming sources feed micro-batches that an append flow adds to a Bronze streaming table, each new input record processed once, and source duplicates land unless removed upstream. Direct UPDATE and DELETE are supported for corrections and GDPR deletes; change feeds go through AUTO CDC INTO (formerly APPLY CHANGES INTO) into a streaming table, and aggregates go to a materialised view.

STREAMING TABLE OR MV

A **Streaming Table** processes each new input record once: good for raw event ingestion where rows arrive monotonically. A **Materialised View** recomputes aggregates and supports updates, since it is rebuilt or incrementally refreshed. Streaming Tables for Bronze, MVs for Gold.

APPLYING CHANGE FEEDS

Do not hand-write CDC with ad hoc DML. Apply change feeds with `AUTO CDC INTO` (formerly `APPLY CHANGES INTO`) a streaming table, and keep the supported direct `UPDATE` and `DELETE` for corrections and GDPR deletes.

DUPLICATES GET THROUGH

Streaming Tables do not auto-deduplicate. Source duplicates land unless watermarked dedup or Auto Loader file tracking removes them upstream.

Lakeflow Jobs.

LAYER FILES TABLE STREAM COMPUTE GOVERN PIPELINE SERVE OPERATE

Lakeflow Jobs (formerly Databricks Workflows) is the native service for running and scheduling Databricks tasks. A Job bundles tasks (notebooks, JARs, Lakeflow pipelines, SQL queries, dbt models, Python wheels/scripts) with their dependencies, retry policies, scheduling and notifications.

DEPENDENCIES AND FAILURES

The tasks in a Job form a DAG of dependency edges. A downstream task runs only when all its upstream tasks succeed. The DAG can fan out, one task triggering parallel children, and fan in, several tasks feeding one consolidation step. Retries are per task and independent: each task declares its own policy (count, interval, backoff, max duration). When an upstream task fails, the downstream task is marked **Upstream failed** and counts as failed, unless its **Run if** condition (for example `All done` or `At least one failed`) says otherwise. A task whose condition is not met is marked Excluded under `At least one failed` or `All failed`, and Upstream failed under the other conditions.

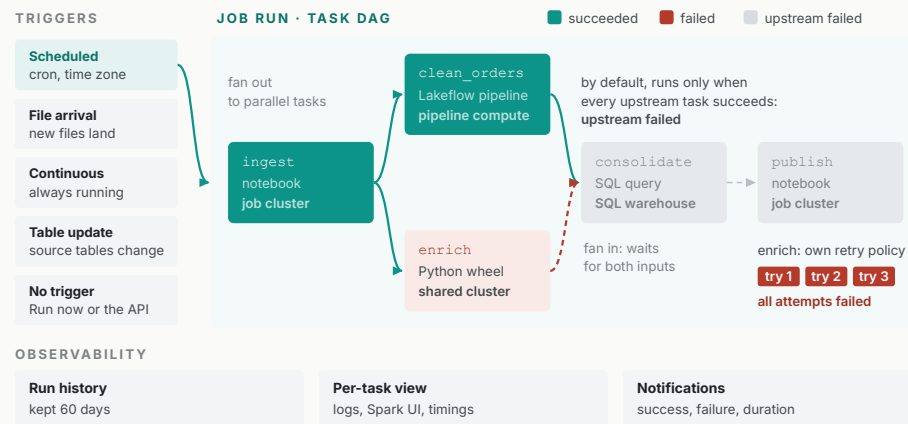


FIG. 10.10 A trigger starts one Lakeflow Jobs run: tasks form a DAG that fans out and back in, each task choosing its own compute. When a task fails after its retries, every task that depends on it is marked Upstream failed unless its Run if condition says otherwise, and the run page shows exactly where.

WHERE TASKS RUN

Compute is chosen per task: serverless compute (the UI default), classic jobs compute or all-purpose compute. A classic **job cluster** is created for the run and terminated after it, and several tasks in the run can share one. Give a task its own cluster to isolate its environment; share one to save startup time when tasks are short and run in sequence. SQL tasks run on SQL warehouses and pipeline tasks on pipeline-managed compute.

STARTING A RUN

Lakeflow Jobs supports several trigger types. **Scheduled** takes cron expressions and is time zone aware. **File arrival** starts a run when new files land in a monitored location. **Table update** starts one when source tables change. **Continuous** keeps the Job always running, for streaming pipelines. **Model update** (Beta) starts a run when a registered model changes. With no trigger, start runs from the UI with **Run now** or through the Jobs API.

WATCHING RUNS

Run history, task-level logs and metrics are retained for 60 days. Open a run page and you get the DAG with status per task, links to driver logs and the Spark UI, and timing breakdowns; the job timeline shows where a failure occurred and which downstream tasks did not run. Email and webhook notifications fire on success, failure or duration thresholds, and failure notifications can also go to Slack. For external monitoring, read run metadata from the Jobs REST API.

WHEN TO USE SOMETHING ELSE

Lakeflow Jobs is the right choice for pipelines that live entirely within Databricks. Once a pipeline spans several systems (Databricks plus Snowflake plus an API call plus a Kubernetes job), a dedicated orchestrator such as Azure Data Factory, Apache Airflow or Prefect gives you broader connector coverage, cross-platform visibility and more sophisticated dependency semantics across heterogeneous task types.

05

Microsoft Fabric.

The same Spark engine, running inside a Fabric capacity that every workload shares. This part covers OneLake and shortcuts as the one storage layer, the runtime and resource profiles that set write defaults such as V-Order, the notebooks, environments and pools your code runs in, and how the capacity is billed and throttled.

XI Running PySpark on Fabric

82	OneLake	77
83	OneLake Shortcuts	77
84	Delta-Parquet as Native Format	78
85	Fabric Runtime 2.0 and resource profiles	78
86	Notebooks and Spark Job Definitions	79
87	High concurrency mode	80
88	Fabric Environments	81
89	Materialized lake views	82
90	Starter vs Custom Pools	83
91	Capacity Units and Bursting	83
92	On-demand billing for Spark	84

READING PATH

Layer heat: A B C D E F G H = Files to Operate

XI

How does Fabric host, store, serve and bill Spark?



OneLake.

Every lakehouse and warehouse in a Fabric tenant stores its data in OneLake automatically. It is one logical, tenant-wide data lake spanning all Fabric workspaces and items: one per organisation, the way OneDrive is one drive per person. An Import semantic model writes Delta tables there only when OneLake integration is turned on.

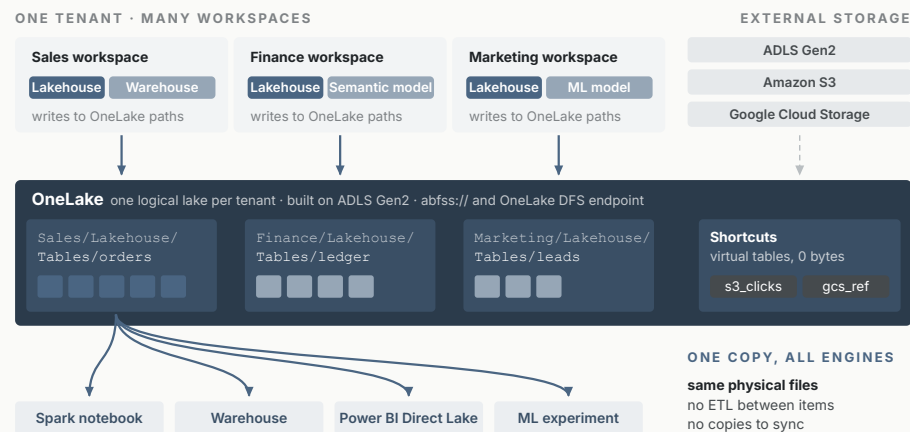


FIG. 11.1 Every workspace in the tenant writes its items into one logical OneLake built on ADLS Gen2; shortcuts link external storage in without copying, and every engine reads the same Delta files.

PATHS AND ENDPOINTS

OneLake is built on ADLS Gen2. You reach data through the ADLS-compatible `abfss://` protocol or through the OneLake-specific DFS endpoint, and every Fabric item writes its data to paths within OneLake.

REACHING OTHER STORES

Shortcuts let one workspace reference data in another workspace, in ADLS, in S3 or in GCS without copying it.

ONE COPY FOR EVERY ENGINE

Data written to a Fabric Lakehouse is immediately accessible to every other Fabric item with the right permissions, with no ETL in between, so Fabric items do not turn into data silos. Write a Delta table from a Spark job in a Lakehouse and the SQL analytics endpoint or a Warehouse, a Power BI Direct Lake model and an ML experiment can all query it directly. They read the same physical files, with no copies or pipelines between them.

OneLake Shortcuts.

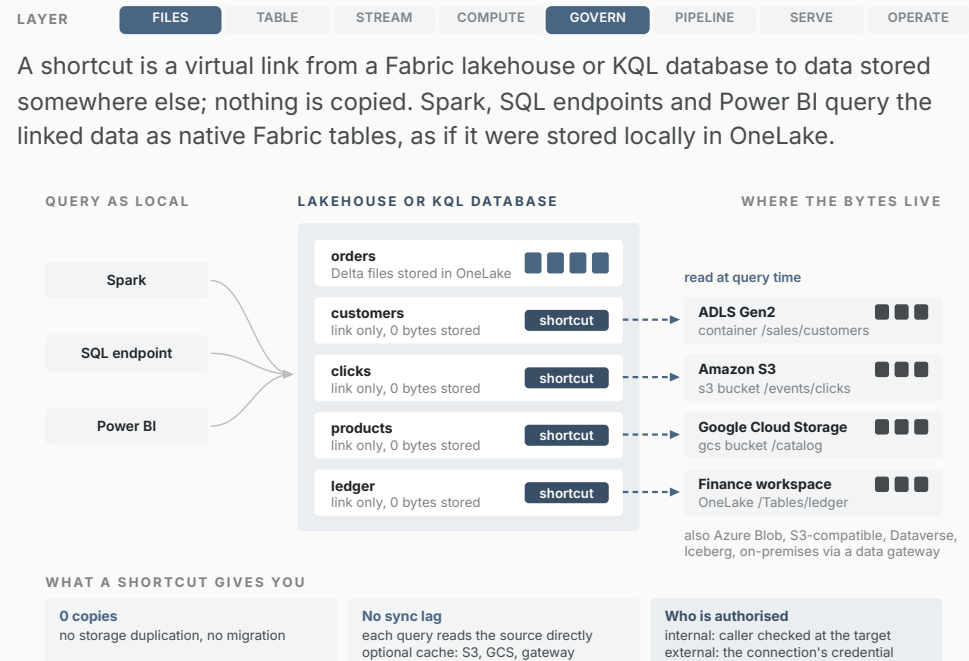


FIG. 11.2 A shortcut is a virtual entry in a lakehouse or KQL database that points at data in another OneLake location, ADLS, S3, GCS or other sources; engines query it as if it were local, and the read goes to the source at query time unless shortcut caching holds a copy.

CREATING ONE

Create it in the Fabric UI or via API, giving the source storage path and credentials. Sources include other OneLake locations, ADLS Gen2, Azure Blob Storage, Amazon S3 and S3-compatible storage, Google Cloud Storage, Dataverse, Iceberg tables, and on-premises storage through a data gateway.

WHAT HAPPENS AT QUERY TIME

Queries read the source when they run, so storage is not duplicated. Shortcut caching can keep copies of files from S3, GCS and gateway sources. Permissions depend on the kind of shortcut: an internal OneLake shortcut authorises the calling user against the target's permissions, while an external shortcut authorises through the connection's credential.

WHEN TO USE ONE

Use shortcuts to expose existing data lake investments to a Fabric workspace without migration. Years of Parquet and Delta data on ADLS become queryable in Fabric from Spark and Power BI immediately, and not a single file moves.

Delta-Parquet as Native Format.

LAYER **FILES** **TABLE** STREAM COMPUTE GOVERN PIPELINE **SERVE** OPERATE

Create a table in a Fabric Lakehouse and you get Delta over Parquet, the native and default format, because `spark.sql.sources.default=delta`. The whole platform is built on Delta as its storage layer; it is no add-on. Spark can still create Parquet, CSV or legacy Hive format tables, but the SQL analytics endpoint and Direct Lake read only Delta, and external data usually comes in through shortcuts.

WRITING A TABLE

Write with `df.write.format('delta').save(path)` or run `spark.sql('CREATE TABLE ...')` and you get a Delta table in OneLake. V-Order, the write-time sort that optimises column loading for the Fabric engine, is off by default for Spark writes in new workspaces (see V-Order).

WHO READS THE FILES

The SQL analytics endpoint, Direct Lake, Power BI and Shortcuts all read those same Delta files. Items never negotiate a format: a Delta file written by Spark is readable by the SQL endpoint and Power BI straight away, with no transformation.

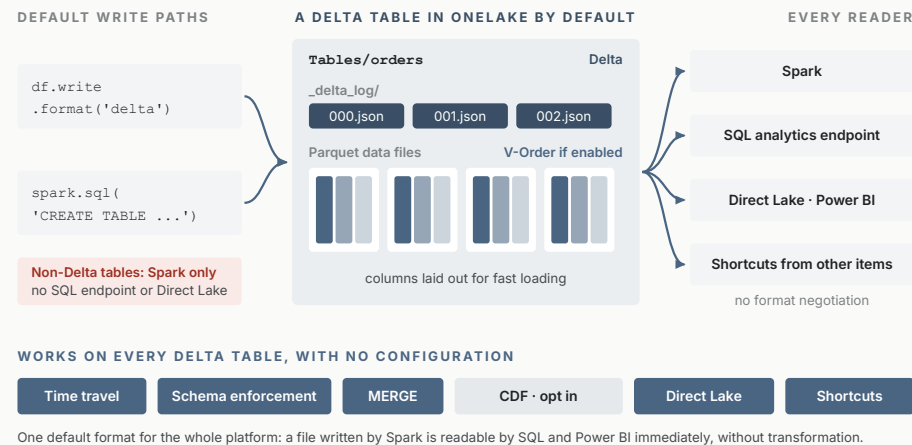


FIG. 11.3 Every default write path in a Fabric Lakehouse produces the same thing: a Delta table of Parquet files (V-Ordered when V-Order is enabled) plus a transaction log, which every other Fabric engine reads directly. Change Data Feed is the exception to "no configuration": it is turned on per table.

WHAT WORKS WITHOUT SETUP

Every Fabric engine reads Delta, so Direct Lake, Shortcuts, time travel, schema enforcement and MERGE work on every Delta table with no configuration. There are two gaps. Newer Delta 4.x features such as VARIANT columns and V2 checkpoints work only in Spark. Change Data Feed still has to be turned on per table with `delta.enableChangeDataFeed`.

Fabric Runtime 2.0 and resource profiles.

LAYER **FILES** **TABLE** STREAM **COMPUTE** GOVERN PIPELINE **SERVE** OPERATE

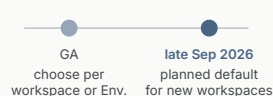
Fabric Runtime 2.0, now generally available, runs Apache Spark 4.1, Delta Lake 4.2, Java 21, Scala 2.13, Python 3.13 and R 4.5.2 on Azure Linux 3.0. **Resource profiles** bundle write settings behind one property, `spark.fabric.resourceProfile`, so you tune a workspace for ingestion or for reads without setting each config by hand.

FABRIC RUNTIME 2.0 · GA

Python 3.13 · R 4.5.2
Delta Lake 4.2
Apache Spark 4.1
Java 21 · Scala 2.13
Azure Linux 3.0

NEE supported, set per Environment
SparkR deprecated in Spark 4.x

DEFAULT RUNTIME



RESOURCE PROFILES · SPARK.FABRIC.RESOURCEPROFILE

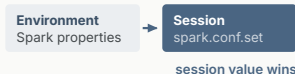
	writeHeavy default	readHeavyForSpark Spark readers	readHeavyForPBI Power BI, Direct Lake
vorder.default	false	not set	true
optimizeWrite	unset	true	true
binSize	128 MB	128 MB	1 GB
partitioned OW	true	true	not set

spark.sql.parquet.* and spark.databricks.delta.optimizeWrite.* prefixes omitted

V-Order is off for new Spark writes

Gold tables for Direct Lake lose the VertiPaq layout unless the profile is readHeavyForPBI or V-Order is set

WHERE IT IS SET



check: spark.version returns 4.1.x

spark.conf.get("spark.fabric.resourceProfile")

FIG. 11.4 Runtime 2.0 stacks Spark 4.1 and Delta 4.2 on Java 21 and Python 3.13; resource profiles switch a bundle of write settings at once, and the writeHeavy default for new workspaces turns V-Order off.

PICKING A PROFILE

There are four values. `writeHeavy` is the default for all new workspaces: it sets `spark.sql.parquet.vorder.default=false`, leaves `optimizeWrite.enabled` unset, and uses a 128 MB optimise-write bin with partitioned optimise write on. `readHeavyForPBI` turns V-Order on, enables optimise write and raises the bin to 1 GB; use it for tables served to Power BI. `readHeavyForSpark` enables optimise write with 128 MB bins and partitioned optimise write. `custom` leaves everything to you. Set the profile in an Environment's Spark properties, or per session with `spark.conf.set("spark.fabric.resourceProfile", "readHeavyForSpark")`. The session value wins. If jobs behave differently from what the Environment says, look for a notebook cell that overrides the profile.

V-ORDER IS OFF

Older guidance describes V-Order as the default. In a new workspace it is off, so Gold tables feeding Direct Lake lose the VertiPaq layout unless the profile is `readHeavyForPBI` or V-Order is enabled for those tables. Check with `spark.conf.get("spark.sql.parquet.vorder.default")`.

CHECK WHAT IS ACTIVE

On Runtime 2.0, `spark.version` returns 4.1.x.

`spark.conf.get("spark.fabric.resourceProfile")` shows the active profile. Read the individual configs it sets as well, to confirm nothing overrode them later in the session.

BEFORE YOU UPGRADE

Runtime 2.0 is GA, and Microsoft plans to make it the default for new workspaces and Environment items in late September 2026; existing workspaces keep their runtime until changed. Custom JARs must target Scala 2.13 and Java 21, so test libraries built for Scala 2.12 before you move pipelines to 2.0. SparkR is deprecated in Spark 4.x. The Native Execution Engine is supported and is enabled at the Environment level.

RELATED CONCEPTS

V-Order is what `readHeavyForPBI` turns on. OPTIMIZE and Bin-packing and the Small-file Problem explain optimise write. Fabric Environments carry the profile. NEE (Native Execution Engine) runs on the runtime.

Notebooks and Spark Job Definitions.

LAYER	FILES	TABLE	STREAM	COMPUTE	GOVERN	PIPELINE	SERVE	OPERATE
-------	-------	-------	--------	---------	--------	----------	-------	---------

You develop in a **notebook**: an interactive, multi-language (PySpark, Scala, SQL, R) environment attached to a Lakehouse. You deploy with a **Spark Job Definition (SJD)**, the production artifact for Spark code: a packaged Python, Scala or R job that runs on a Fabric Spark cluster with no notebook interface.

WORKING IN A NOTEBOOK

You choose the notebook's default lakehouse, and relative paths and table names resolve against it. Run cells interactively, or let a scheduled pipeline trigger run them.

SETTING UP AN SJD

Upload a `.py`, `.jar` or R file, choose the main definition file and attach a default lakehouse, which an SJD requires. Schedule it with the item's own schedule or from a data pipeline. SJDs run on the same starter or custom pools as notebooks; Fabric has no separate job-cluster concept.

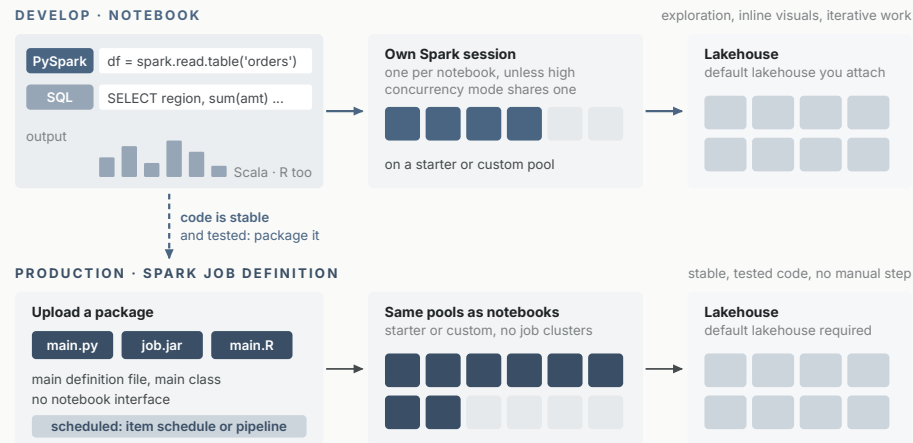


FIG. 11.5 Notebooks run code cell by cell in their own Spark session (high concurrency mode can share one) against the default lakehouse you attach; once the code is stable, it is packaged as a Spark Job Definition that runs on the same starter or custom pools, on an item schedule or from a data pipeline.

WHICH ONE TO REACH FOR

Develop, explore and iterate in a notebook, where inline visualisation and cell-by-cell execution earn their keep. Once the code is stable and tested, move scheduled production jobs to an SJD so they run without a notebook interface or manual intervention.

High concurrency mode.

LAYER FILES TABLE STREAM **COMPUTE** GOVERN PIPELINE SERVE OPERATE

In Fabric Spark, several notebooks, or several notebook activities in a pipeline, can share one running Spark session instead of each starting its own. Each notebook gets its own **REPL core**, so variables and execution state stay isolated while the driver, executors and warm startup are shared.

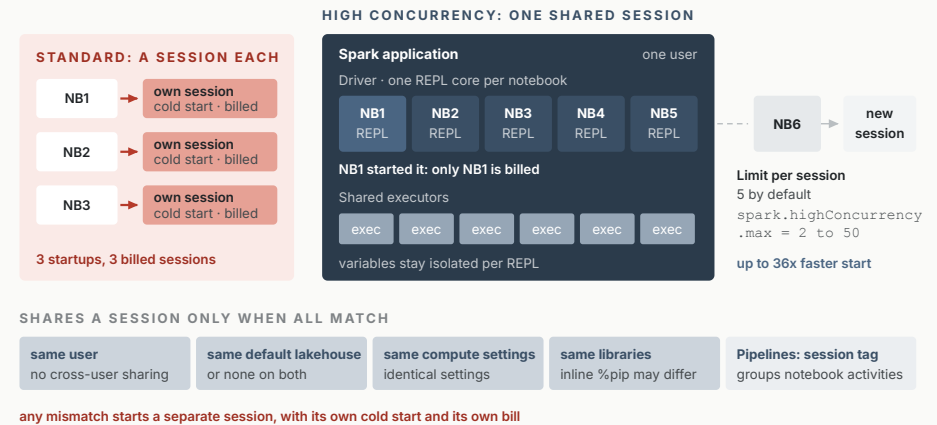


FIG. 11.6 In standard mode every notebook starts and pays for its own session; in high concurrency mode up to 5 notebooks from the same user share one Spark application, each in its own REPL, and only the notebook that started it is billed.

WHEN NOTEBOOKS SHARE

Every condition has to hold: the **same user**, the **same default lakehouse** (or none), the **same Spark compute settings** and the same library packages. Inline `%pip` installs may differ. Miss one and Fabric starts a separate session, which is why notebooks pointed at different default lakehouses each get their own. Sharing never crosses users, so a team does not pool one session.

TURNING IT ON

For interactive work the workspace setting **For notebooks** is on by default. On the Run tab, choose **New high concurrency session** from the session type list, then attach other notebooks to it. For pipelines, enable **For pipeline running multiple notebooks**; notebook activities are then packed into existing sessions. A **session tag** in the activity's advanced settings groups the notebooks that should share, so fill it in whenever only some of them should. A session holds 5 notebooks by default; `spark.highConcurrency.max` in an Environment raises it to between 2 and 50. Past the limit, a new session starts.

WHAT YOU SAVE, WHAT YOU SHARE

Session start can be up to 36 times faster than a standard session on custom pools. Only the notebook or pipeline activity that started the shared application is billed, and Capacity Metrics reports all usage against it. A pipeline with ten small notebooks stops paying ten cold starts. In exchange, shared notebooks compete for the same executors, so one heavy notebook slows the others. State stays separate: a DataFrame defined in one notebook is not visible in another.

CHECK IT ATTACHED

The notebook toolbar shows the high concurrency session it is attached to. The Monitoring hub lists one Spark application for all attached notebooks, and Capacity Metrics shows usage only under the notebook or activity that started it.

RELATED CONCEPTS

Notebooks and Spark Job Definitions, Starter vs Custom Pools, Fabric Environments, Capacity Units and Bursting.

Fabric Environments.

LAYER FILES TABLE STREAM COMPUTE GOVERN PIPELINE SERVE OPERATE

You put the Spark compute settings (runtime version included), libraries and resource files into one Fabric item, the Environment, and attach it to the notebooks and SJDs that need them. It is reusable and versioned, so the software stack lives apart from individual items. Spark then runs consistently and reproducibly across the workspace without configuring each item separately.

SETTING ONE UP

Create an Environment in the Fabric UI, pick the Spark runtime version, add pip or conda packages, and set Spark configuration properties such as executor memory and shuffle partitions. Attach it to a Notebook or SJD from the item settings panel. When a Spark session starts, Fabric provisions the cluster with the Environment's full package set and configuration before any code runs. The Environment holds the runtime version, Python and Spark library versions, custom packages, resource files and Spark properties. Notebooks, pipelines, lakehouses and semantic models belong to the workspace. Several notebooks can share one Environment for consistent dependencies, and switching a notebook to another Environment changes the runtime without changing your code.

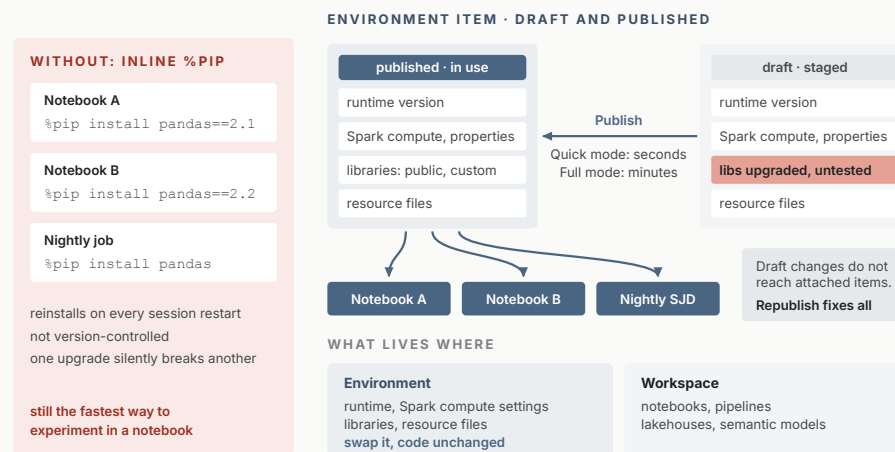


FIG. 11.7 Inline pip installs drift per notebook; an Environment holds one published set of Spark compute settings, libraries and resources that every attached notebook and job uses, while edits wait in a draft until published (Quick mode in seconds, Full mode several minutes).

PUBLISHING CHANGES

Edits stay in draft and do not affect attached items until you save and publish the Environment. That two-phase model is what stops a library update during development from breaking production jobs that are running. Quick mode publishes in seconds. Full mode installs the library set and takes several minutes, because a new Python environment has to be built and distributed across compute nodes; custom libraries also lengthen session start.

ENVIRONMENTS OR %PIP

Without Environments, each notebook manages libraries with `%pip install` commands. They run at session start on every restart, are not version-controlled and differ between notebooks in unpredictable ways, so a package upgrade in one notebook silently breaks another that assumed the prior version. An Environment is a single source of truth for the stack: dependency upgrades become reviewable and reversible before deployment, and every production SJD in the workspace runs the same tested library versions. When a library introduces a breaking change, fix the Environment and republish; every attached item is repaired without touching notebook code. Keep Environments in Git integration to track changes. For rapid iteration in development, inline `%pip install` in a notebook is faster. Experiment with inline installs; run production on Environments.

FABRIC ONLY

Environments exist only in Microsoft Fabric, and Fabric Spark has no init scripts. On Databricks the equivalent is Cluster Libraries for cluster-level packages and init scripts for system-level configuration, with Declarative Automation Bundles for version-controlled deployment.

RELATED CONCEPTS

Notebooks and Spark Job Definitions consume Environments. Cluster Policies and DBR is the analogous concept on Databricks.

Materialized lake views.

LAYER FILES TABLE STREAM COMPUTE GOVERN PIPELINE SERVE OPERATE

You write a Spark SQL query; Fabric persists the result as a Delta table in the Lakehouse and refreshes it for you. With materialized lake views (MLVs) a medallion pipeline becomes a chain of `SELECT` statements instead of notebooks that rewrite tables. They need a schema-enabled lakehouse. SQL authoring is generally available; PySpark authoring is in preview and always does a full refresh.

DEFINING ONE

A typical definition: `CREATE OR REPLACE MATERIALIZED LAKE VIEW silver.orders (CONSTRAINT qty_pos CHECK (quantity > 0) ON MISMATCH DROP) PARTITIONED BY (category) AS SELECT ...`. Constraints are Boolean checks. **FAIL**, the default, stops the refresh at the first bad row. **DROP** removes failing rows, and the lineage view shows how many. If you expect bad rows, use DROP instead of relying on the default.

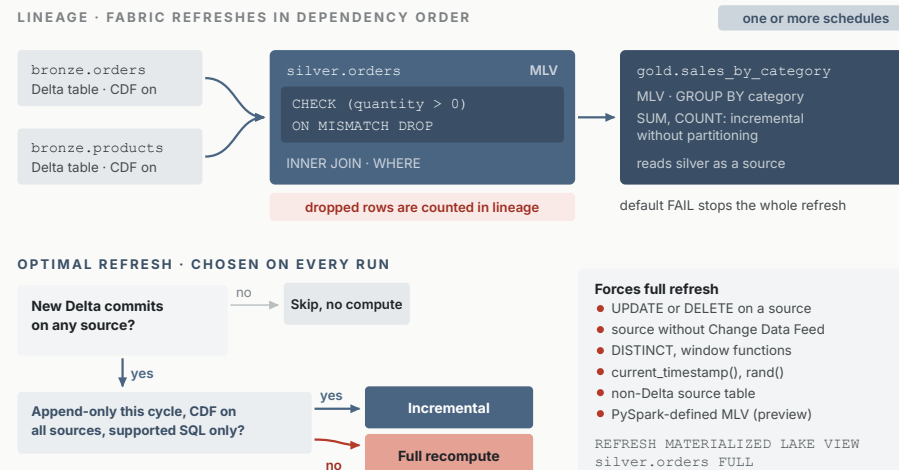


FIG. 11.8 Two Bronze tables feed a Silver MLV with a DROP constraint and a Gold MLV that Fabric refreshes in lineage order; on each run optimal refresh skips, refreshes incrementally, or falls back to a full recompute.

HOW REFRESH DECIDES

Fabric reads the dependencies between MLVs, across lakehouses too, draws them as lineage and refreshes them in order on one or more schedules. With **optimal refresh** on, each run picks a strategy: skip when no source table has new Delta commits, **incremental** when sources are append-only and have `delta.enableChangeDataFeed=true`, otherwise **full**. `SUM`, `MIN`, `MAX` and `COUNT` refresh incrementally without partitioning; other aggregates need a partitioned source with the partition column in the `GROUP BY`. To force a rebuild, run `REFRESH MATERIALIZED LAKE VIEW silver.orders FULL`.

WHY RUNS GO FULL

Forget Change Data Feed on a source and every run silently goes full. Run `UPDATE` or `MERGE` on a source and the same happens: any update or delete in the cycle falls back to full. `DISTINCT`, window functions and `current_timestamp()` block incremental refresh. Some things do not work at all: `INSERT` into an MLV, time travel on it, UDFs, and temporary views as sources.

WHERE THEY FIT

Orchestration, dependency order, data quality counts and monitoring come built in. Recent runs, trends and failures show in the lakehouse, and system tables in the `_mlv_system` schema expose run history to queries. Use MLVs for SQL-expressible Silver and Gold transforms over Delta sources that refresh on a schedule. Keep ML inference, API calls, UDF logic and sub-second streaming elsewhere.

RELATED CONCEPTS

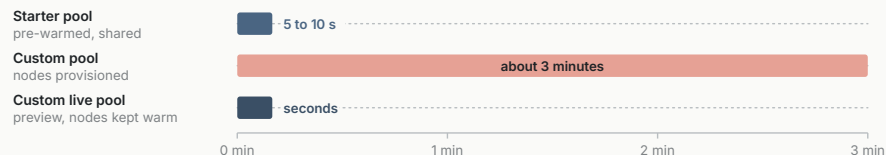
Materialised Views and Lakeflow pipelines (formerly DLT) are the Databricks equivalents. Change Data Feed powers incremental refresh. Schema Enforcement and Evolution covers the table side.

Starter vs Custom Pools.

LAYER FILES TABLE STREAM **COMPUTE** GOVERN PIPELINE SERVE OPERATE

Fabric Spark runs on the **Starter pool** by default: shared, pre-warmed compute that every workspace user gets immediately, with no configuration. When a workload needs more control, create a **custom pool** for the workspace with the node size and autoscaling bounds you choose.

TIME TO FIRST SPARK SESSION



Starter pool		Custom pool	
capacity	pre-warmed, shared	defined in	workspace settings
setup	none, on by default	node family	your choice
node size	Medium	autoscale	min and max nodes
availability	every workspace user	runtime	not here: workspace or Environment
Use for: development, light workloads		Use for: production, controlled scaling	

Both pool types consume the same capacity units and accept an attached Environment for runtime, libraries and Spark config.

FIG. 11.9 Starter pools start a session in 5 to 10 seconds from pre-warmed Medium nodes; custom pools let you choose node size and autoscale range, at the cost of about 3 minutes to start, unless a custom live pool (preview) keeps nodes warm. The runtime comes from workspace settings or the Environment, not the pool.

HOW FAST THEY START

The Starter pool starts a session in 5 to 10 seconds from pre-warmed Medium nodes. A custom pool provisions nodes on demand, so a session takes about 3 minutes to start. Custom live pools (preview) keep nodes warm and start in seconds.

DEFINING A CUSTOM POOL

Define custom pools in workspace settings: pick the node family, min/max nodes and autoscaling range. The pool does not set the runtime version; that comes from workspace settings or the Environment.

WHICH TO PICK

Use Starter for development and light workloads where instant availability matters. Move to a custom pool for production jobs that need larger nodes, specific memory or controlled autoscaling, and pin the runtime version in an Environment.

RELATED CONCEPTS

Fabric Environments can be attached to either pool type to control libraries and Spark configuration. Capacity Units govern how much compute both pool types can consume before throttling applies.

Capacity Units and Bursting.

LAYER FILES TABLE STREAM **COMPUTE** GOVERN PIPELINE SERVE OPERATE

Every Fabric operation pays in **Capacity Units (CUs)**: Spark jobs, SQL queries, Power BI refreshes. A Fabric capacity (F SKU) gives you a fixed CU rate per second. **Bursting** lets an operation run faster than that provisioned rate, and **smoothing** then spreads its cost over future timepoints.

HOW A SPIKE IS PAID BACK

Idle periods bank nothing, so a spike borrows future capacity. Interactive operations are smoothed over at least 5 minutes (up to 64), background operations over 24 hours. Keep borrowing and throttling escalates in stages. Up to 10 minutes of future use is overage protection. From 10 to 60 minutes, new interactive requests are delayed 20 seconds. From 60 minutes to 24 hours, interactive requests are rejected, and beyond 24 hours background requests are rejected too. Operations already running always finish.

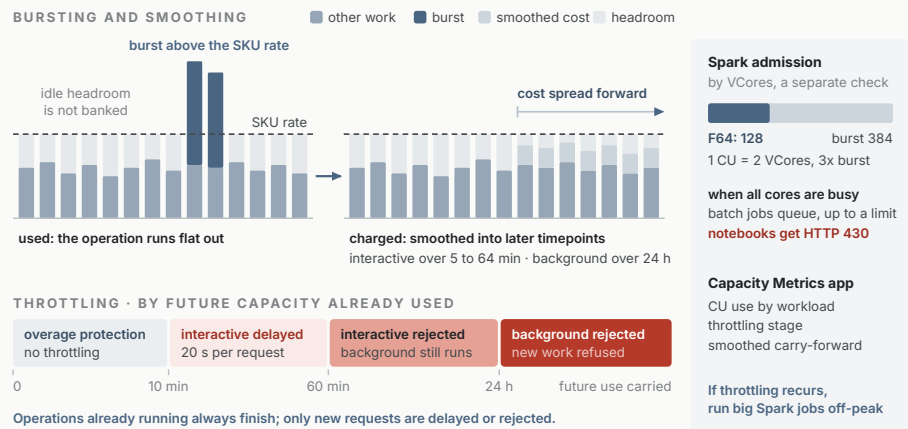


FIG. 11.10 Bursting lets an operation run above the SKU rate, and smoothing spreads its cost over later timepoints (interactive over 5 to 64 minutes, background over 24 hours); nothing is banked from idle time. Throttling escalates by how much future capacity is already used, running operations always finish, and Spark is admitted separately by VCores.

HOW SPARK GETS ADMITTED

Spark is admitted separately by VCores: 1 CU = 2 VCores, bursting up to 3x. When the cores are full, pipeline, scheduled and job definition runs queue, up to a per-SKU queue limit (trial capacities have none), and notebooks get HTTP 430. A throttled capacity rejects new Spark jobs instead of queueing them.

READING THE METRICS APP

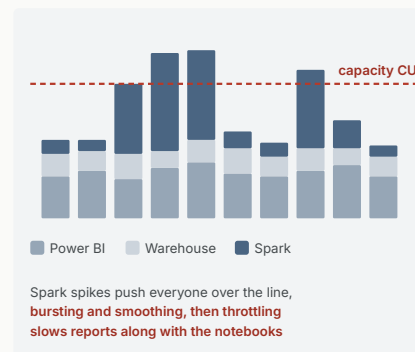
Open the Fabric Capacity Metrics app for CU consumption by workload, the throttling stage and the carry-forward of smoothed usage. At 95% sustained utilisation a capacity has no burst headroom; at 60% it can absorb significant spikes. If throttling recurs, schedule the large Spark jobs off-peak.

On-demand billing for Spark.

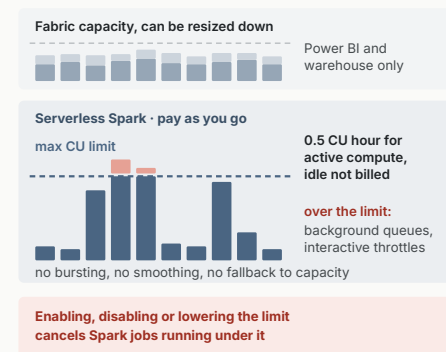
LAYER FILES TABLE STREAM **COMPUTE** GOVERN PIPELINE SERVE OPERATE

Turn on On-demand billing for Spark, an opt-in setting per capacity, and Fabric Spark leaves the shared capacity for dedicated serverless compute billed pay-as-you-go. Spark jobs stop consuming the capacity's CUs, so they no longer compete with Power BI, warehouse or pipeline workloads. It became generally available in July 2025 and needs an F SKU, F2 or larger; P SKUs and trial capacities are not supported.

CAPACITY MODEL · SPARK SHARES CUS



ON-DEMAND BILLING ON · OPT IN PER CAPACITY



F SKUs from F2 · set by the capacity admin · cost meter: Autoscale for Spark Capacity Usage CU

FIG. 11.11 Under the capacity model Spark spikes share CUs with Power BI and warehouse work; with On-demand billing for Spark, Spark runs on separate pay-as-you-go compute capped by a max CU limit, where excess jobs queue or throttle instead of bursting.

TURNING IT ON

A Fabric capacity administrator switches on **On-demand billing for Spark** in the capacity settings and sets a **Maximum Capacity Units** limit with the slider. The ceiling depends on the subscription's approved Azure quota; raise it through Azure Quotas. Active Spark compute is billed at 0.5 CU hour; idle compute is not billed. The limit also caps concurrency: in Microsoft's example, a 2048 CU limit lets the Spark queue hold up to 2048 jobs. It covers interactive notebooks, lakehouse operations such as Load to Table and table preview, and background jobs from pipelines, schedules, APIs and Spark job definitions. Usage appears under the meter `Autoscale for Spark Capacity Usage CU` in Azure Cost Analysis, and separately in the Capacity Metrics app.

AT THE LIMIT

There is no bursting or smoothing, and Spark never falls back to the capacity. When the CU limit is reached, **interactive jobs are throttled** and **background jobs queue**. A limit set too low shows up as queued pipelines, and that is no platform fault. You also give up a fixed bill for a variable one. Enabling, disabling or lowering the limit **cancels active Spark jobs** running under On-demand billing for Spark. Make those changes outside business hours.

WHEN TO MOVE SPARK OFF

Move it for ad hoc, exploratory or highly variable Spark load, or when Spark spikes throttle reports on a shared capacity. Then resize the capacity down for the workloads that remain: pause, wait about 5 minutes, resume, then resize. Leave it at its old size and you pay twice for Spark. For steady, always-on mixed workloads where you want predictable cost, stay on the capacity model.

RELATED CONCEPTS

Capacity Units and Bursting explains the shared model it replaces for Spark. Starter vs Custom Pools still decide node sizes. Classic vs serverless compute covers the Databricks analogue. High concurrency mode cuts the number of sessions billed.

BACK MATTER

Compute, utilities and reference.

Pages that apply to both platforms. Chapter XII covers the work around Spark: choosing classic or serverless compute, the notebook utilities, and running jobs in production. Chapter XIII starts from a symptom you are looking at, or a choice you are stuck on, and points you to the page that explains it. The feature matrix and glossary follow.

XII Compute choices, utilities and testing		
93	Classic vs serverless compute	87
94	Notebook utilities: dbutils and notebookutils	88
95	Operating in production	89
XIII Reference		
96	Diagnose by Symptom	90
97	Which Tool When	92
T1	Where each feature lives	94
	Glossary	96
	Conclusion	98

READING PATH

Layer heat: A B C D E F G H = Files to Operate



Classic vs serverless compute.

LAYER FILES TABLE STREAM **COMPUTE** GOVERN PIPELINE SERVE OPERATE

You either define the compute yourself or let the platform define it. On Azure Databricks, **classic compute** (all-purpose, jobs and pipeline clusters) runs VMs in your own Azure subscription, and **serverless compute** for notebooks, jobs and Lakeflow pipelines runs in a serverless compute plane managed by Databricks. Fabric has no customer-side VMs: you choose between the pre-warmed **starter pool** and a **custom pool**, and separately between capacity billing and **On-demand billing**.

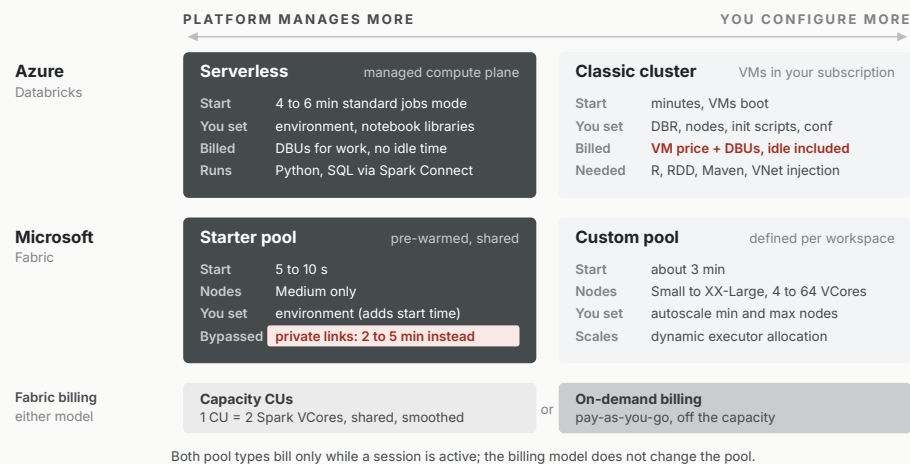


FIG. 12.1 Azure Databricks offers serverless compute in a Databricks-managed plane or classic clusters in your own subscription; Fabric offers a pre-warmed starter pool or a custom pool, billed either against capacity CUs or through On-demand billing for Spark.

WHAT YOU PICK AND PAY FOR

On classic you pick the runtime, node type, access mode, init scripts, cluster libraries and Spark properties, and you pay the VM price plus DBUs from start to termination, idle time included. On serverless you pick an environment and notebook-scoped libraries; Databricks sizes the compute and bills DBUs. Serverless jobs default to **standard** performance mode, with 4 to 6 minutes of startup latency and lower DBU use. The **performance optimized** mode starts faster for more DBUs. In Fabric, starter pools use Medium nodes only and start in 5 to 10 seconds. Custom pools offer Small (4 VCores) to XX-Large (64 VCores) nodes with autoscale bounds and start in about 3 minutes. Both bill only while a session is active, against capacity CUs (1 CU = 2 Spark VCores), unless On-demand billing moves Spark to pay-as-you-go.

WHAT SERVERLESS RULES OUT

Databricks serverless requires Unity Catalog and runs Python and SQL through Spark Connect only. That means no R, no RDD APIs, no Scala notebooks, no compute policies, init scripts, instance pools or Maven libraries, and `cache()`, `persist()` and `CACHE TABLE` raise an exception. You cannot set most Spark properties, `ProcessingTime` and continuous triggers are unsupported, and runs stop after 7 days. On networking, classic clusters can sit in your VNet; serverless reaches private resources through network connectivity configurations (NCCs) and private endpoints. In Fabric, managed VNets and private links bypass starter pools, so sessions take 2 to 5 minutes.

CHOOSING ONE

Default to serverless for notebooks, scheduled ETL and pipelines that fit those restrictions: you get no idle billing and no cluster tuning. Go classic for R, RDD code, init scripts, deep Spark tuning, VNet-injected networking, or libraries serverless cannot install. In Fabric, stay on the starter pool for development and use a custom pool for larger nodes, fixed autoscale bounds or heavy environments.

WHERE MOVES GO WRONG

Test any notebook that calls `df.cache()` or `sc.parallelize` before you move it to serverless. When you compare a serverless bill with an all-purpose cluster, count the cluster's idle hours. Do not expect 5 second starts in a Fabric workspace with a managed VNet or large custom libraries. On-demand billing does not give you a different engine: pools and node sizes still apply.

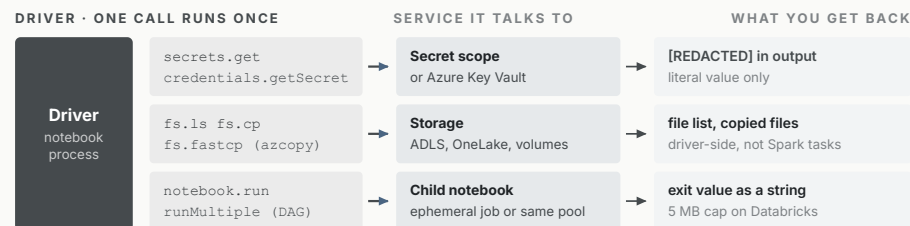
RELATED CONCEPTS

All-purpose vs Job Clusters, Access modes, Starter vs Custom Pools, On-demand billing for Spark, Fabric Environments, Capacity Units and Bursting.

Notebook utilities: dbutils and notebookutils.

LAYER **FILES** TABLE STREAM COMPUTE GOVERN PIPELINE SERVE OPERATE

Each platform ships a helper library for the work around Spark: files, secrets, parameters and orchestration. Azure Databricks has `dbutils`. Fabric has `notebookutils`, the new name for `MsSparkUtils`; `mssparkutils` code stays backward compatible, but Microsoft says that namespace will be retired and recommends switching. NotebookUtils needs Runtime 1.2 (Spark 3.4) or later.



EXECUTORS

UDF, foreach and mapPartitions bodies run on executors: calling dbutils there gives unexpected results or errors

SAME JOBS, TWO NAMES

	Azure Databricks: dbutils.	Microsoft Fabric: notebookutils.
Files	fs.ls, fs.cp, fs.mount	fs.ls, fs.cp, fs.fastcp, fs.mount
Secrets	secrets.get(scope, key)	credentials.getSecret(vault, name)
Run notebooks	notebook.run, notebook.exit	notebook.run, runMultiple, exit
Old name	none	mssparkutils: still works, to retire

FIG. 12.2 Every dbutils or notebookutils call runs once in the driver process and calls a service: a secret store, storage or a child notebook. Calling them from executor code fails, and each module has a counterpart on the other platform.

WHAT YOU CAN CALL

Both are driver-side APIs. Each call runs once in the notebook process and talks to a service; nothing is distributed. Databricks warns that calling `dbutils` inside executors gives unexpected results or errors. `dbutils` gives you `fs`, `secrets` (Databricks-backed or Azure Key Vault-backed scopes), `widgets` for parameters, `notebook.run` with `notebook.exit`, and `jobs.taskValues`. `notebook.run` starts an ephemeral job and caps the return string at 5 MB; `jobs.taskValues` passes up to 250 values of at most 48 KiB JSON each between job tasks. `notebookutils` gives you `fs` (`fastcp` uses `azcopy` for bulk copies), `credentials.getSecret` for Key Vault and `getToken` for Entra tokens, `notebook.run` (default timeout 90 seconds, same Spark pool as the caller), `runMultiple` for a DAG of notebooks, plus `lakehouse`, `runtime` and `variableLibrary`.

SECRETS AND FAN-OUT

Secrets stay out of code, and both platforms print a retrieved secret as `[REDACTED]`. Redaction matches only the literal value, so a transformed secret prints in clear, and anyone who can run code with the secret can read it. Child notebooks are cheap because they share the caller's compute. The flip side is that `runMultiple` runs them as threads in one session, with default concurrency of 3 times the available cores, so a wide fan-out causes memory pressure and CPU contention.

CONNECT, SERVERLESS AND MOUNTS

Through Databricks Connect you get only `dbutils.fs` and `dbutils.secrets`, via the SDK's `WorkspaceClient`. `dbutils.fs.mount` is not compatible with serverless; Databricks recommends Unity Catalog volumes and external locations instead. On Fabric, `fs.mount` attaches remote storage to Spark nodes as local paths.

WHERE SCRIPTS TRIP

Looping `fs.cp` over thousands of files gets you no Spark parallelism. Do not wrap `notebookutils.notebook.exit` in try/except; it must be at top level. Exit values are always strings. Heavy orchestration belongs in a job or pipeline, not in `notebook.run`.

RELATED CONCEPTS

Lakeflow Jobs, Notebooks and Spark Job Definitions, Spark Connect, Unity Catalog, Fabric Environments, High concurrency mode.

Operating in production.

LAYER FILES TABLE STREAM COMPUTE GOVERN PIPELINE SERVE OPERATE

Scheduled Spark work needs four things beyond the code: a record of what ran and what it cost, a place to see why a run was slow or failed, alerts that fire before downstream users notice, and tests that stop a broken change reaching production. Both platforms provide the first three. The fourth is ordinary PySpark practice wired into CI.

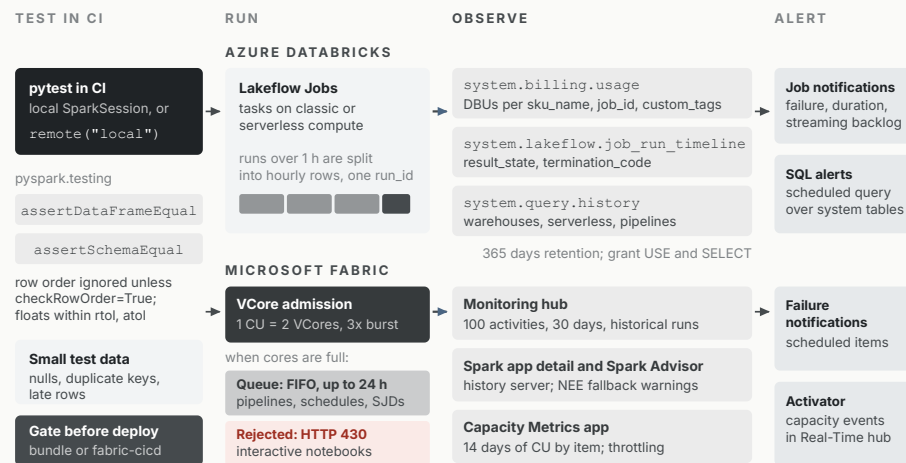


FIG. 12.3 Tests gate every deploy; each run is then admitted, recorded and alerted on. Databricks answers what ran and what it cost from system tables in SQL; Fabric admits Spark by VCores, queues scheduled work and rejects interactive work with HTTP 430, and reports through the Monitoring hub and Capacity Metrics app.

ON DATABRICKS

Query the **system tables** in Unity Catalog with SQL. `system.billing.usage` records DBUs per `sku_name`, with `usage_metadata.job_id` and `custom_tags` for attribution. `system.lakeflow.job_run_timeline` and `job_task_run_timeline` hold `result_state` and `termination_code`; runs longer than an hour are split into hourly rows. `system.query.history` covers SQL warehouses, serverless compute and Lakeflow pipelines, not classic clusters. Retention is 365 days, and an admin has to grant `USE` and `SELECT` on the system schemas first.

ON FABRIC

Open the **Monitoring hub**: it lists up to 100 activities from the last 30 days, with historical runs per item. Each Spark application has a detail page and the Spark history server. The **Spark Advisor** writes info, warning and error advice into notebook output, including NEE fallback to JVM execution. The **Capacity Metrics app** shows 14 days of CU use by item and operation, plus throttling.

WHEN FABRIC RUNS OUT OF CORES

Fabric admits Spark jobs by VCores: 1 CU = 2 VCores, bursting to 3x, so an F64 has 128 and can reach 384. When cores run out, pipeline, scheduled and Spark job definition runs queue first in, first out for up to 24 hours. The queue has a per-SKU limit: 4 on F2 and F4, 64 on F64, none on trial capacities. Interactive notebooks and notebooks submitted through the public API fail at once with HTTP 430 `TooManyRequestsForCapacity`. Do not retry those in a loop; schedule them where they can queue. Capacity throttling is a separate layer: while the capacity is throttled, new Spark jobs are refused outright, not queued. Throttling does not apply to Spark under On-demand billing for Spark.

SETTING UP ALERTS

Lakeflow Jobs notify on start, success, failure, duration warning and streaming backlog, to email, Slack, Teams, PagerDuty or webhooks. Databricks SQL alerts run a query on a schedule, such as one over system tables. Fabric has failure notifications for scheduled items, and Activator alerts on capacity events in Real-Time hub. Do not alert only on failure, or a job that doubles in duration goes unnoticed.

TESTING BEFORE DEPLOY

Start testing with the first scheduled job. Write transformations as functions that take and return DataFrames, and run them under pytest against a local `SparkSession`, or `SparkSession.builder.remote("local")` for Spark Connect. `pyspark.testing` (Spark 3.5 and later) gives you `assertDataFrameEqual`, which ignores row order unless `checkRowOrder=True` and compares floats within `rtol` and `atol`, and `assertSchemaEqual`. Build small inline test data with nulls, duplicate keys and late rows; do not test against production tables. Run the suite in CI before the bundle or `fabric-cicd` deploy.

RELATED CONCEPTS

The Spark UI, Lakeflow Jobs, Capacity Units and Bursting, On-demand billing for Spark, Spark Connect, Unity Catalog.

Diagnose by Symptom.

LAYER FILES TABLE STREAM COMPUTE GOVERN PIPELINE SERVE OPERATE

Find the symptom you see. Each entry gives the likely cause, where to look to confirm it, the fix, and a platform note where classic Databricks, serverless and Fabric differ. Look in one of five layers: the Spark UI (Stages, Executors and SQL tabs; the query profile on Databricks serverless), the plan from `explain(True)`, the Delta log through `DESCRIBE HISTORY` and `DESCRIBE DETAIL`, the platform and capacity monitors, or the exception itself (error class and stack trace). Confirm the cause before changing a setting, change one thing at a time, and rerun.

WHERE YOU LOOK	SIGNAL	SYMPTOMS CONFIRMED THERE
		colour shows the effect: job slow job fails
Spark UI Stages, Executors tabs		one task far longer spill non-zero thousands of tiny tasks executor OOM, high GC heartbeat timeout
Query plan <code>explain(True)</code> , SQL tab		reads too much data sort-merge, not broadcast Photon or NEE fallback broadcast timeout
Exception error class, stack trace		CAST_INVALID_INPUT driver OOM, maxResultSize multiple MERGE matches DELTA_METADATA_MISMATCH
Delta log and stream <code>DESCRIBE HISTORY</code> , progress		small files ConcurrentAppendException state keeps growing reprocessed from batch 0
Platform monitors Metrics app, Monitoring hub		throttled: delayed or rejected HTTP 430 slow session start

FIG. 13.1 The twenty symptoms grouped by the layer where each is confirmed: the Spark UI, the query plan, the exception text, the Delta log and stream progress, or the platform monitors. Tinted chips mark symptoms that make a job slow; warning-coloured chips mark those that make it fail.

One task runs far longer than the rest

Cause: data skew; one key holds far more rows than the others. **Look:** Stages tab task summary: max duration and shuffle read far above the median. **Fix:** leave AQE skew join handling on (`spark.sql.adaptive.skewJoin.enabled`, default true), filter null keys early, or salt the hot key. **Platform:** serverless does not accept AQE skew settings, so salt in code. See: Data Skew and Salting; Adaptive Query Execution (AQE).

All tasks slow and the Spill columns are non-zero

Cause: execution memory runs out, so sort, join and aggregation buffers spill to local disk and are read back. **Look:** Stages tab Spill (Memory) and Spill (Disk); on serverless, `spilled_local_bytes` in `system.query.history`. **Fix:** smaller partitions (raise `spark.sql.shuffle.partitions`, or `auto` on Databricks) and more memory per core. **Platform:** on Fabric, pick a larger node size or set `executorMemory` with `%%configure`. See: Spill; Unified Memory.

Executor OutOfMemoryError or high GC time

Two cases. **Heap too small** per task: GC time climbs, then `Java heap space`. Give each task more memory (larger executors or fewer cores each), more partitions, or unpersist cached data. **Heap very large:** rare but long full collections; run more, smaller executors and tune G1. **Look:** Stages tab GC Time; Executors tab. **Platform:** serverless exposes no memory settings. See: GC Pressure; On-Heap Memory.

Driver OutOfMemoryError or maxResultSize exceeded

Cause: `collect()`, `toPandas()` or an oversized broadcast pulls data into the driver; results above `spark.driver.maxResultSize` (default 1g) abort the job. **Look:** the action named in the stack trace. **Fix:** aggregate or `limit()` first, or write to a table instead of collecting. **Platform:** serverless notebooks: switch the environment to High memory (32 GB, REPL memory only); Fabric: `driverMemory` in `%%configure`. See: Driver, Executor, Cores, Slots; Broadcast join failure modes.

Query reads far more data than the filter suggests

Cause: predicate pushdown, partition pruning or data skipping is not firing: the filter wraps the column in a UDF or function, compares mismatched types, or filters a derived column. **Look:** in `explain(True)`, `PushedFilters` and `PartitionFilters` on the scan, and files pruned in the SQL tab. **Fix:** filter raw columns against literals of the same type, and cluster the table on common filter columns. See: Predicate Pushdown; Partition Pruning; Data Skipping.

Sort-merge join where a broadcast was expected

Cause: missing or stale statistics push the small side's estimated size over `spark.sql.autoBroadcastJoinThreshold` (10 MB). **Look:** `explain()` shows `SortMergeJoin`. **Fix:** `ANALYZE TABLE ... COMPUTE STATISTICS`, a `broadcast()` hint, and AQE, which switches to broadcast at runtime when the shuffled side proves small. **Platform:** Databricks uses a separate runtime threshold, `spark.databricks.adaptive.autoBroadcastJoinThreshold` (30 MB). See: Broadcast Hash Join; Physical Plan and Join Strategy.

Could not execute broadcast in 300 secs

Cause: running the build-side query, collecting it and building the relation took longer than `spark.sql.broadcastTimeout` (300 s). Usually that is an expensive upstream subquery or a large table forced by a hint, not the network. **Look:** SQL tab, `BroadcastExchange` time and size. **Fix:** drop the hint so the join runs as sort-merge, or materialise the small side first. Raise the timeout only for genuinely small tables. **Platform:** serverless rejects the setting. See: Broadcast join failure modes; Broadcast Hash Join.

Executors lost with heartbeat timed out

Cause: an executor stayed silent past `spark.network.timeout` (120 s), from long GC pauses, an evicted spot VM, or a local disk full of spill. **Look:** executor logs for GC pauses; `ExecutorLostFailure` in failed tasks. **Fix:** remove the cause; as a stopgap raise `spark.network.timeout`, for example to 300s, and keep `spark.executor.heartbeatInterval` (10 s) well below it. **Platform:** serverless accepts neither setting. See: GC Pressure; Spill; Instance Pools and Spot VMs.

Thousands of tasks that each run for milliseconds

Cause: too many partitions: 200 shuffle partitions on small data with no AQE coalescing, many small files, or `repartition(n)` set too high. **Look:** Stages tab task count and shuffle read per task in KB. **Fix:** keep AQE coalescing on (64 MB advisory size), lower `spark.sql.shuffle.partitions`, compact the input. **Platform:** Databricks accepts `spark.sql.shuffle.partitions=auto`, one of the few settings serverless allows. See: Repartition, Coalesce, and RepartitionByRange; Adaptive Query Execution (AQE).

Reads are slow although the table is not large

Cause: small files from frequent appends, short streaming batches or over-partitioning. **Look:** `numFiles` against `sizeInBytes` in `DESCRIBE DETAIL`, files read per scan in the SQL tab. **Fix:** `OPTIMIZE`, Auto Compaction and Optimised Writes, or Liquid Clustering. **Platform:** Databricks always compacts after `MERGE`, `UPDATE` and `DELETE`, and Predictive Optimization runs `OPTIMIZE` on Unity Catalog managed tables. Fabric turns on Auto Compaction with `delta.autoOptimize.autoCompact`. See: Small-file Problem; `OPTIMIZE` and Bin-packing; Delta compaction strategies.

Photon or NEE is on but the query is no faster

Cause: part of the plan runs on the JVM: UDFs, RDD code or stateful streaming (Photon); JSON, XML or Structured Streaming (NEE). **Look:** the Databricks SQL/DataFrame tab draws Photon operators orange, the rest blue. On Fabric, native operators end in `Transformer`, the Gluten SQL / DataFrame tab counts fallbacks, and Spark Advisor warns in the cell. **Fix:** built-in functions instead of UDFs. See: Photon; NEE (Native Execution Engine); PySpark UDF variants.

CAST_INVALID_INPUT or DIVIDE_BY_ZERO after an upgrade

Cause: ANSI mode, the default since Spark 4.0, raises errors for bad casts, overflow and division by zero instead of returning `NULL`. **Look:** the error class and expression in the exception. **Fix:** clean the input, or use `try_cast` or `try_divide` where `NULL` is intended; `spark.sql.ansi.enabled=false` only as a short bridge. **Platform:** on by default in Databricks Runtime 17.0 and above and Fabric Runtime 2.0, not in Runtime 1.3. See: ANSI mode by default.

AnalysisException: schema mismatch on write

Cause: the DataFrame's columns or types differ from the Delta table's (`DELTA_METADATA_MISMATCH`, subclass `SCHEMA_MISMATCH`). **Look:** compare `df.printSchema()` with `DESCRIBE TABLE`; `DESCRIBE HISTORY` for the last schema change. **Fix:** for intended new columns, `mergeSchema` on that write or `MERGE WITH SCHEMA EVOLUTION`. For a deliberate replacement, `overwriteSchema` with `overwrite` mode. Otherwise fix the cast upstream. Avoid the session-wide `autoMerge` setting. See: Schema Enforcement and Evolution; Writing data correctly.

ConcurrentAppendException or another write conflict

Cause: a concurrent commit added files that your operation read, in the same partition or anywhere in an unpartitioned table. **Look:** `DESCRIBE HISTORY` for the conflicting operation. **Fix:** explicit partition predicates in `MERGE`, `UPDATE` and `DELETE` conditions, or serialise writers with task dependencies. **Platform:** Databricks row-level concurrency (14.3 LTS and above, deletion vectors, no partitions) resolves conflicts on different rows automatically. See: Optimistic Concurrency; Deletion Vectors; Liquid Clustering.

MERGE fails: multiple source rows matched one target row

Cause: duplicate keys in the source make an update or delete ambiguous (`DELTA_MULTIPLE_SOURCE_ROW_MATCHING_TARGET_ROW_IN_MERGE`); insert-only merges are exempt. **Look:** count source rows per merge key. **Fix:** deduplicate before the merge; keep the latest row per key with `row_number()` over a window. **Platform:** Databricks Runtime 15.4 LTS and below check only the `ON` clause for multiple matches, not `WHEN MATCHED` conditions. See: `MERGE` / Upsert; Writing data correctly.

Streaming batches slow down and state keeps growing

Cause: unbounded state from aggregations, `dropDuplicates` or stream-stream joins with no watermark, or a watermark far wider than needed. **Look:** `numRowsTotal` and `memoryUsedBytes` rising under `stateOperators` in query progress; the State Data Source reads state directly. **Fix:** add `withWatermark` on event time and use `dropDuplicatesWithinWatermark`. **Platform:** Databricks recommends the RocksDB state store with changelog checkpointing (13.3 LTS and above). See: Watermarks and State; transformWithState and State Data Source; Streaming joins.

Stream reprocesses everything and the sink has duplicates

Cause: the checkpoint was deleted or `checkpointLocation` changed, so the next run starts from scratch. **Look:** `offsets/` and `commits/` under the checkpoint; batch 0 in query progress. **Fix:** restore the checkpoint. Failing that, rebuild the sink, or start a Delta source at `startingVersion`, then `MERGE` on a business key. **Platform:** a new Auto Loader checkpoint reingests existing files unless `cloudFiles.includeExistingFiles` is false. See: Streaming checkpoints; Output Modes and Exactly-once; Auto Loader.

Fabric jobs are delayed or rejected across a capacity

Cause: smoothing spreads CU use forward, interactive over at least 5 minutes and background over 24 hours. Once future usage passes 10 minutes, interactive jobs wait 20 s; past 60 minutes they are rejected; past 24 hours every request is. **Look:** Capacity Metrics app throttling charts. **Fix:** shrink or move heavy jobs, scale up, or put Spark on On-demand billing for Spark. See: Capacity Units and Bursting; On-demand billing for Spark.

Fabric Spark job fails at once with HTTP 430

Cause: `TooManyRequestsForCapacity` comes from Spark admission, not throttling. Running sessions hold every VCore (2 per CU, bursting to 3x). **Look:** Monitoring hub for active sessions, idle notebooks included. **Fix:** stop idle sessions, disable job-level bursting, or run the work as a pipeline, schedule or Spark job definition. Those queue first in, first out for up to 24 hours, up to the SKU's queue limit. See: Capacity Units and Bursting; Operating in production.

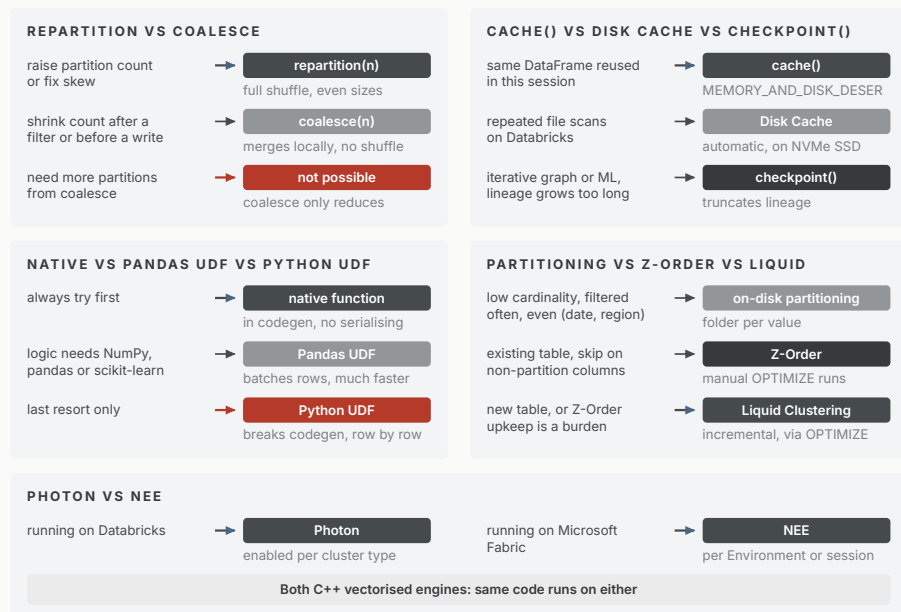
Session or job takes minutes to start

Cause: no pre-warmed compute. **Platform:** Fabric starter pools take 5 to 10 seconds, custom pools about 3 minutes; libraries add 30 s to 5 min; managed VNet or private links skip starter pools (2 to 5 minutes). Databricks classic clusters acquire VMs; standard-mode serverless jobs take 4 to 6 minutes. **Fix:** slimmer environments, instance pools, performance optimized mode, or Fabric custom live pools (preview). See: Starter vs Custom Pools; Classic vs serverless compute.

Which Tool When.

LAYER FILES TABLE STREAM **COMPUTE** GOVERN PIPELINE SERVE OPERATE

Each entry gives the rule for choosing between tools or approaches that are easy to confuse. Treat the rule as your default; the trade-off says when to deviate. For choices that change the physical plan (joins, caching, repartitioning), confirm the expected strategy in `explain(True)`. For choices that change data layout (partitioning, clustering, ordering), run a small scan and check files pruned on the scan node in the Spark UI SQL tab.



Check plan choices in `explain(True)`; check layout choices with files pruned on the scan in the SQL tab.

FIG. 13.2 Five commonly confused choices, each reduced to its decision rule: match your situation on the left of a card to the tool on the right; amber marks the option to avoid or the move that is not possible.

REPARTITION VS COALESCE

repartition(n) does a full shuffle into exactly n evenly sized partitions. Use it to raise the partition count or fix skew. **coalesce(n)** merges partitions on the same executor with no shuffle. Use it only to reduce the count, for example after a filter that leaves mostly empty partitions, or before a write to get fewer output files. It cannot increase partitions.

cache() vs Disk Cache vs checkpoint()

cache(): explicit; in PySpark it uses `MEMORY_AND_DISK_DESER` (memory first, spilling to disk). Use it when you know a DataFrame will be reused in the same session. **Disk Cache**: automatic and transparent, on NVMe SSD. Nothing to do; repeated file scans on Databricks benefit automatically. **checkpoint()**: truncates the lineage graph and writes to reliable storage. Use it only for iterative algorithms (graph, ML) where long lineage causes stack overflows or planning overhead.

PHOTON VS NEE

Photon: Databricks only, a C++ vectorised engine, enabled per cluster type, most effective on SQL and ETL workloads. **NEE**: Fabric only, a C++ vectorised engine, enabled per Environment or session with `spark.native.enabled`, with equivalent benefit on scan and aggregation workloads. They are platform equivalents; code targeting one runs on the other without modification.

UDF vs Pandas UDF vs native function

Native Spark function: always first choice; stays inside codegen, no serialisation overhead. **Pandas UDF**: use it when the logic needs NumPy, pandas or scikit-learn. Batches rows across the process boundary; much faster than a row-at-a-time UDF.

Python UDF: last resort. It breaks codegen and serialises rows one at a time across the JVM/Python boundary unless Arrow optimisation is on (`useArrow=True`, the default from Spark 4.2).

Z-Order vs Liquid Clustering vs on-disk partitioning

On-disk partitioning: for low-cardinality columns (date, region) with very frequent filter use and even distribution. **Z-Order**: on existing tables, to improve skipping on non-partition columns; needs periodic manual OPTIMIZE runs. **Liquid Clustering**: for new tables, or where Z-Order maintenance is a burden. Incremental (large writes are clustered on write, the rest by `OPTIMIZE` or Predictive Optimization) with no partition column constraint.

Where each feature lives.

OSS = open-source Apache Spark and Delta Lake (vanilla cluster, EMR, Dataproc, self-hosted), without the Databricks or Fabric managed runtime. Coverage as of September 2026: Databricks Runtime 18 LTS and later, Fabric Runtime 2.0, and Apache Spark 4.1 / Delta Lake 4.x, with Spark 4.2 (July 2026) noted where it changes an answer. **Y** = generally available; **P** = preview; **-** = not available.

DEFAULTS THAT CHANGED RECENTLY the full configuration list lives in PySpark Cheat Sheet 4.0

<code>spark.sql.ansi.enabled</code>	<code>true</code> (Spark 4.x, DBR 17.0+)	Leave on. Invalid casts, integer overflow, and division by zero raise errors instead of returning NULL or wrapping; use <code>try_cast</code> and <code>try_divide</code> where bad input is expected. Set to false only as a short-term migration aid.
<code>spark.sql.execution.pythonUDF.arrow.enabled</code>	<code>false</code> (4.1); <code>true</code> from Spark 4.2	Rows move to Python in Arrow batches, not pickled rows; per UDF with <code>useArrow=True</code> .
<code>spark.sql.parquet.vorder.default</code>	<code>false</code> (Spark writes, new Fabric workspaces)	Fabric only. Set true, or set <code>delta.parquet.vorder.enabled</code> per table, for tables read through Direct Lake or other read-heavy paths; leave off for ingestion-heavy tables. The <code>readHeavyForPBI</code> resource profile turns it on.
<code>spark.native.enabled</code>	<code>off</code> unless enabled in the Environment	Fabric only. Turns on NEE for the session; set false in a single cell to isolate a query that misbehaves on the native path.

FEATURE	DATABRICKS	FABRIC	OSS	NOTES
Catalyst, Codegen, AQE	Y	Y	Y	Core Spark; the platforms tune some defaults.
Photon	Y	-	-	Native execution engine specific to Databricks.
NEE (Native Execution Engine)	-	Y	-	Fabric Spark's native engine; enable per Environment or with <code>spark.native.enabled</code> .
Tungsten / JVM Codegen	Y	Y	Y	Fallback when Photon/NEE doesn't support an operator.
Delta Lake	Y	Y	Y	Open standard; Fabric ships the open-source release (4.2), Databricks its own build.
Liquid Clustering	Y (DBR 15.4 LTS+)	Y	Y (Delta 3.2+)	Preview from DBR 13.3 and Delta 3.1. On Databricks, large writes are clustered on write and the rest by <code>OPTIMIZE</code> or Predictive Optimization; open-source Delta clusters only on <code>OPTIMIZE</code> .
Z-Order	Y	Y	Y	Manual Z-Order via <code>OPTIMIZE ZORDER BY</code> .
V-Order	-	Y	-	Fabric write-time sort for VertiPaq; off for Spark writes in new workspaces.
Deletion Vectors	Y	Y	Y	Open Delta 3.0+; default in DBR 14+.
Predictive Optimization	Y	-	-	Databricks maintenance for UC managed tables; on by default for new accounts, and for existing ones by August 2026.
Auto Loader	Y	-	-	Databricks-only <code>cloudFiles</code> source: file tracking, schema inference and evolution. On Fabric, use a Structured Streaming file source or a storage event trigger.

Where each feature lives

CONTINUED

Lakeflow pipelines (DLT) / Spark Declarative Pipelines	Y	-	Y (4.1)	Lakeflow pipelines (formerly DLT) extend open-source SDP; expectations and SCD Type 2 AUTO CDC are Databricks-only; Spark 4.2 adds Auto CDC for SCD Type 1. Fabric documentation does not list SDP.
Materialised Views	Y	Y	Y (4.1, SDP)	Lakeflow and Databricks SQL MVs; open source inside SDP; Fabric materialized lake views (PySpark in preview).
Unity Catalog	Y	-	-	Databricks governance. Fabric uses OneLake security and workspace roles.
Spark Connect / Databricks Connect	Y (DBR 13+)	-	Y (3.4+)	Decoupled client-server architecture.
Serverless compute	Y	Y	-	Databricks serverless notebooks, jobs, pipelines and SQL warehouses. Fabric Spark has no cluster to manage: starter pools and On-demand billing give the serverless experience, though you still pick a starter or custom pool.
OneLake	-	Y	-	Fabric unified storage layer.
Change Data Feed	Y	Y	Y	Open Delta 2.0+.
Time Travel	Y	Y	Y	Read by version or timestamp; open Delta.
VARIANT type	Y (DBR 15.3+)	Y (Spark)	Y (Spark 4.0+)	Spark 4.1 adds shredding. Delta tables need DBR 15.4 LTS+. Fabric: experimental in Delta tables, Spark only.
Real-time mode	Y (DBR 16.4 LTS+)	-	Y (4.1)	Sub-second streaming trigger on classic compute with Photon off; stream-stream joins need DBR 18 LTS. OSS 4.1 is Scala and stateless only; Spark 4.2 adds PySpark. Not documented for Fabric.
Automatic Liquid Clustering	Y (DBR 15.4 LTS+)	-	-	CLUSTER BY AUTO; Predictive Optimization picks keys on Unity Catalog managed tables.
Resource profiles	-	Y	-	Preset Spark configs: <code>writeHeavy</code> (new-workspace default), <code>readHeavyForSpark</code> , <code>readHeavyForPBI</code> .
High concurrency mode	-	Y	-	Notebooks and pipeline activities share one Spark session (5 by default, up to 50).
On-demand billing for Spark	-	Y	-	Pay-as-you-go Spark (formerly Autoscale Billing) billed outside capacity CUs; no bursting or smoothing.
Lakehouse Federation	Y	-	-	Databricks federated queries; Fabric uses shortcuts and mirroring instead.
Delta Sharing	Y	Y (recipient)	Y	Open sharing protocol; Fabric reads shares through a Data Factory connector but cannot provide them.

Abbreviations and core terms.

SPARK ENGINE

Catalyst

Spark's query optimiser; owns parsing, analysis, logical optimisation, and physical planning.

AQE

Adaptive Query Execution; runtime replanning that adjusts shuffle partitions, join strategies, and skew handling based on actual statistics.

Tungsten

Spark's memory-management and code-generation layer; foundation for JVM execution acceleration.

Codegen

Whole-Stage Code Generation; compiles operator chains within a stage into one JVM function for tight-loop execution.

Photon

Databricks' native C++ vectorised execution engine; replaces JVM execution for supported operators.

NEE

Native Execution Engine; the Fabric equivalent of Photon.

DAG

Directed Acyclic Graph; the physical execution plan as a graph of stages connected by shuffles.

CBO

Cost-Based Optimization; uses table statistics to choose join strategies and order.

UDF

User-Defined Function; custom code invoked by Spark. Multiple variants: Python, Pandas, Pandas Function API.

ANSI mode

SQL standard semantics, on by default in Spark 4; invalid casts, overflow, and division by zero raise errors instead of returning NULL.

VARIANT

Spark 4 data type that stores semi-structured values such as JSON in a binary form that is queried without a fixed schema.

RTM

Real-time mode; Structured Streaming trigger in Spark 4.1 with long-running batches and streaming shuffle for sub-second latency.

Spark Connect

Client and server protocol, introduced in Spark 3.4, that sends unresolved DataFrame plans from a thin client to a remote Spark driver; Databricks standard access mode and serverless compute run on it.

REPL

Read-eval-print loop; the interactive language process behind a notebook. On Databricks serverless its memory (16 GB standard, 32 GB high) holds driver-side Python objects such as `toPandas()` results.

EXECUTION AND SHUFFLE

Driver

JVM process running the application's main; builds the plan, schedules tasks on executors, receives collected results and builds broadcast relations. It does not process partitions itself.

Executor

JVM process on a worker node that runs tasks. Has fixed cores and memory.

Shuffle

Network transfer of data between executors. Triggered by wide transformations (joins, groupBys, repartitions).

Spill

Write of execution memory to local disk when in-memory buffers are exhausted; degrades performance significantly.

Skew

Uneven row distribution across partitions; causes one task to run far longer than its siblings.

BHJ / SMJ / SHJ

Broadcast Hash Join / Sort-Merge Join / Shuffle Hash Join; the three main join strategies.

DPP

Dynamic Partition Pruning; runtime partition elimination from join-build-side filters.

DELTA AND LAKEHOUSE

ACID

Atomic, Consistent, Isolated, Durable; transactional guarantees that Delta provides over Parquet.

Transaction Log

The append-only log of JSON commit files in `_delta_log/` that drives Delta semantics.

MERGE

SQL operation performing insert + update + delete in one atomic pass; the standard upsert mechanism.

CDF

Change Data Feed; row-level change records (insert, update, delete) emitted by Delta for downstream propagation.

SCD Type 2

Slowly Changing Dimension Type 2; pattern that preserves history by closing old rows and inserting new versions.

Z-Order

Multi-dimensional clustering of data files using a space-filling curve; tightens column min/max stats for skipping.

Liquid Clustering

Incremental clustering that replaces partitioning + Z-Order with a managed, on-demand reclustering process.

V-Order

Fabric-specific write-time sort that optimises Parquet files for VertiPaq column loading in Direct Lake.

UniForm

Universal Format; Delta table setting that also writes Iceberg metadata so Iceberg clients can read the same Parquet files.

Row group

Horizontal slice of a Parquet file holding one column chunk per column; per-chunk min/max statistics let readers skip row groups.

Protocol versions / table features

Delta's `minReaderVersion` and `minWriterVersion` declare what a client must support; from reader version 3 and writer version 7, the table lists individual features such as deletion vectors instead.

PLATFORMS

OSS

Open-source Apache Spark and Delta Lake; the engines without the Databricks or Fabric managed runtime. Runs on vanilla clusters, EMR, Dataproc, or self-hosted.

DBR

Databricks Runtime; the specific Spark + Delta + library bundle versioned on Databricks.

DLT

Former name of Lakeflow pipelines.

SJD

Spark Job Definition; Fabric concept for non-notebook Spark jobs.

UC

Unity Catalog; Databricks' three-level namespace (catalog.schema.table) for governance and metadata.

MV

Materialised View; precomputed query result stored as a managed Delta table, refreshed incrementally where possible.

OneLake

Fabric's unified storage layer; one logical lake per tenant, with multi-format access for all Fabric workloads.

Direct Lake

Power BI / Fabric connection mode that reads Delta tables directly into VertiPaq without import; framed against a Delta version.

CU

Capacity Unit; Fabric's compute currency. Workloads in every workspace assigned to a capacity consume that capacity's CUs.

Lakeflow
Databricks' data engineering family: Lakeflow Connect (ingestion connectors), Lakeflow pipelines (formerly DLT) and Lakeflow Jobs (formerly Workflows).
SDP
Spark Declarative Pipelines; open-source declarative pipeline framework in Spark 4.1 that Lakeflow pipelines extend.
Access mode
Databricks compute security setting: standard (formerly shared, multi-user, runs on Spark Connect) or dedicated (formerly single user).
MLV
Materialized lake view; Fabric's Spark SQL materialised view over lakehouse tables with dependency-aware refresh.

Resource profile
Fabric preset of Spark settings (writeHeavy, readHeavyForSpark, readHeavyForPBI); writeHeavy is the new-workspace default.
VertiPaq
In-memory columnar engine that answers DAX queries for Power BI semantic models in Import and Direct Lake mode.
Semantic model
Power BI model (formerly a dataset) of tables, relationships and DAX measures that reports query; storage modes include Import, DirectQuery and Direct Lake.
DBU
Databricks Unit; normalised unit of processing power used for pricing. Consumption depends on the compute and data processed; recorded per SKU in <code>system.billing.usage</code> .

SKU
Fabric capacity size, F2 upward, where the number is the capacity's CUs (F64 = 64 CUs); bought in Azure, pay-as-you-go billed per second or reserved yearly.
NCC
Network connectivity configuration; account-level, regional Databricks object, attached to workspaces, that manages private endpoints and firewall access for serverless compute.
Entra ID
Microsoft Entra ID (formerly Azure Active Directory); Microsoft's cloud identity service for users, groups and service principals in Azure Databricks and Fabric.

Lakeguard
Databricks isolation technology that separates user code from the Spark engine and from other users on standard compute, serverless and SQL warehouses, so row filters and column masks are enforced.
Volume
Unity Catalog object that governs non-tabular files in cloud storage, managed or external, addressed as <code>/Volumes/catalog/schema/volume/path</code> .
LTS
Long-term support; a Databricks Runtime release that receives major stability and security fixes for three years after GA.
Declarative Automation Bundles
Databricks deployment tool (formerly Databricks Asset Bundles) that declares jobs, pipelines and other resources as source files in <code>databricks.yml</code> and deploys them per target.

CONCLUSION

How seniors, leads and architects think.

If you can picture the plan, the shuffle and the log, most production problems point to their own cause.

- 01 Native functions beat hand-rolled UDFs nearly every time.
- 02 Explicit schemas catch bad data before it spreads.
- 03 AQE handles most of the shuffle tuning people used to do by hand.
- 04 Delta lets you undo a bad write.

Version 4.0 continues the PySpark Cheat Sheet (1.0 through 3.0) and replaces **PySpark Cheat Sheet 3.0** with two companions, this handbook and the **PySpark Cheat Sheet 4.0** for code patterns, both scoped to Fabric and Databricks.

FREE

Sanjay Chandra

The Databricks + Fabric guy on LinkedIn

WEBSITE

www.ssanjaychandra.com

GITHUB

github.com/ssanjaychandra123

LINKEDIN

linkedin.com/in/ssanjaychandra

EMAIL

ssanjaychandra@outlook.com

This reference owes its shape to the PySpark, Delta Lake, Databricks and Microsoft Fabric communities whose documentation, blogs, conference talks and source code make the platform legible. Specific gratitude to the engineers and writers maintaining Apache Spark and Delta Lake, and to the product teams at Databricks and Microsoft whose technical writing was a continuous reference.

Claude (Anthropic) was used as a writing assistant while compiling, reviewing and red-teaming this reference. Errors are mine alone; corrections via the channels above are welcome.